DATA STRUCTURES – Trees & Hashing

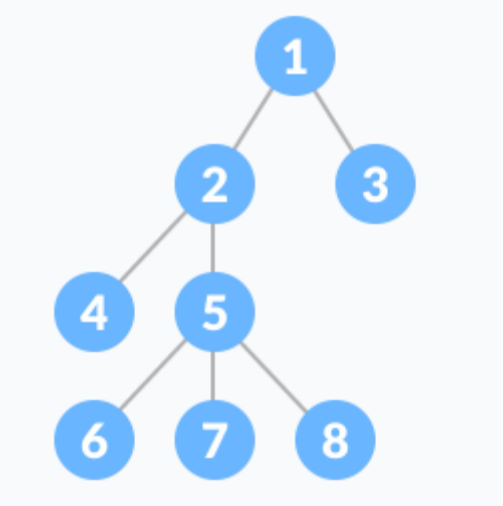
UNIT-4&5

Dr. SELVA KUMAR S
PROFESSOR & ASSOCIATE HEAD
B.M.S. COLLEGE OF ENGINEERING



Trees - Introduction

- A tree is a nonlinear hierarchical data structure that consists of nodes connected by edges.
- Different tree data structures allow quicker and easier access to the data as it is a non-linear data structure.



Properties of Trees

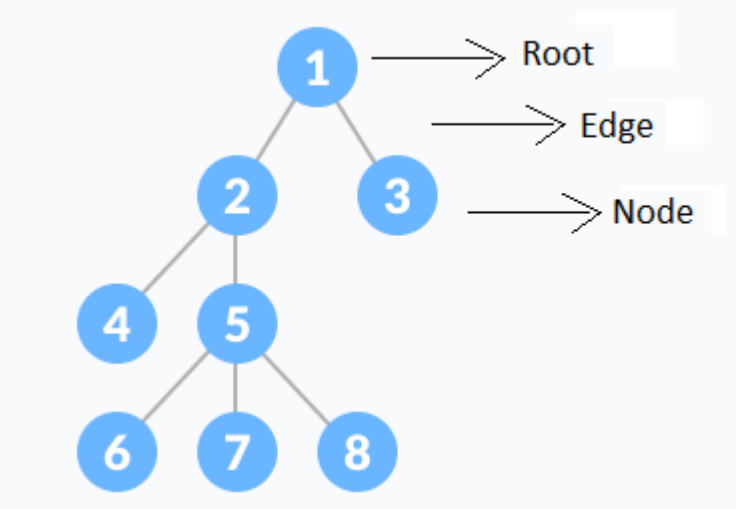
- There is one and only one path between every pair of vertices in a tree.
- A tree with n vertices has n-1 edges.
- A graph is a tree if and if only if it is minimally connected.
- Any connected graph with n vertices and n-1 edges is a tree.

Tree Applications

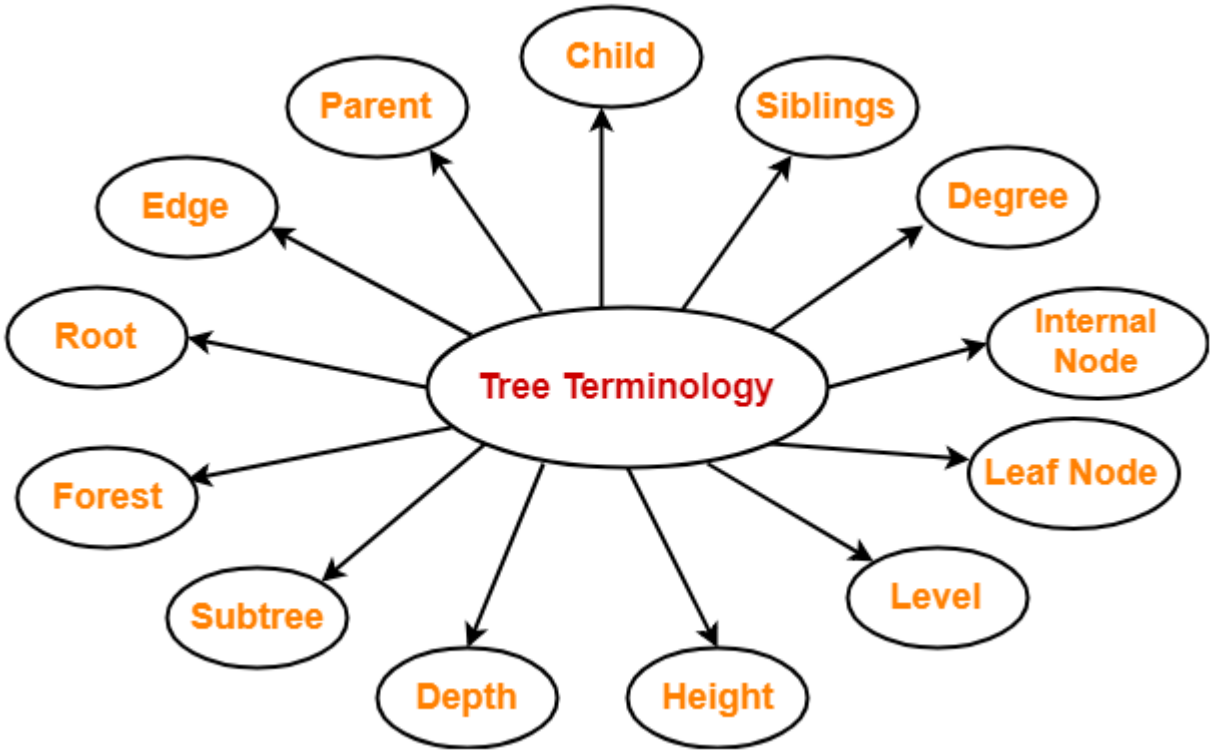
- Binary Search Trees(BSTs) are used to quickly check whether an element is present in a set or not.
- Heap is a kind of tree that is used for heap sort.
- A modified version of a tree called Tries is used in modern routers to store routing information.
- Most popular databases use B-Trees and T-Trees, which are variants of the tree structure we learned above to store their data
- Compilers use a syntax tree to validate the syntax of every program you write.

Tree terminologies

- **Node** - A node is an entity that contains a key or value and pointers to its child nodes.
  - The last nodes of each path are called **leaf nodes or external nodes** that do not contain a link/pointer to child nodes.
  - The node having at least a child node is called an **internal node**.
- **Edge -**It is the link between any two nodes.
- **Root -** It is the topmost node of a tree.

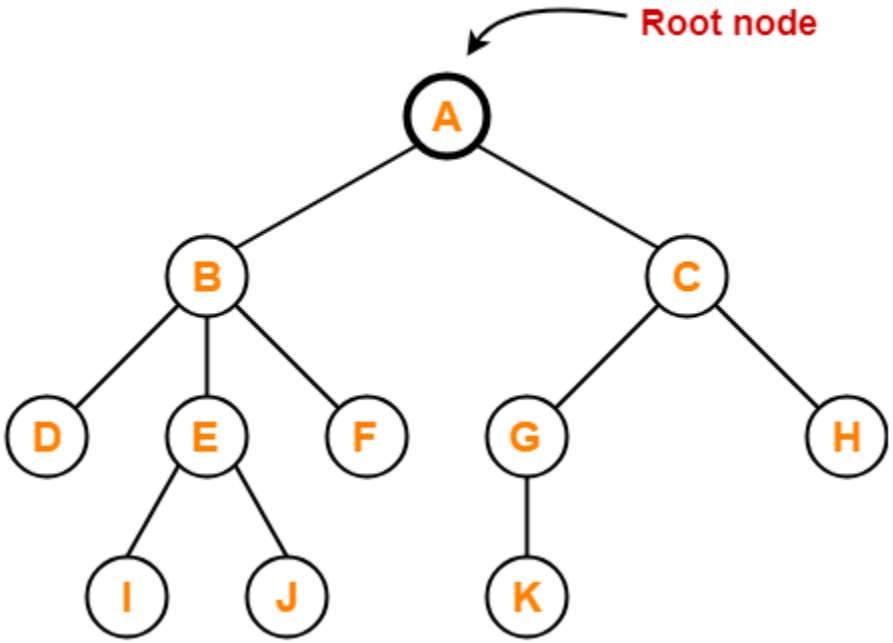


Tree terminologies



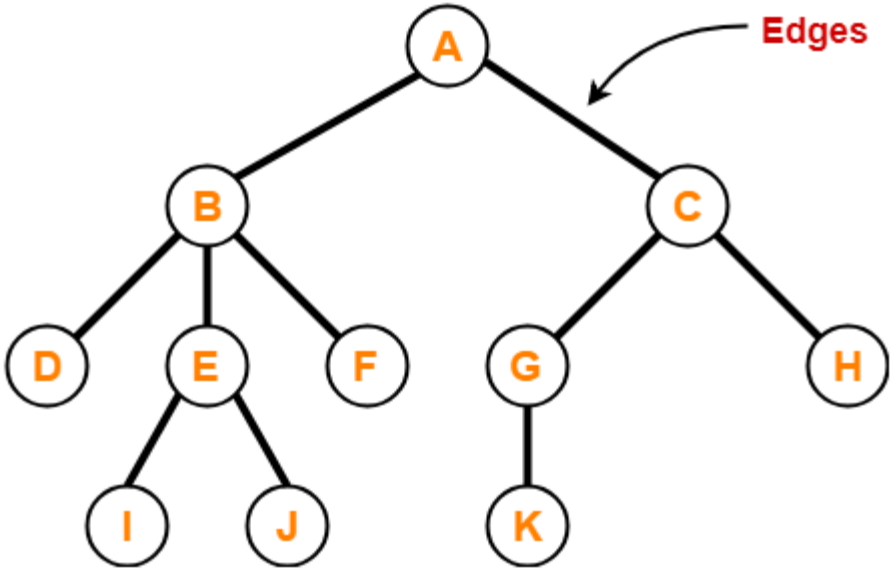
Root

- The first node from where the tree originates is called as a **root node**.
- In any tree, there must be only one root node.
- We can never have multiple root nodes in a tree data structure.



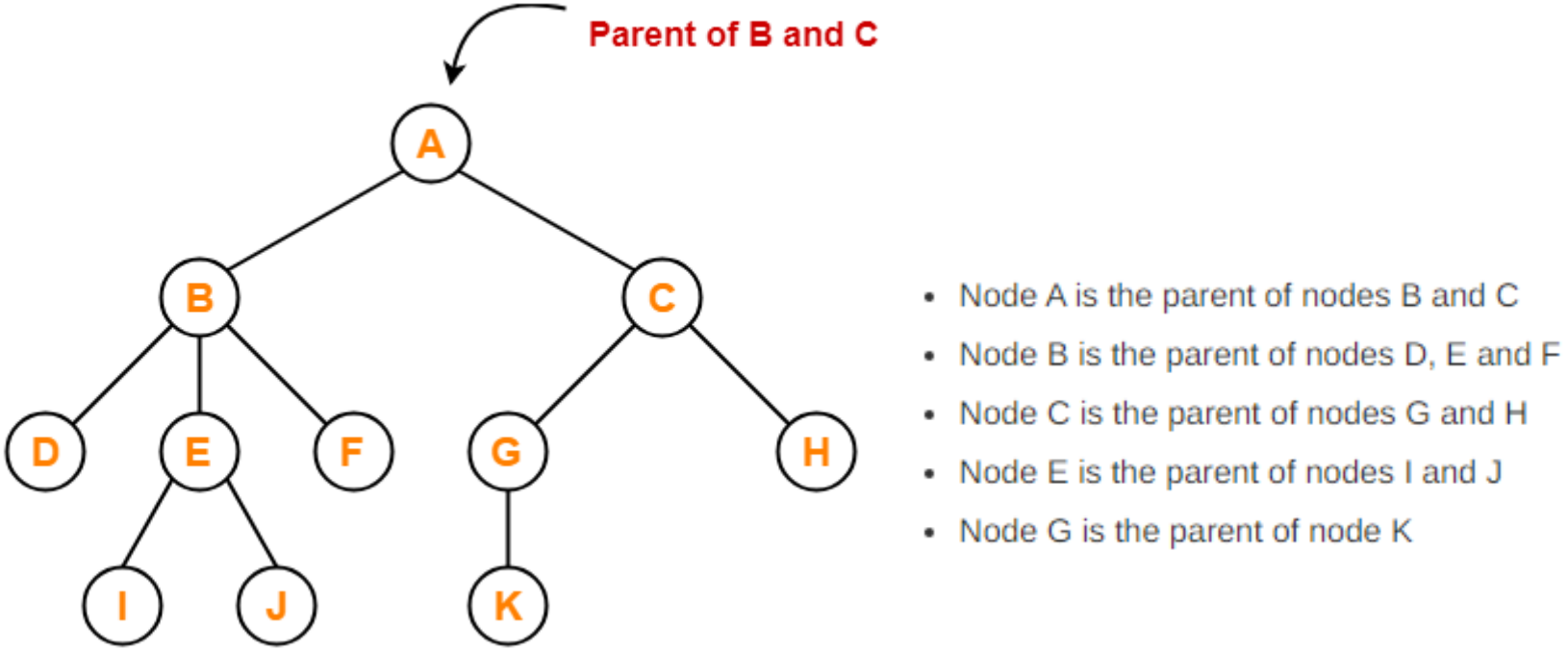
Edge

- The connecting link between any two nodes is called as an **edge**.
- In a tree with n number of nodes, there are exactly (n-1) number of edges.



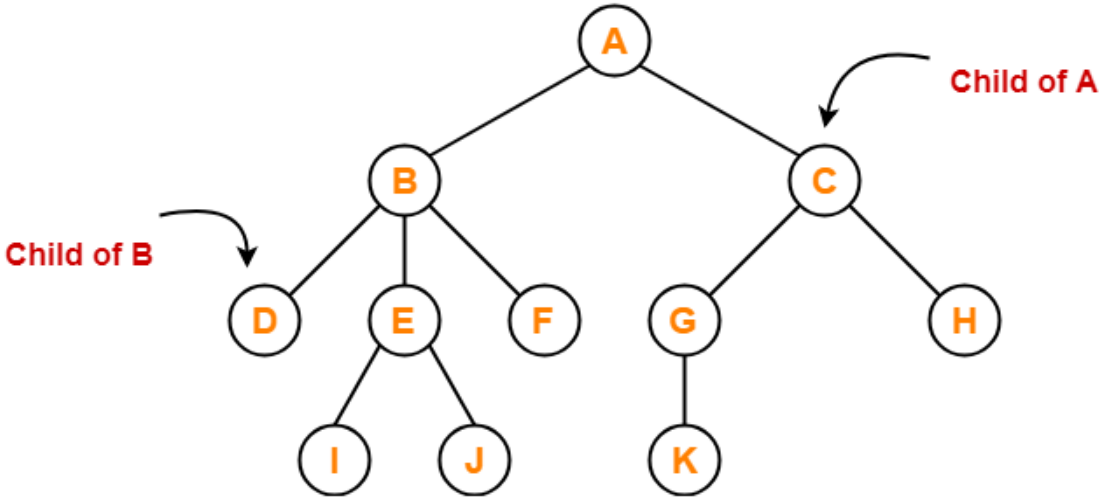
Parent

- The node which has a branch from it to any other node is called as a **parent node**.
- In other words, the node which has one or more children is called as a parent node.
- In a tree, a parent node can have any number of child nodes.



Child

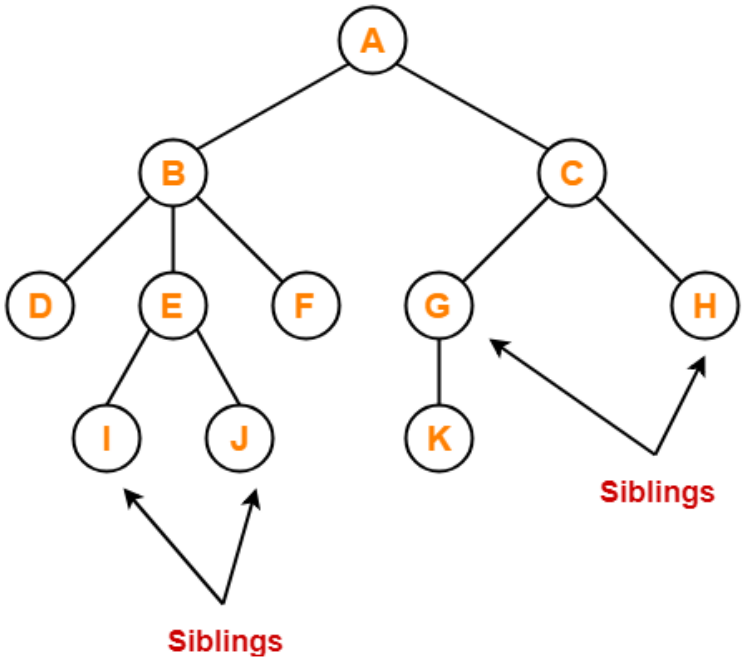
- The node which is a descendant of some node is called as a **child node**.
- All the nodes except root node are child nodes.



- Nodes B and C are the children of node A
- Nodes D, E and F are the children of node B
- Nodes G and H are the children of node C
- Nodes I and J are the children of node E
- Node K is the child of node G

Siblings

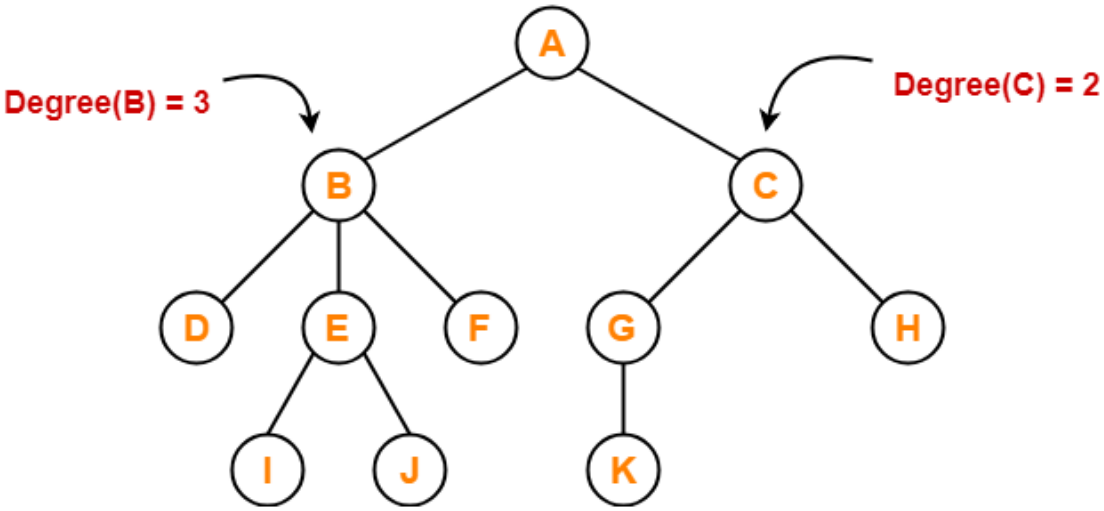
- Nodes which belong to the same parent are called as **siblings**.
- In other words, nodes with the same parent are sibling nodes.



- Nodes B and C are siblings
- Nodes D, E and F are siblings
- Nodes G and H are siblings
- Nodes I and J are siblings

Degree

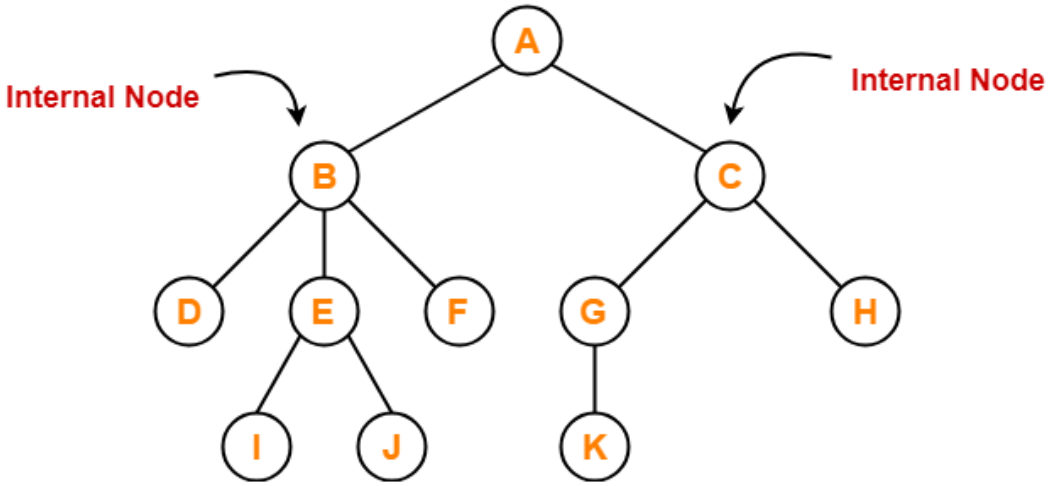
- **Degree of a node** is the total number of children of that node.
- **Degree of a tree** is the highest degree of a node among all the nodes in the tree.



- Degree of node A = 2
- Degree of node B = 3
- Degree of node C = 2
- Degree of node D = 0
- Degree of node E = 2
- Degree of node F = 0
- Degree of node G = 1
- Degree of node H = 0
- Degree of node I = 0
- Degree of node J = 0
- Degree of node K = 0

Internal node

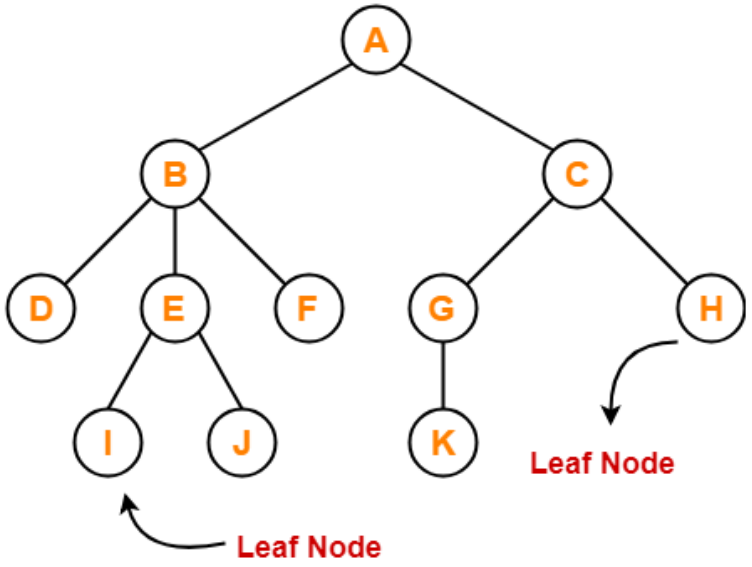
- The node which has at least one child is called as an **internal node**.
- Internal nodes are also called as **non-terminal nodes**.
- Every non-leaf node is an internal node.



Here, nodes A, B, C, E and G are internal nodes.

Leaf node

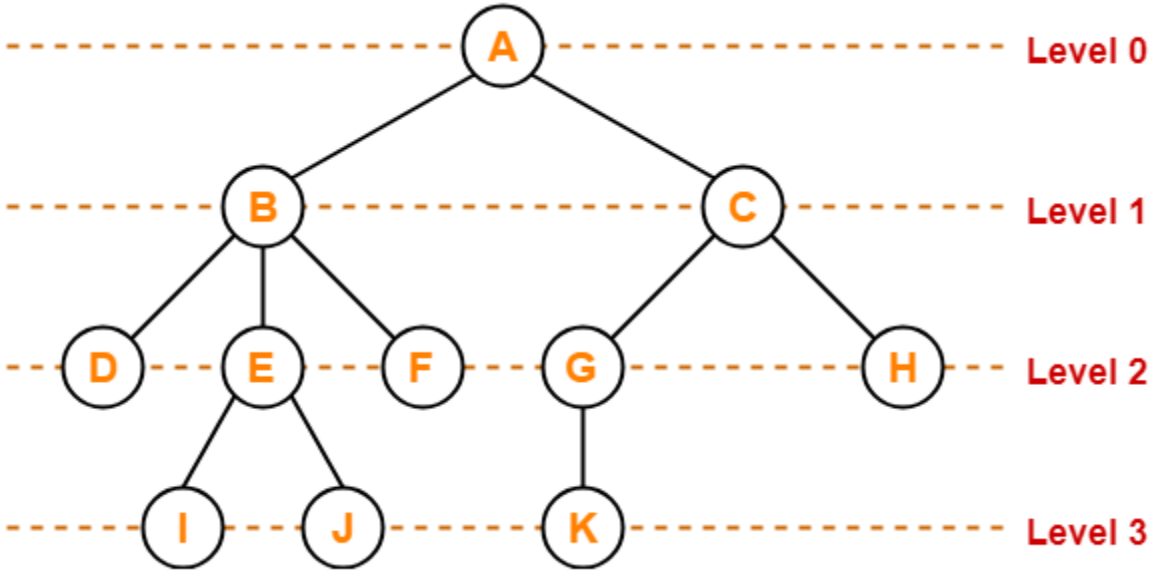
- The node which does not have any child is called as a **leaf node**.
- Leaf nodes are also called as **external nodes** or **terminal nodes**.



Here, nodes D, I, J, F, K and H are leaf nodes.

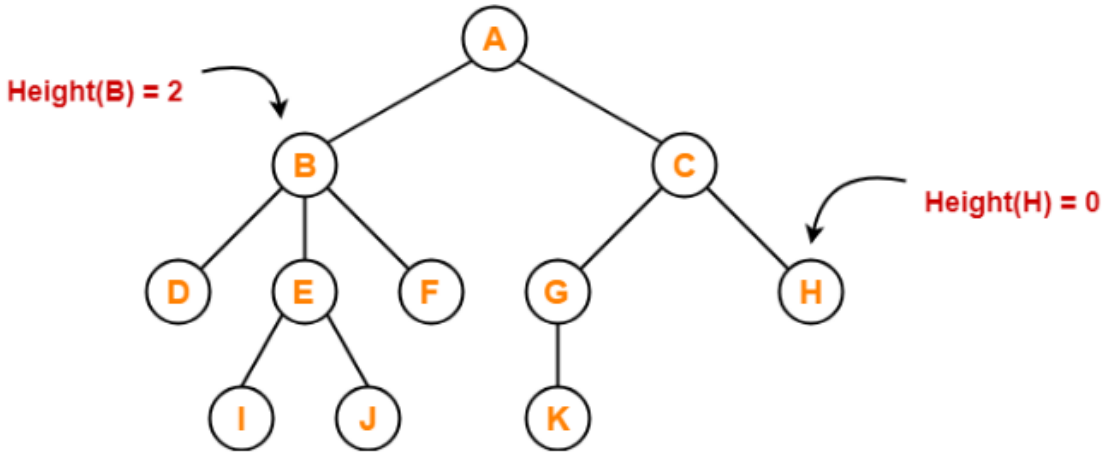
Level

- In a tree, each step from top to bottom is called as **level of a tree**.
- The level count starts with 0 and increments by 1 at each level or step.



Height

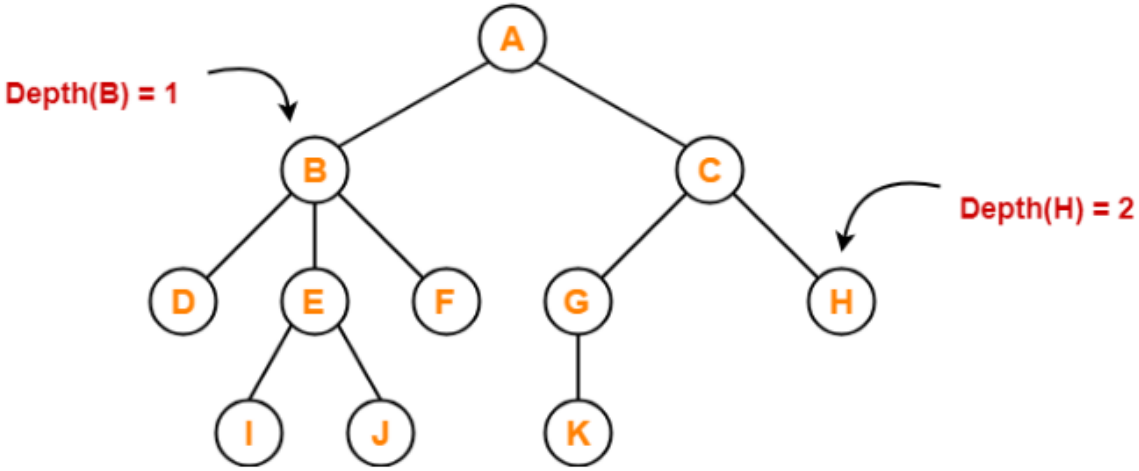
- Total number of edges that lies on the longest path from any leaf node to a particular node is called as **height of that node**.
- **Height of a tree** is the height of root node.
- Height of all leaf nodes = 0



- Height of node A = 3
- Height of node B = 2
- Height of node C = 2
- Height of node D = 0
- Height of node E = 1
- Height of node F = 0
- Height of node G = 1
- Height of node H = 0
- Height of node I = 0
- Height of node J = 0
- Height of node K = 0

Depth

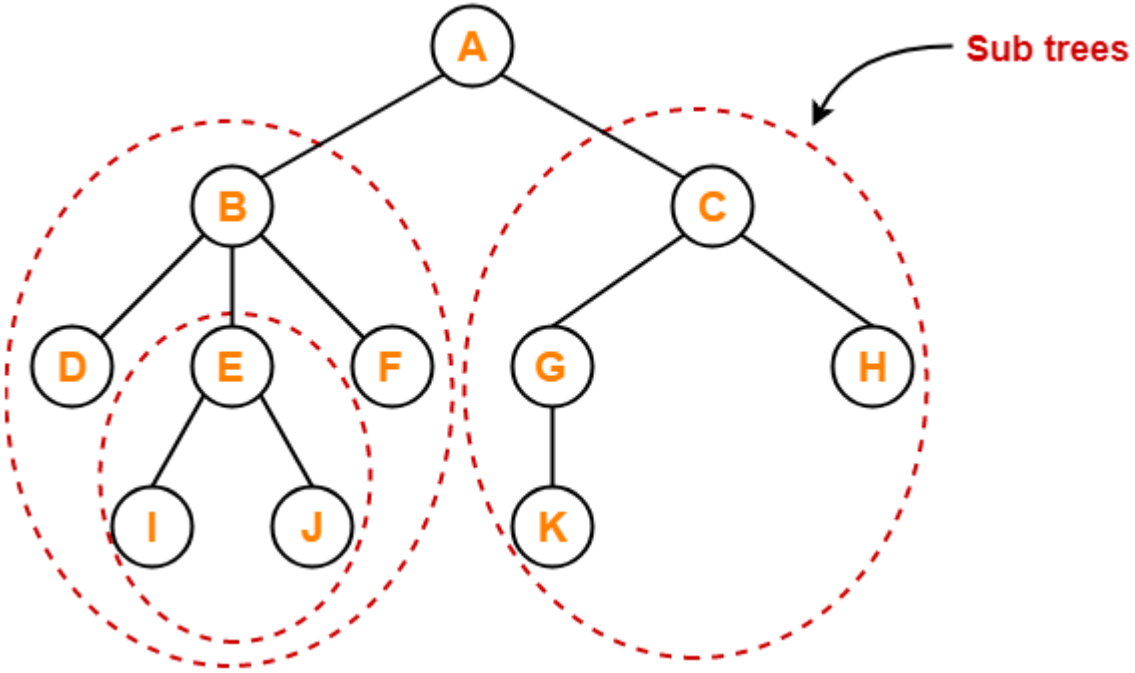
- Total number of edges from root node to a particular node is called as **depth of that node**.
- **Depth of a tree** is the total number of edges from root node to a leaf node in the longest path.
- Depth of the root node = 0
- The terms “level” and “depth” are used interchangeably.



- Depth of node A = 0
- Depth of node B = 1
- Depth of node C = 1
- Depth of node D = 2
- Depth of node E = 2
- Depth of node F = 2
- Depth of node G = 2
- Depth of node H = 2
- Depth of node I = 3
- Depth of node J = 3
- Depth of node K = 3

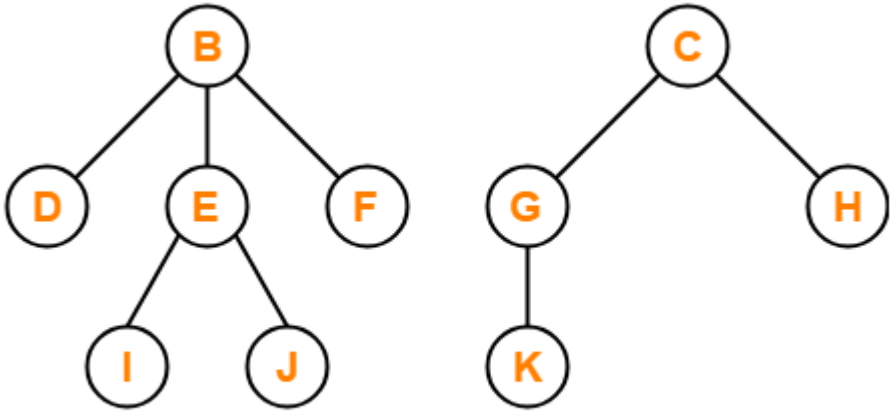
Subtree

- In a tree, each child from a node forms a **subtree** recursively.
- Every child node forms a subtree on its parent node.



Forest

- A forest is a set of disjoint trees.



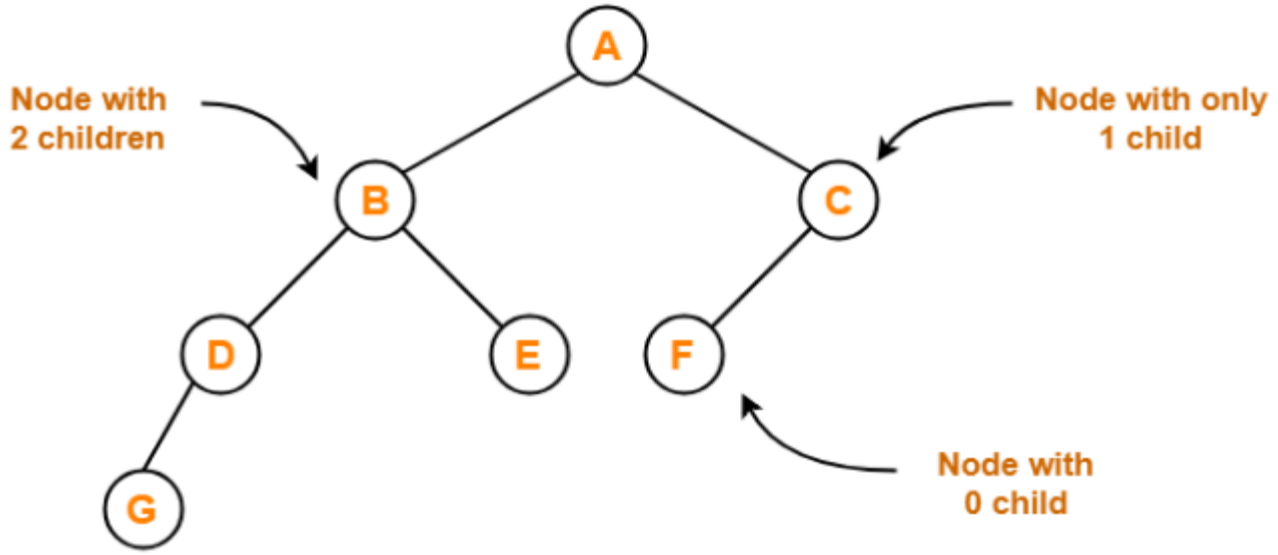
Forest

Types of Tree

- General Tree
- Binary Tree
- Binary Search Tree
- AVL Tree
- Red-Black Tree
- N-ary Tree

Binary Tree

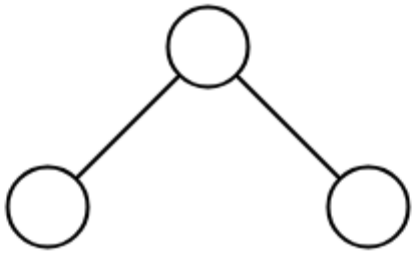
- Binary tree is a special tree data structure in which each node can have at most 2 children.
- Thus, in a binary tree, Each node has either 0 child or 1 child or 2 children.



Binary Tree Example

Unlabeled Binary Tree

- A binary tree is unlabeled if its nodes are not assigned any label.

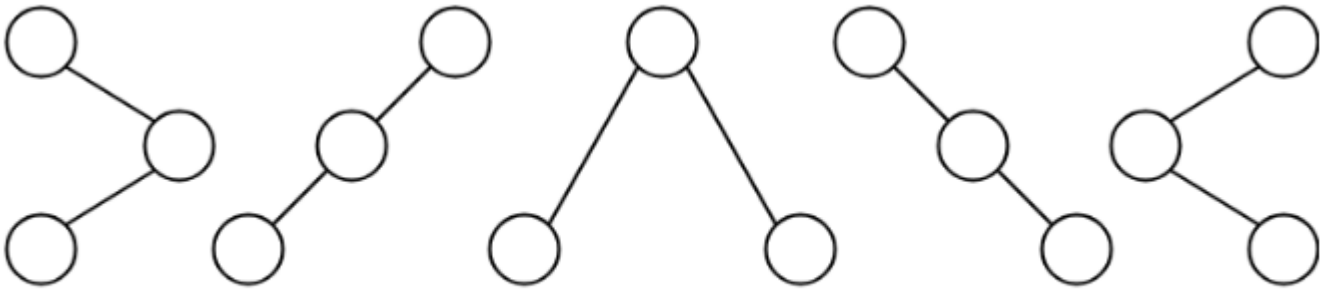


Unlabeled Binary Tree

Number of different Binary Trees possible with 'n' unlabeled nodes $= \frac{{}^{2n}C_n}{n+1}$

Example

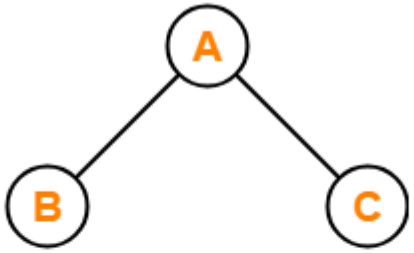
- Consider we want to draw all the binary trees possible
- Number of binary trees possible with 3 unlabeled nodes
- $= {}^{2 \times 3}C_3 / (3 + 1)$
- $= {}^{6}C_3 / 4$
- $= 5$



Binary Trees Possible With 3 Unlabeled Nodes

Labeled Binary Tree

- A binary tree is labelled if all its nodes are assigned a label.



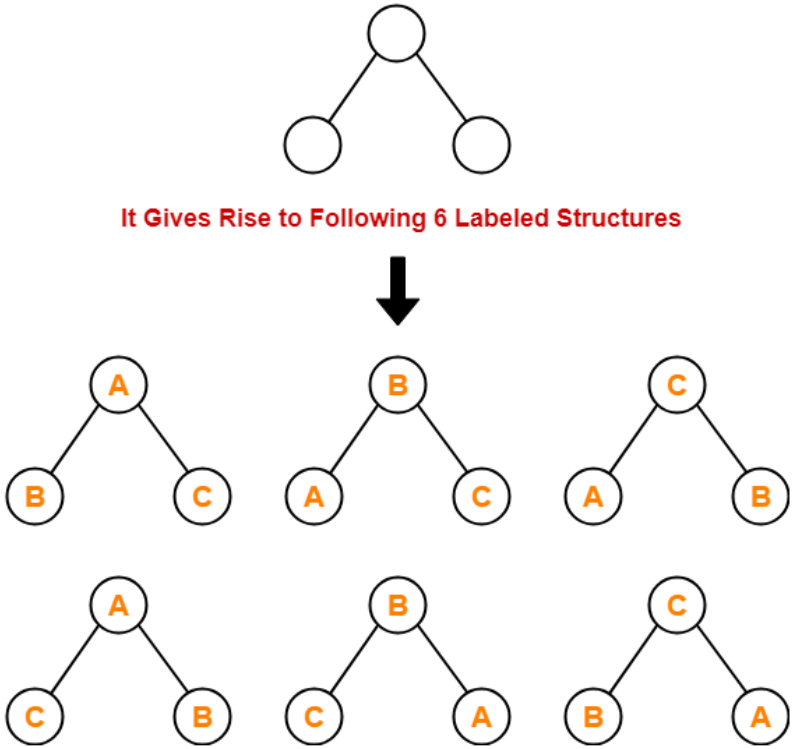
Labeled Binary Tree

Number of different Binary Trees possible with 'n' labeled nodes

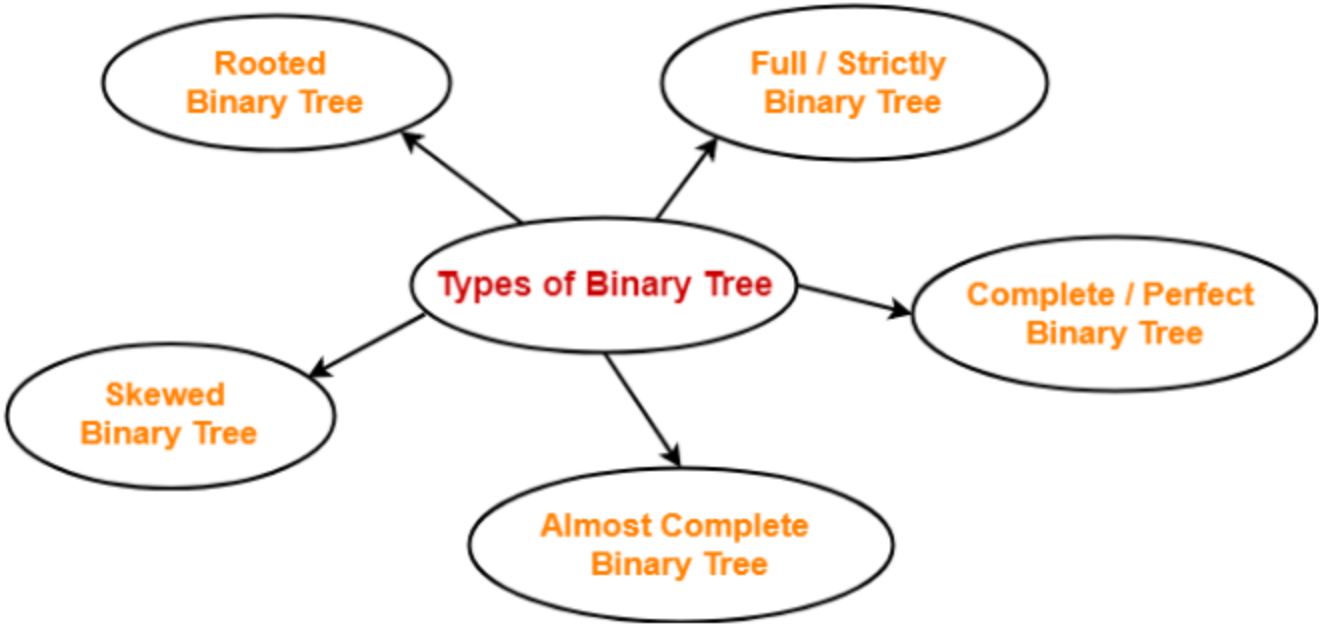
$$= \frac{{}^{2n}C_n}{n+1} \times n!$$

Example

- Consider we want to draw all the binary trees possible with 3 labeled nodes.
- Number of binary trees possible with 3 labeled nodes
- = $\{ {}^{2 \times 3}C_3 / (3 + 1) \} \times 3!$
- = $\{ {}^{6}C_3 / 4 \} \times 6$
- = 5 x 6
- = 30

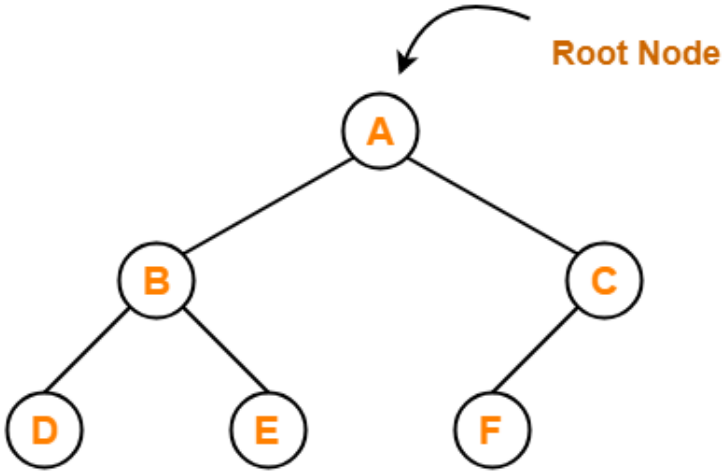


Types of Binary Trees



Rooted Binary Tree

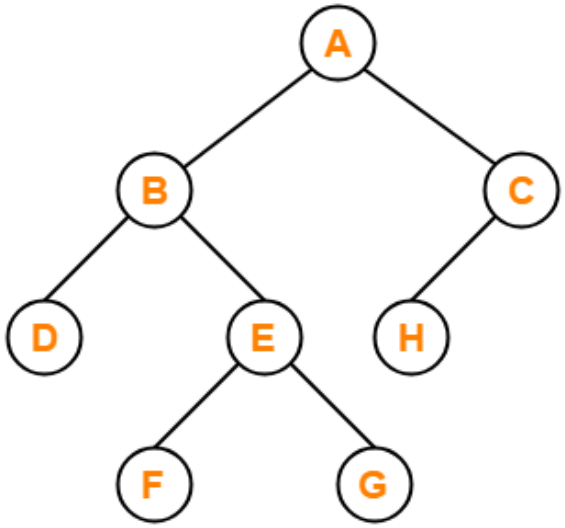
- A **rooted binary tree** is a binary tree that satisfies the following 2 properties:
  - It has a root node.
  - Each node has at most 2 children.



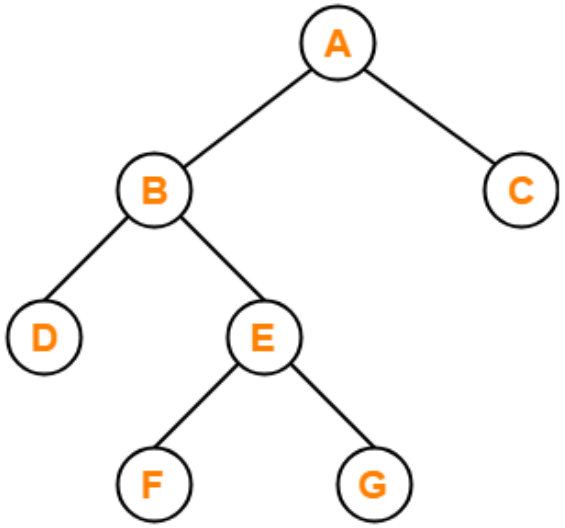
Rooted Binary Tree

Full/Strictly Binary Tree

- A binary tree in which every node has either 0 or 2 children is called as a **Full binary tree**.
- Full binary tree is also called as **Strictly binary tree**.



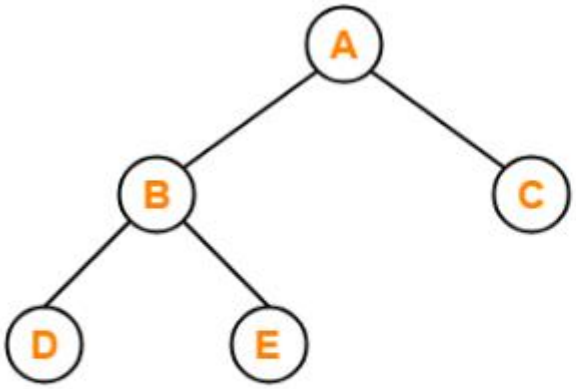




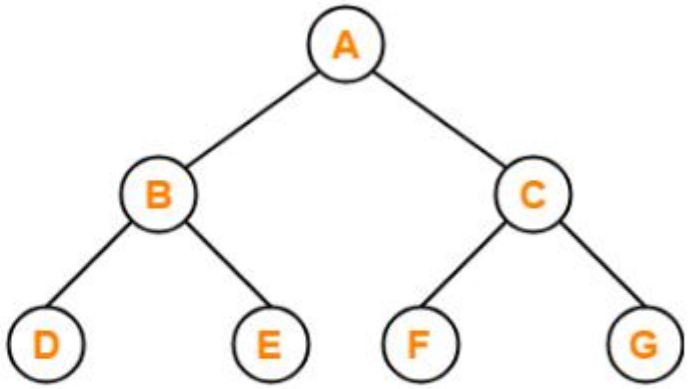


Complete /Perfect Binary Tree

- A **complete binary tree** is a binary tree that satisfies the following 2 properties:
  - Every internal node has exactly 2 children.
  - All the leaf nodes are at the same level.



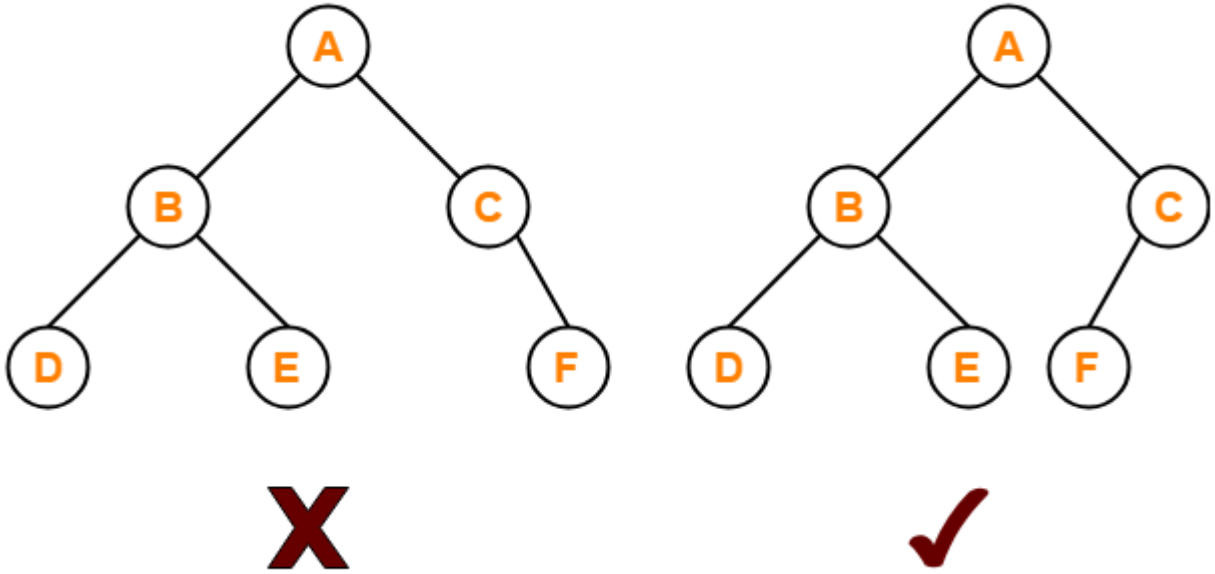
X



✓

Almost Complete Binary Tree

- An **almost complete binary tree** is a binary tree that satisfies the following 2 properties-
  - All the levels are completely filled except possibly the last level.
  - The last level must be strictly filled from left to right.

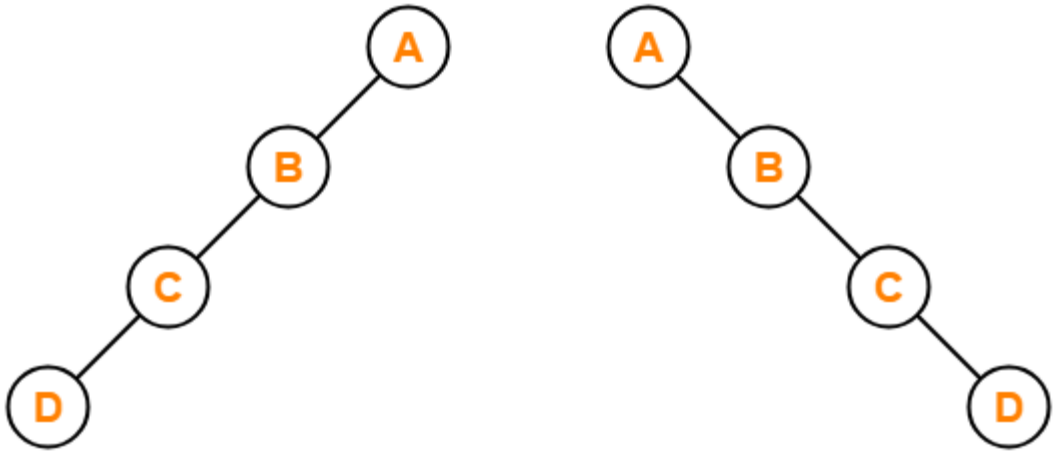


Skewed Binary Tree

- A **skewed binary tree** is a binary tree that satisfies the following 2 properties-
- All the nodes except one node has one and only one child.
- The remaining node has no child.

OR

- A **skewed binary tree** is a binary tree of n nodes such that its depth is (n-1).



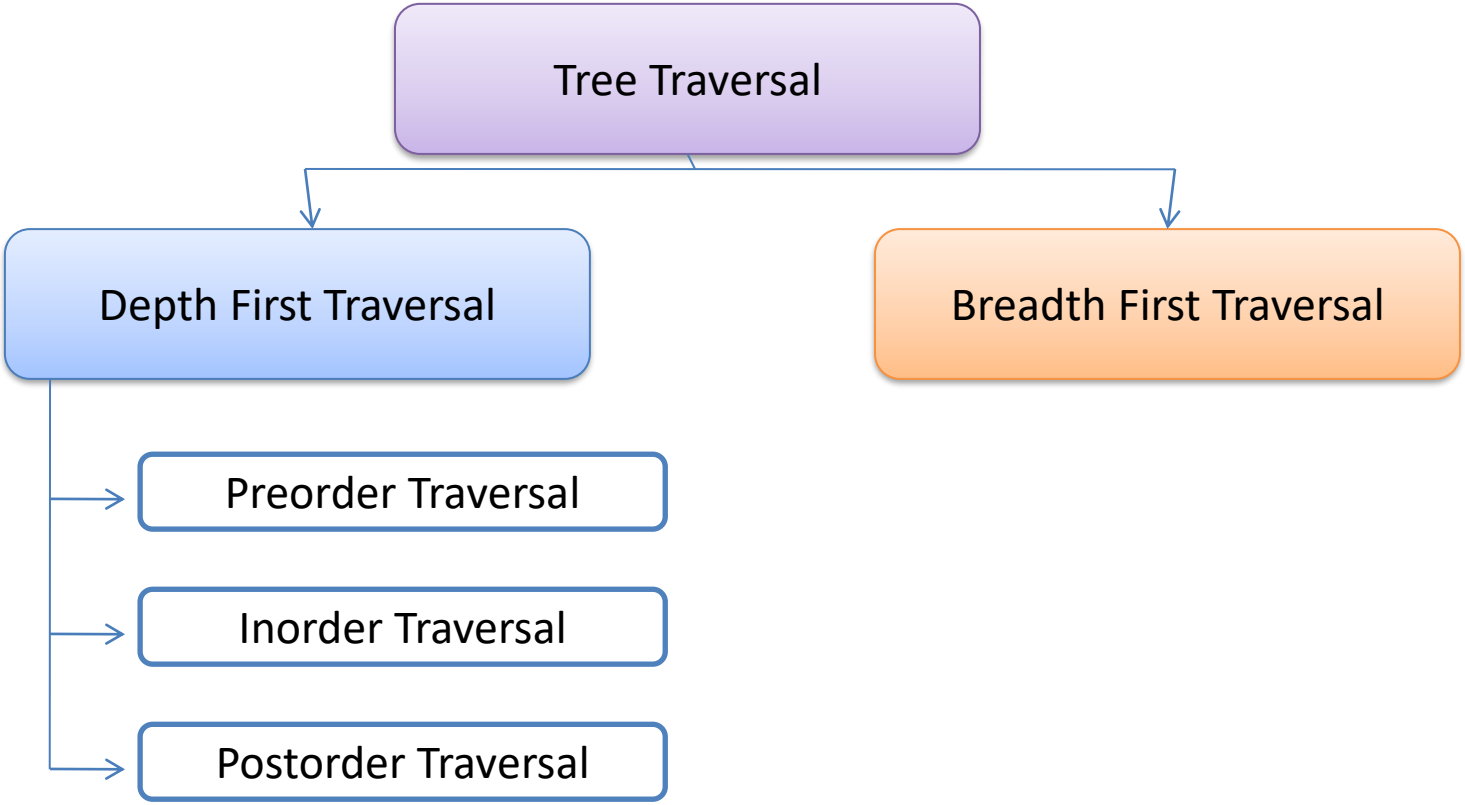
Left Skewed Binary Tree

Right Skewed Binary Tree

Tree Traversal

- In order to perform any operation on a tree, you need to reach to the specific node. The tree traversal algorithm helps in visiting a required node in the tree.
- Tree Traversal refers to the process of visiting each node in a tree data structure exactly once.

Tree traversal techniques

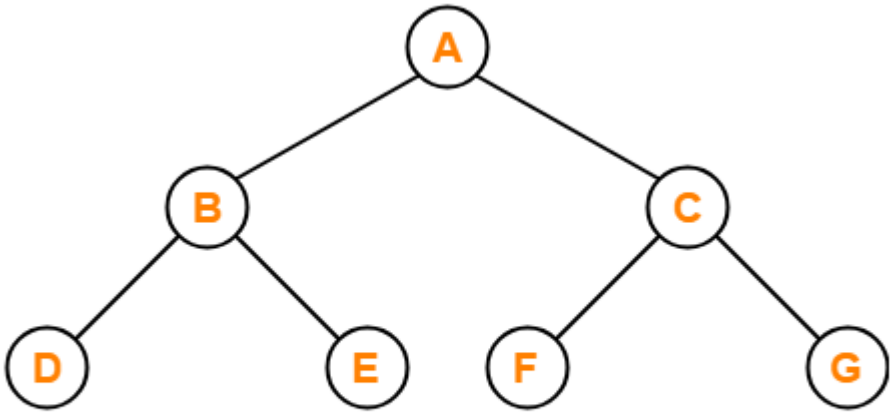


Depth First Traversal

- Following three traversal techniques fall under Depth First Traversal-
  1. Preorder Traversal
  2. Inorder Traversal
  3. Postorder Traversal

Preorder Traversal

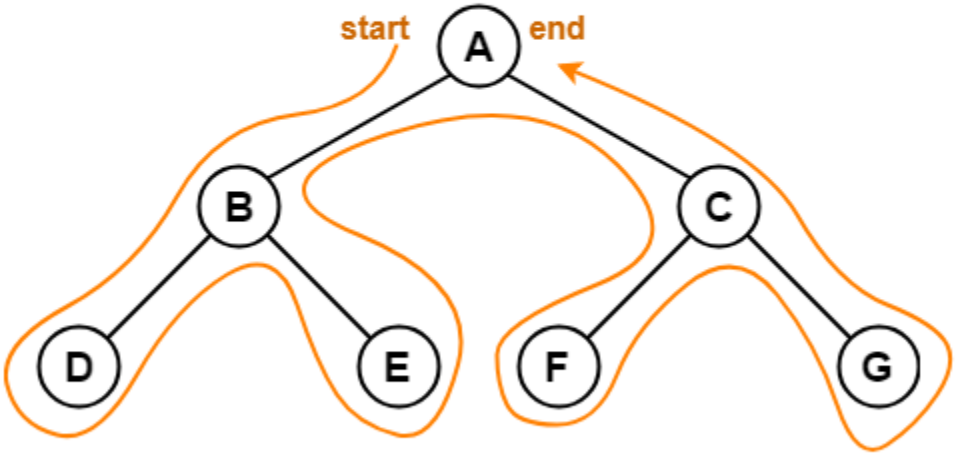
- **Algorithm-**
  - Visit the root
  - Traverse the left sub tree i.e. call Preorder (left sub tree)
  - Traverse the right sub tree i.e. call Preorder (right sub tree)



Preorder Traversal : A , B , D , E , C , F , G

Preorder Traversal Shortcut

Traverse the entire tree starting from the root node keeping yourself to the left.



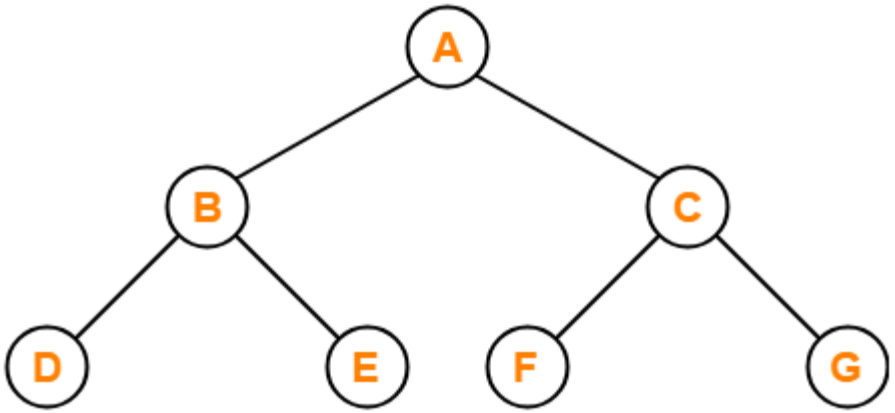
Preorder Traversal : A , B , D , E , C , F , G

Real-time applications of Preorder traversal

Domain	Preorder Usage	Why
File Explorer (Copy/Paste folder)	When copying a folder → copy the folder first, then its subfolders and files	Parent directory must exist first
Game AI (Decision Trees)	Game engine evaluates parent decision node first before exploring next moves	Root decision controls next actions
Organization Hierarchy Report	CEO → Directors → Managers → Employees	Higher authority processed first
Web Crawlers (URL scan)	Scan a root URL first, then follow internal links	Root URL is starting point
XML/HTML Parsing (serialize DOM tree)	Write parent tag before nested tags	Correct DOM structure generation

Inorder Traversal

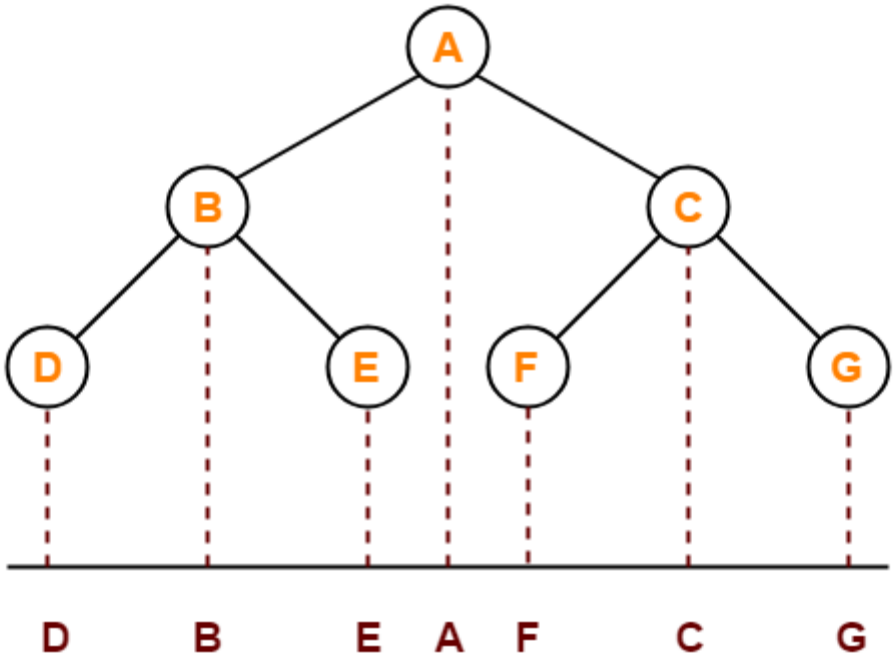
- **Algorithm-**
  - Traverse the left sub tree i.e. call Inorder (left sub tree)
  - Visit the root
  - Traverse the right sub tree i.e. call Inorder (right sub tree)



Inorder Traversal : D , B , E , A , F , C , G

Inorder Traversal Shortcut

Keep a plane mirror horizontally at the bottom of the tree and take the projection of all the nodes.



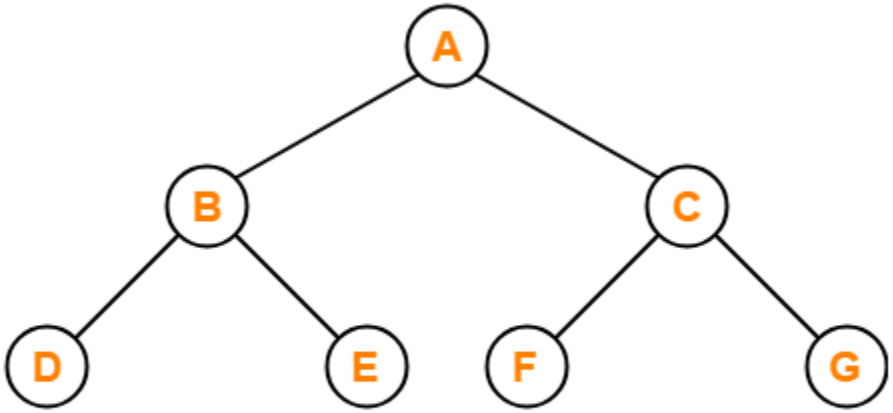
Inorder Traversal : D , B , E , A , F , C , G

Real-time applications of Inorder traversal

Domain	Preorder Usage	Why
Searching sorted values (BST)	Extract marks, salaries, roll numbers in ascending order	Produces sorted sequence
Music/Video Playlist Browsing	Traverse by alphabetical ordering (A–Z / Z–A)	Ordered access
Database Indexing (B-Trees/B+ Trees)	Retrieve records range-wise (10–50, A–F)	Inorder gives sorted keys
E-commerce Price Filtering	Show products from low price → high	Uses BST on prices internally
Leaderboard in games	Print players by ascending ranking	BST traversal for sorted order

Postorder Traversal

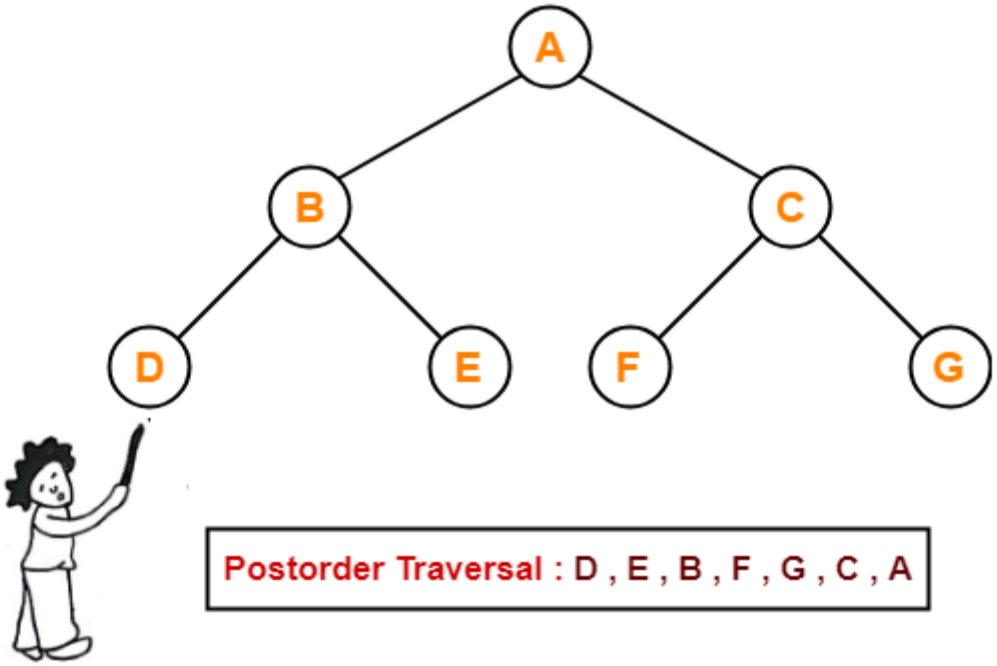
- **Algorithm-**
  - Traverse the left sub tree i.e. call Postorder (left sub tree)
  - Traverse the right sub tree i.e. call Postorder (right sub tree)
  - Visit the root



Postorder Traversal : D , E , B , F , G , C , A

Postorder Traversal Shortcut

Pluck all the leftmost leaf nodes one by one.

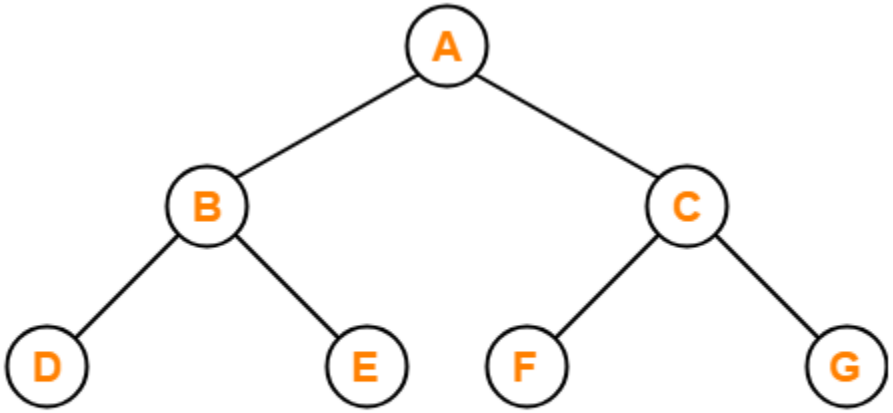


Real-time applications of Postorder traversal

Domain	Preorder Usage	Why
Safe deletion of folders	Delete inside files & subfolders before deleting main folder	Avoid dangling folders
Uninstalling software	Remove subcomponents → drivers → main program	Dependencies removed first
Freeing memory (Garbage collectors)	Free children nodes before parent to avoid memory leaks	Prevents invalid pointers
Evaluating expression calculators	Evaluate operands first → operation last	Postfix evaluation
Project Management (Task scheduling)	Finish subtasks before marking main task as completed	Bottom-up completion

Breadth First Search

- Breadth First Traversal of a tree prints all the nodes of a tree level by level.
- Breadth First Traversal is also called as **Level Order Traversal**.



Level Order Traversal : A , B , C , D , E , F , G

Real-time applications of BFS traversal

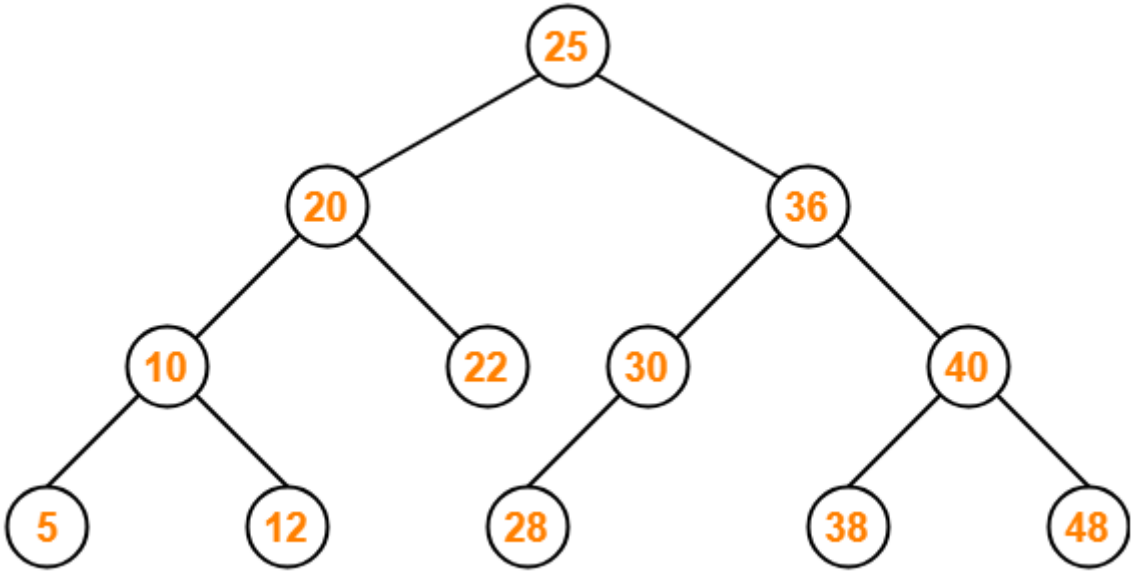
Domain	Preorder Usage	Why
Shortest Path in Maps & Navigation (GPS, routing apps)	Explore nodes level-by-level to find the shortest path in an unweighted graph	BFS guarantees the shortest route in terms of number of steps/hops
Network Broadcasting (Messaging, packet spreading)	Send data to all immediate neighbors, then their neighbors, etc.	Models real broadcast propagation; reaches closest devices first
Social Media Reach / Friend Suggestions	Discover users by degrees of connection: 1st-degree → 2nd-degree → etc.	BFS naturally explores "layers" of a social graph
Real-time Customer Support Chatbots / Query Systems	Search closest relevant answer node first before going deeper	Fast retrieval by checking most directly related concepts early
Web Crawlers (Level-limited crawling)	Crawl pages layer-by-layer from a seed URL	Prevents deep diving into one branch and improves real-time freshness

Real-time applications of BFS traversal

Domain	Preorder Usage	Why
AI Pathfinding in Games (real-time movement)	Find shortest route across grid-based maps	Ensures fast, minimal-step paths for NPC movement
Router Path Selection / Network Switching	Identify shortest hop count to destination nodes	Crucial for low-latency packet routing
Emergency Response Systems	Compute nearest available ambulance/police/fire unit	BFS prioritizes geographically closest units first
Robot Exploration (frontier-based)	Robots explore nearest unvisited cells before farther ones	Smooth, predictable motion and reduced travel overhead
Real-time Recommendation Systems	Expand user/item relationships in layers to find closest matches	BFS ensures recommendations are relevant and not overly distant

Binary Search Tree construction

- In a binary search tree (BST), each node contains:
  - Only smaller values in its left sub tree
  - Only larger values in its right sub tree



BST construction

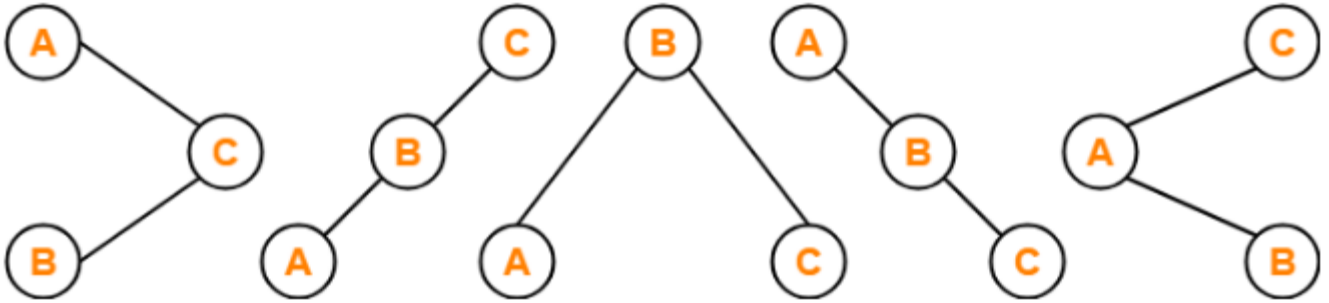
- Number of distinct binary search trees possible with 3 distinct Nodes

  $= {}^{2\times3}C_3 / 3+1$

  $= {}^{6}C_3 / 4$

  $= 5$

- If three distinct Nodes are A, B and C, then 5 distinct binary search trees are:



Example

- Construct a Binary Search Tree (BST) for the following sequence of numbers:

  50, 70, 60, 20, 90, 10, 40, 100

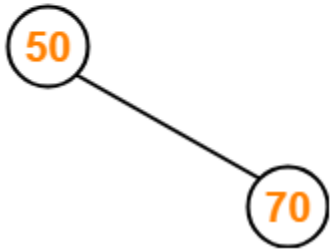
- When elements are given in a sequence,
  - Always consider the first element as the root node.
  - Consider the given elements and insert them in the BST one by one.

Insert 50-



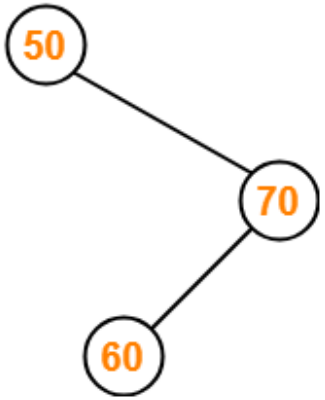
Insert 70-

- As 70 > 50, so insert 70 to the right of 50.



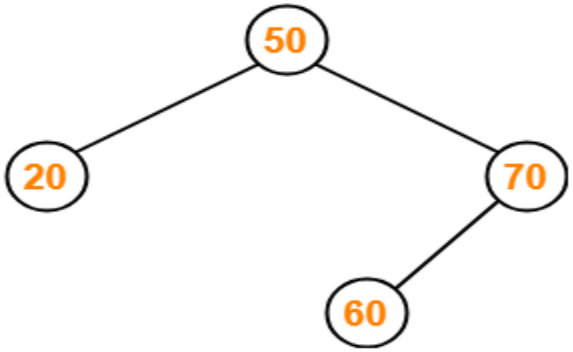
Insert 60-

- As 60 > 50, so insert 60 to the right of 50.
- As 60 < 70, so insert 60 to the left of 70.



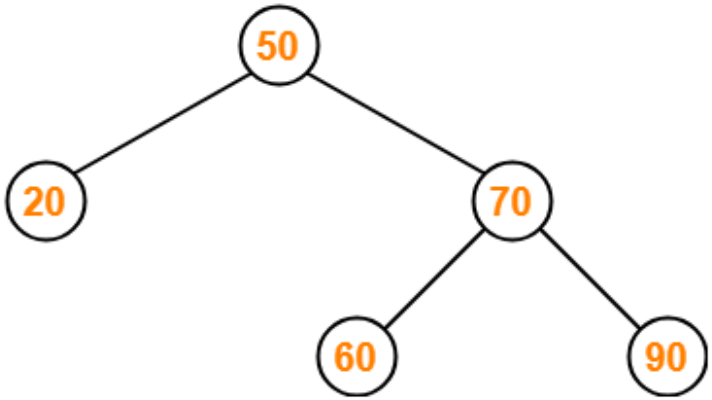
Insert 20-

- As 20 < 50, so insert 20 to the left of 50.



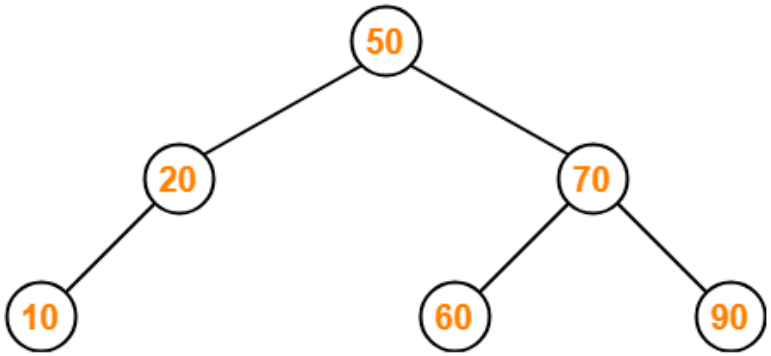
Insert 90-

- As 90 > 50, so insert 90 to the right of 50.
- As 90 > 70, so insert 90 to the right of 70.



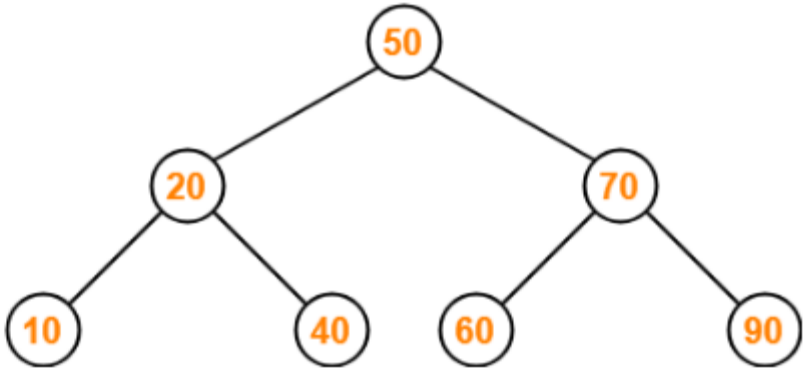
Insert 10-

- As 10 < 50, so insert 10 to the left of 50.
- As 10 < 20, so insert 10 to the left of 20.



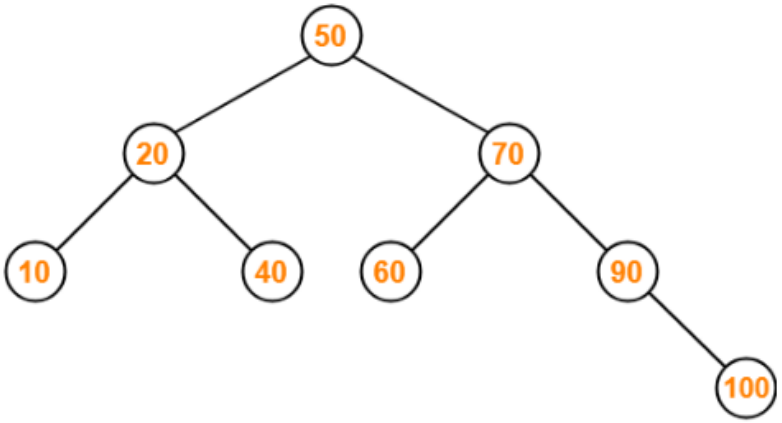
Insert 40-

- As 40 < 50, so insert 40 to the left of 50.
- As 40 > 20, so insert 40 to the right of 20.



Insert 100-

- As 100 > 50, so insert 100 to the right of 50.
- As 100 > 70, so insert 100 to the right of 70.
- As 100 > 90, so insert 100 to the right of 90.



Practice problem

- **Problem-01:**
- A binary search tree is generated by inserting in order of the following integers-

  50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24

- The number of nodes in the left subtree and right subtree of the root respectively is _____.
  - (4, 7)
  - (7, 4)
  - (8, 3)
  - (3, 8)

Practice problem

- **Problem-02:**
- How many distinct binary search trees can be constructed out of 4 distinct keys?
  - 5
  - 14
  - 24
  - 35

C code – BST construction

```
typedef struct BST
{
        int data;
        struct BST *left;
        struct BST *right;
}node;

node *create()
{
        node *temp;
        printf("nEnter data:");
        temp=(node*)malloc(sizeof(node));
        scanf("%d",&temp->data);
        temp->left=temp->right=NULL;
        return temp;
}

void insert(node *root,node *temp)
{
        if(temp->data<root->data)
        {
                if(root->left!=NULL)
                        insert(root->left,temp);
                else
                        root->left=temp;
        }

        if(temp->data>root->data)
        {
                if(root->right!=NULL)
                        insert(root->right,temp);
                else
                        root->right=temp;
        }
}
```

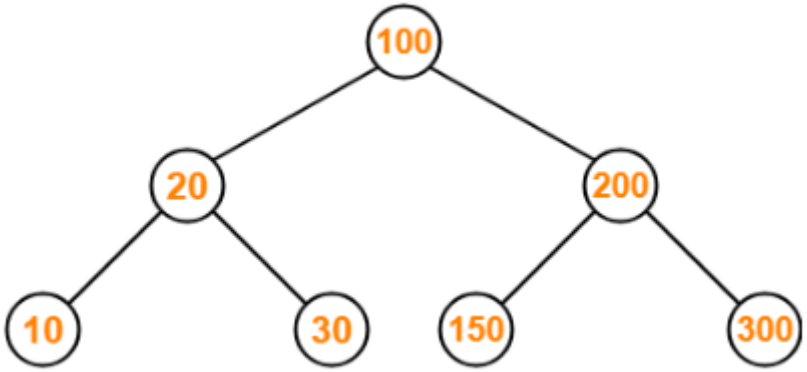
```
int main()
{
        char ch;
        node *root=NULL,*temp;

        do
        {
                temp=create();
                if(root==NULL)
                        root=temp;
                else
                        insert(root,temp);

                printf("nDo you want to enter more(y/n)?");
                getchar();
                scanf("%c",&ch);
        }while(ch=='y'|ch=='Y');

        printf("nPreorder Traversal: ");
        preorder(root);
        return 0;
```

BST Traversal



Preorder Traversal-

100 , 20 , 10 , 30 , 200 , 150 , 300

Inorder Traversal-

10 , 20 , 30 , 100 , 150 , 200 , 300

Postorder Traversal-

10 , 30 , 20 , 150 , 300 , 200 , 100

- Inorder traversal of a binary search tree always yields all the nodes in increasing order.

C Code – BST Traversal

```
Void inorder(node *root)
{
   if(root!=NULL)
   {
        inorder(root->left);
        printf("%d ",root->data);
        inorder(root->right);
   }
}
```

C Code – BST Traversal

```
void postorder(node *root)
{
   if(root!=NULL)
   {
        postorder(root->left);
        postorder(root->right);
        printf("%d ",root->data);
   }
}
```

C Code – BST Traversal

```
void preorder(node *root)
{
   if(root!=NULL)
   {
        printf("%d ",root->data);
        preorder(root->left);
        preorder(root->right);
   }
}
```

Practice

- Suppose the numbers 7 , 5 , 1 , 8 , 3 , 6 , 0 , 9 , 4 , 2 are inserted in that order into an initially empty binary search tree. The binary search tree uses the usual ordering on natural numbers.
- What is the inorder traversal sequence of the resultant tree?

A. 7 , 5 , 1 , 0 , 3 , 2 , 4 , 6 , 8 , 9
B. 0 , 2 , 4 , 3 , 1 , 6 , 5 , 9 , 8 , 7
C. 0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9
D. 9 , 8 , 6 , 4 , 2 , 3 , 0 , 1 , 5 , 7

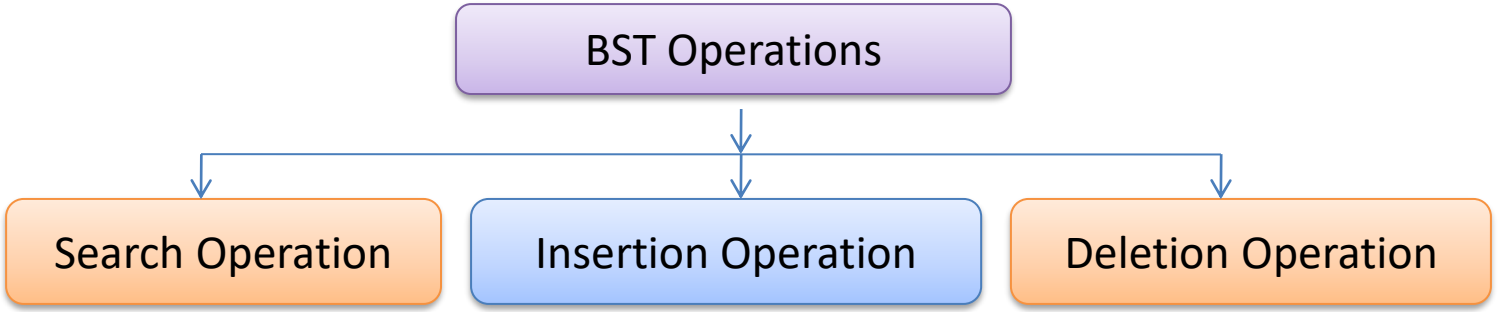
Problem-2

- The preorder traversal sequence of a binary search tree is-
  - 30 , 20 , 10 , 15 , 25 , 23 , 39 , 35 , 42
- Which one of the following is the postorder traversal sequence of the same tree?

A. 10 , 20 , 15 , 23 , 25 , 35 , 42 , 39 , 30
B. 15 , 10 , 25 , 23 , 20 , 42 , 35 , 39 , 30
C. 15 , 20 , 10 , 23 , 25 , 42 , 35 , 39 , 30
D. 15 , 10 , 23 , 25 , 20 , 35 , 42 , 39 , 30

BST Operations

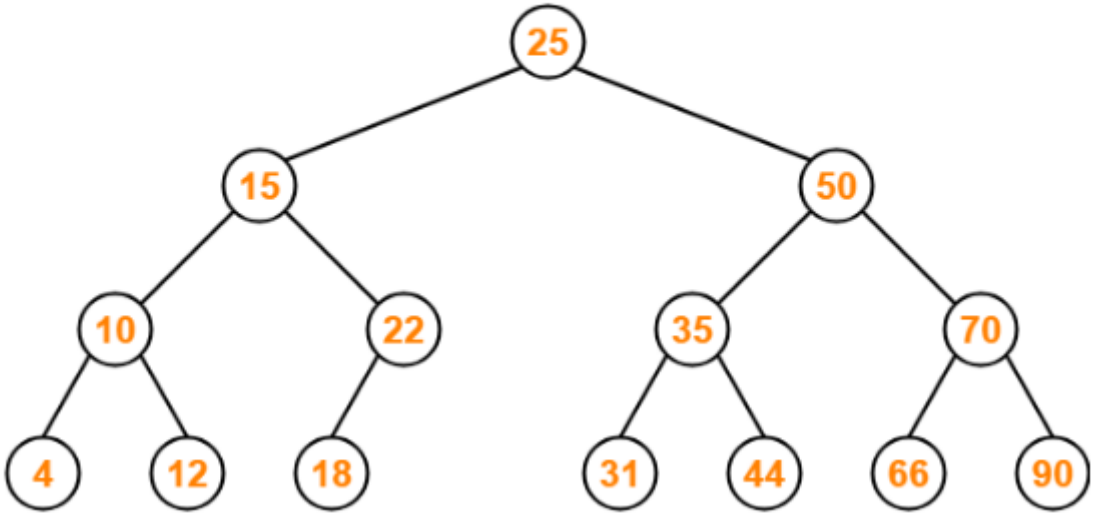
- Commonly performed binary search tree operations are:



Search Operation

- Search Operation is performed to search a particular element in the Binary Search Tree.
- For searching a given key in the BST,
  - Compare the key with the value of root node.
  - If the key is present at the root node, then return the root node.
  - If the key is greater than the root node value, then recur for the root node's right subtree.
  - If the key is smaller than the root node value, then recur for the root node's left subtree.

Consider key = 45 has to be searched in the given BST-



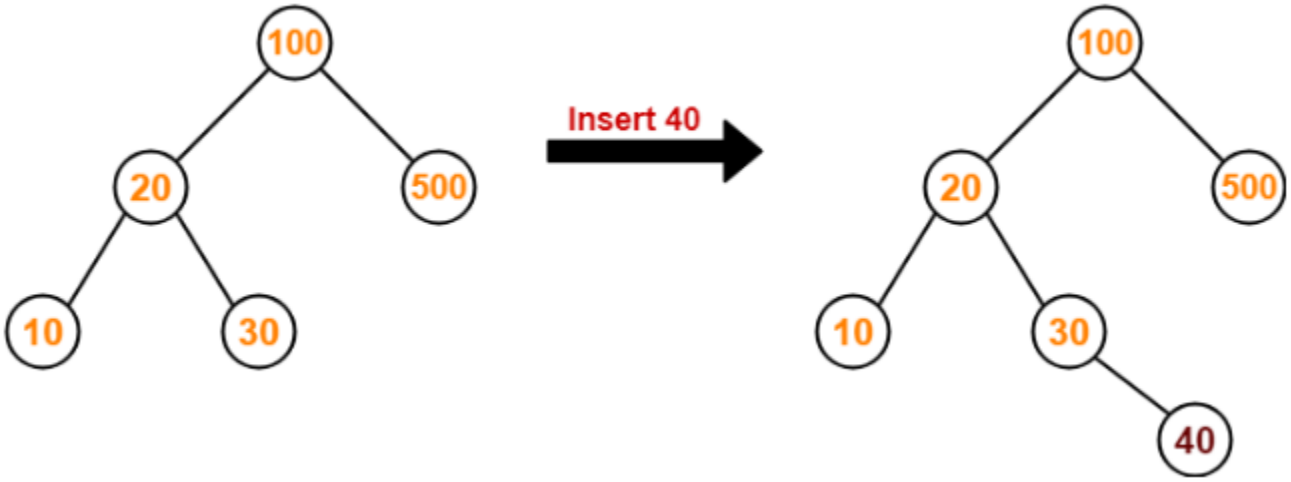
- We start our search from the root node 25.
- As 45 > 25, so we search in 25's right subtree.
- As 45 < 50, so we search in 50's left subtree.
- As 45 > 35, so we search in 35's right subtree.
- As 45 > 44, so we search in 44's right subtree but 44 has no subtrees.
- So, we conclude that 45 is not present in the above BST.

C Code – BST Search Operation

```
struct node *search(struct node *node, int key) {
  // Return NULL if the tree is empty
  if (node == NULL) return NULL;
  if (node->key == key) return node->data;
  if (key < node->key)
    search(node->left, key);
  else
    search(node->right, key);
}
```

Insertion operation

- The insertion of a new key always takes place as the child of some leaf node.
- For finding out the suitable leaf node,
  - Search the key to be inserted from the root node till some leaf node is reached.
  - Once a leaf node is reached, insert the key as child of that leaf node.



- We start searching for value 40 from the root node 100.
- As 40 < 100, so we search in 100's left subtree.
- As 40 > 20, so we search in 20's right subtree.
- As 40 > 30, so we add 40 to 30's right subtree.

C Code – Insertion Operation

```
// Create a node
struct node *newNode(int item) {
  struct node *temp = (struct node *)malloc(sizeof(struct node));
  temp->key = item;
  temp->left = temp->right = NULL;
  return temp;
}
// Insert a node
struct node *insert(struct node *node, int key) {
  // Return a new node if the tree is empty
  if (node == NULL) return newNode(key);

  // Traverse to the right place and insert the node
  if (key < node->key)
    node->left = insert(node->left, key);
  else
    node->right = insert(node->right, key);

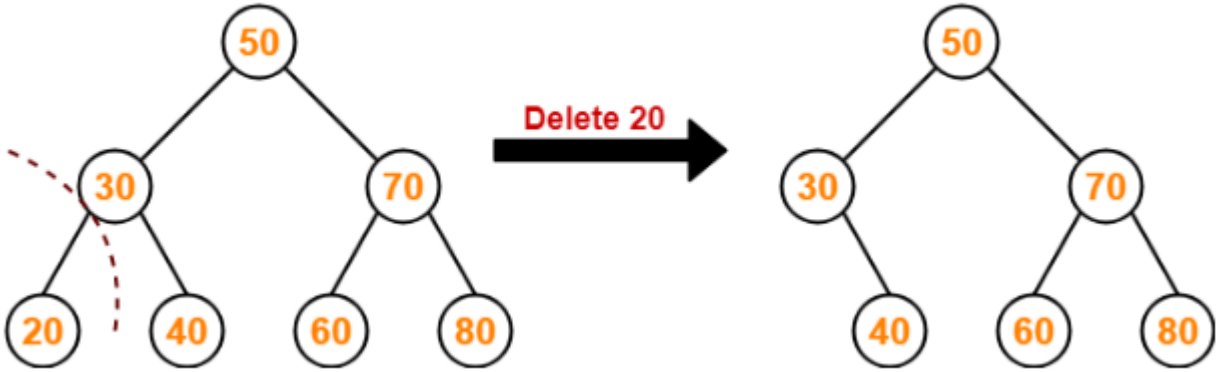
  return node;
}
```

Deletion Operation

- Deletion Operation is performed to delete a particular element from the Binary Search Tree.
- When it comes to deleting a node from the binary search tree, three cases are possible.

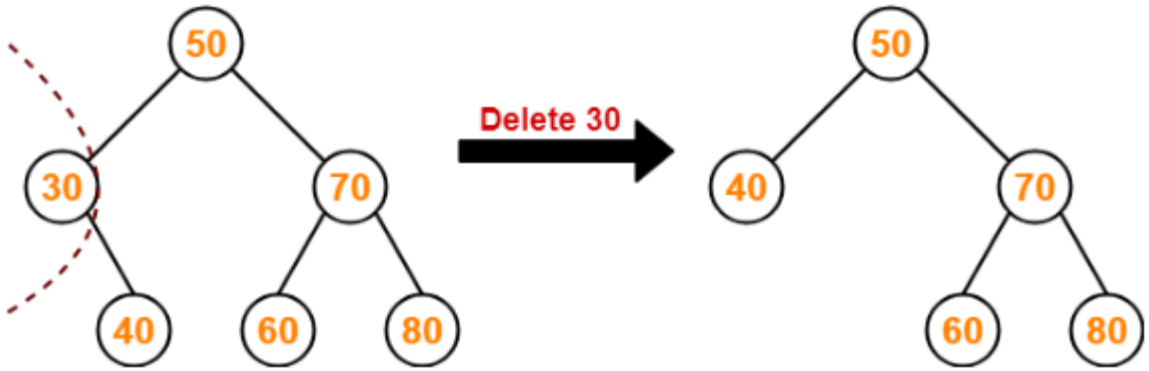
Case-01: Deletion Of A Node Having No Child (Leaf Node)

- Just remove / disconnect the leaf node that is to deleted from the tree.



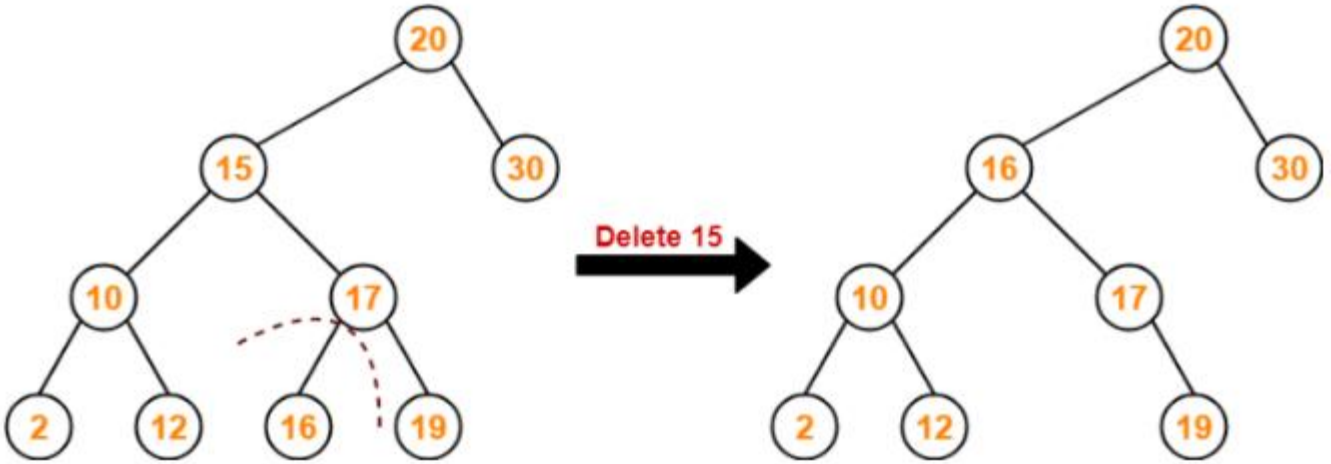
Case-02: Deletion Of A Node Having Only One Child

- Consider the following example where node with value = 30 is deleted from the BST.

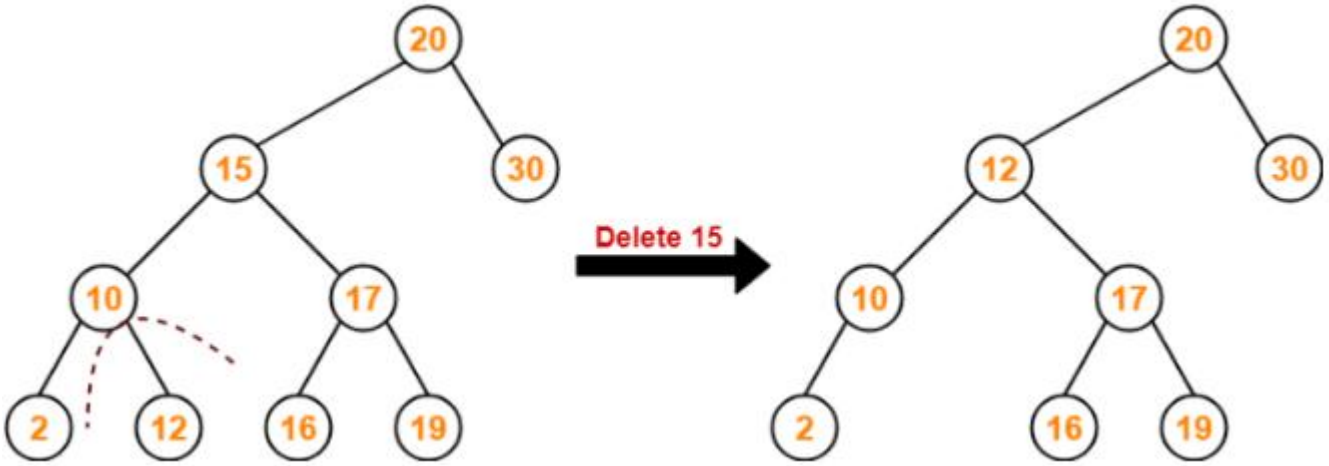


Case-03: Deletion Of A Node Having Two Children

- Consider the following example where node with value = 15 is deleted from the BST
- Method-1:
  - Visit to the right subtree of the deleting node.
  - Pluck the least value element called as inorder successor.
  - Replace the deleting element with its inorder successor.



- Method-2:
  - Visit to the left subtree of the deleting node.
  - Pluck the greatest value element called as inorder successor.
  - Replace the deleting element with its inorder successor.



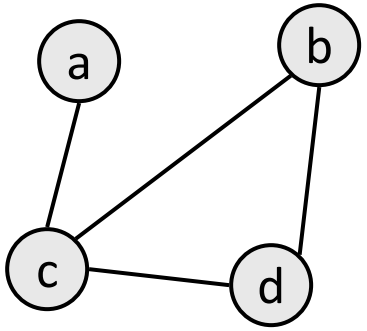
C code – BST deletion

```
//Function to find minimum in a tree.
Node* FindMin(Node* root)
{
        while(root->left != NULL) root = root->left;
        return root;
}

// Function to search a delete a value from tree.
struct Node* Delete(struct Node *root, int data) {
        if(root == NULL) return root;
        else if(data < root->data) root->left = Delete(root->left,data);
        else if (data > root->data) root->right = Delete(root->right,data);
        else {
                // Case 1:  No child
                if(root->left == NULL && root->right == NULL) {
                        free(root);
                        root = NULL;
                }
                //Case 2: One child
                else if(root->left == NULL) {
                        struct Node *temp = root;
                        root = root->right;
                        free(temp);
                }
                else if(root->right == NULL) {
                        struct Node *temp = root;
                        root = root->left;
                        free(temp);
                }
                // case 3: 2 children
                else {
                        struct Node *temp = FindMin(root->right);
                        root->data = temp->data;
                        root->right = Delete(root->right,temp->data);
                }
        }
        return root;
}
```

Graph

- **graph**: A data structure containing:
  - a set of **vertices** *V*, *(sometimes called nodes)*
  - a set of **edges** *E*, where an edge represents a connection between 2 vertices.
    - Graph *G* = (*V*, *E*)
    - an edge is a pair (*v*, *w*) where *v*, *w* are in *V*



- the graph at right:
  - *V* = {a, b, c, d}
  - *E* = {(a, c), (b, c), (b, d), (c, d)}

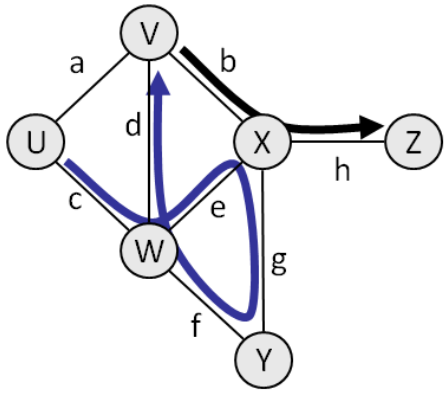
- **degree**: number of edges touching a given vertex.
  - at right: a=1, b=2, c=3, d=2

Graph Examples

Scenario	Vertices (Nodes)	Edges (Connections)
Web pages with links	Individual web pages	A hyperlink from one page to another (directed edge)
Methods in a program that call each other	Methods / functions	A method calling another method (directed edge)
Road maps (e.g., Google Maps)	Intersections, cities, or geographic points	Roads connecting them (edges may be weighted by distance/time)
Airline routes	Airports	Direct flights between airports (typically directed edges, sometimes bidirectional)
Facebook friends	Users (people)	Friendship connections (undirected edges)
Course prerequisites	Courses	A prerequisite relationship: Course A → Course B (directed edge)
Family trees	Individuals	Parent → child relationships (directed edges)
Paths through a maze	Maze cells or intersections	Passages between cells (undirected edges)

Paths

- A **path** from vertex **a** to vertex **b** is a sequence of edges that can be followed starting at **a** and ending at **b**.
  - A path may be represented by the **vertices visited** or by the **edges taken**.
  - Example: one path from **V** to **Z** is **{b, h}** or equivalently **{V, X, Z}**.
  - **Question:** What are *two* possible paths from **U** to **Y**?
- The **path length** is the number of **vertices** or **edges** contained in the path.
- Two vertices are **neighbors** (or **adjacent**) if they are connected directly by an edge.
  - Example: **V** and **X** are adjacent.



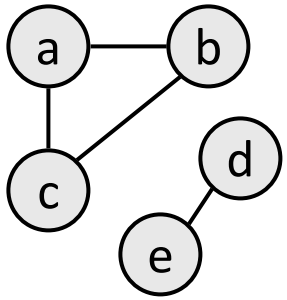
Graph Properties

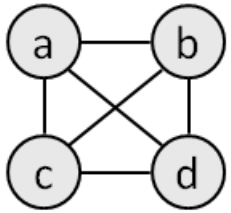
Reachable

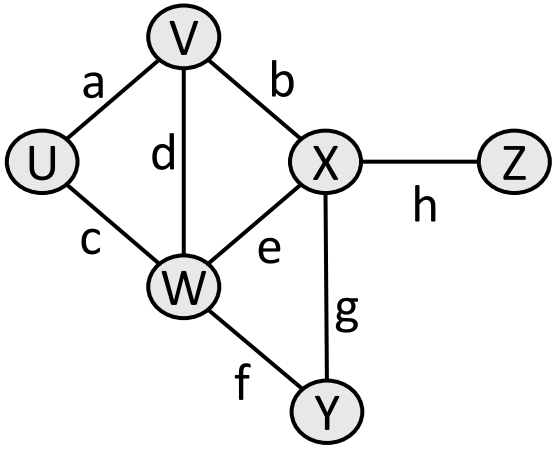
- A vertex **a** is *reachable* from vertex **b** if there exists a path from **b** to **a**.

Connected

- A graph is *connected* if **every vertex is reachable from every other vertex**.
  - Question: *Is the 3rd graph connected?*
- **Strongly Connected**
- A graph is *strongly connected* when **every vertex has a path to every other vertex** (i.e., mutual reachability in directed graphs).

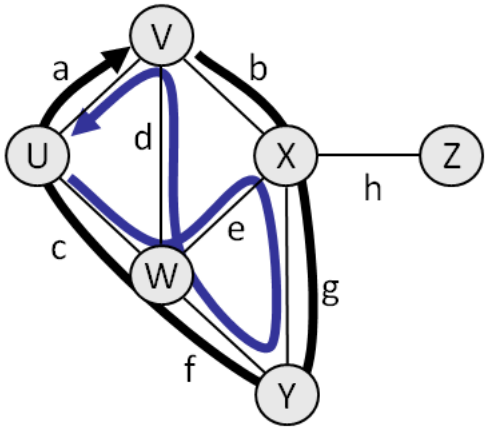






Graph Properties

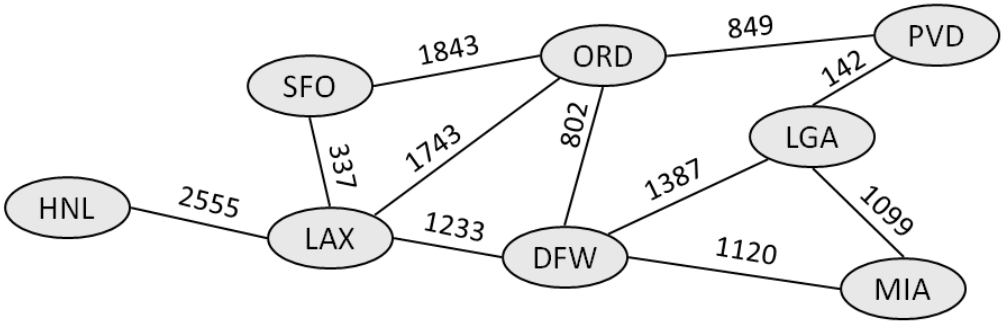
- **cycle**: A path that begins and ends at the same node.
  - example: {b, g, f, c, a} or {V, X, Y, W, U, V}.
  - example: {c, d, a} or {U, W, V, U}.
  - **acyclic graph**: One that does not contain any cycles.



- **loop**: An edge directly from a node to itself.
  - Many graphs don't allow loops.

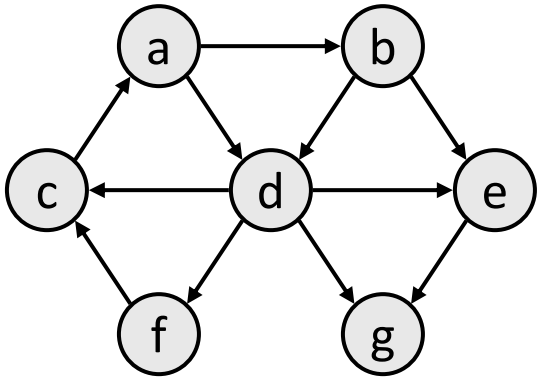
Graph Properties

- The **weight** of an edge is the **cost** associated with traversing that edge.
- Some graphs have **weighted** edges, while others are **unweighted**.
- Edges in an unweighted graph can be treated as having **equal weight** (e.g., all 0 or all 1).
- Most graph algorithms do **not** allow negative edge weights.
- **Example :** A graph of **airline flights**, where each edge is weighted by the **number of miles** between the cities.



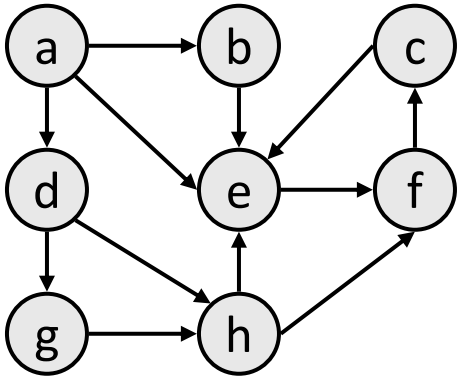
Graph Properties

- **directed graph** ("digraph"): One where edges are *one-way* connections between vertices.
  - If graph is directed, a vertex has a separate in/out degree.
  - A digraph can be weighted or unweighted.
  - Is the graph below connected? Why or why not?



Depth-first Search (DFS)

- DFS finds a path between two vertices by exploring each possible route **as far as possible before backtracking**.
- Often implemented **recursively**.
- Many graph algorithms rely on **visiting** or **marking** vertices during traversal.
- **Depth-first paths from *a* to all vertices**
  - **to b:** {a, b}
  - **to c:** {a, b, e, f, c}
  - **to d:** {a, d}
  - **to e:** {a, b, e}
  - **to f:** {a, b, e, f}
  - **to g:** {a, d, g}
  - **to h:** {a, d, g, h}



DFS Pseudo code

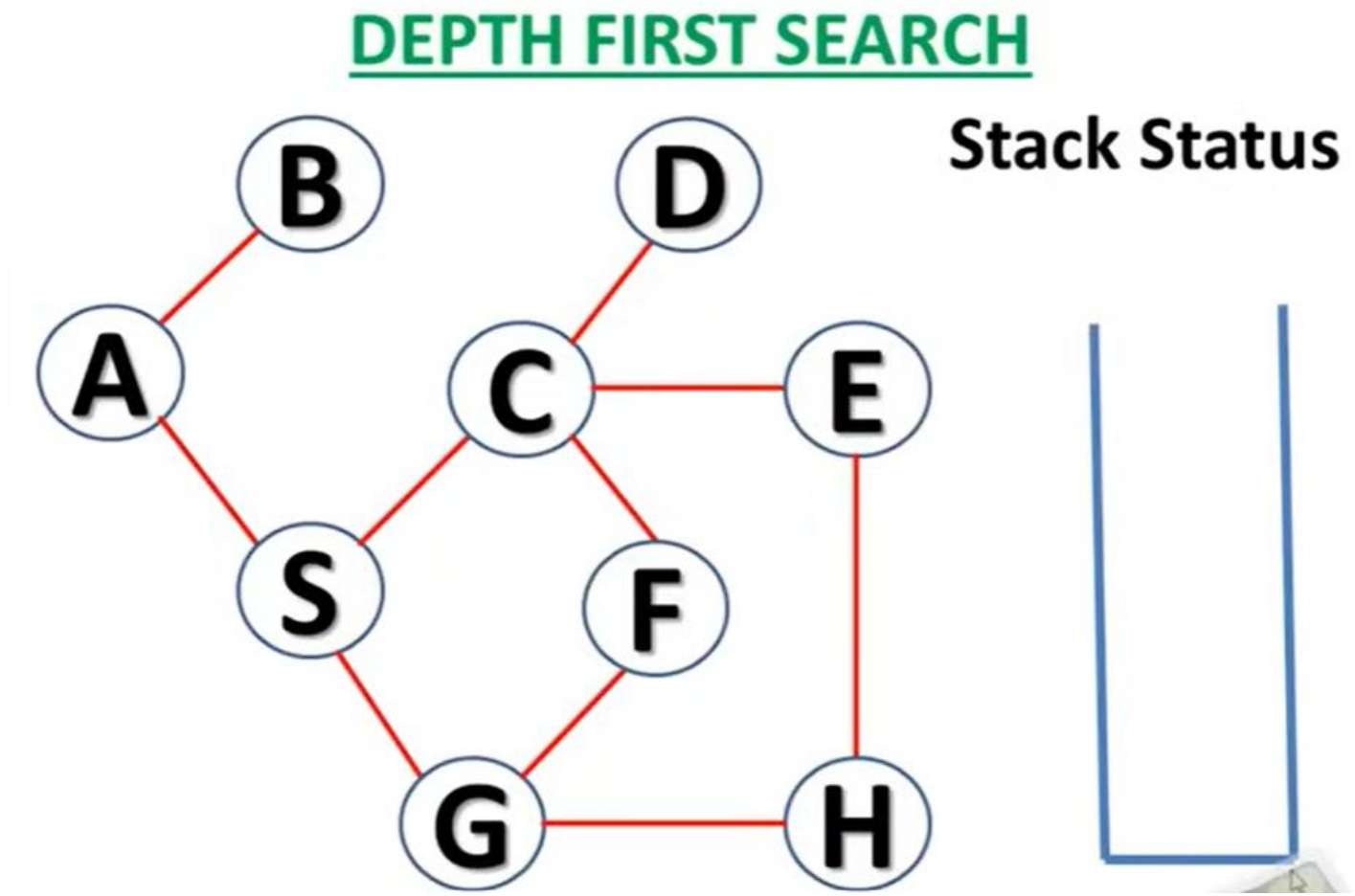
function **dfs**(v_1, v_2):
    dfs(v_1, v_2, { }).

function **dfs**(v_1, v_2, *path*):
    path += v_1.
    mark v_1 as visited.
    if v_1 is v_2:
        a path is found!

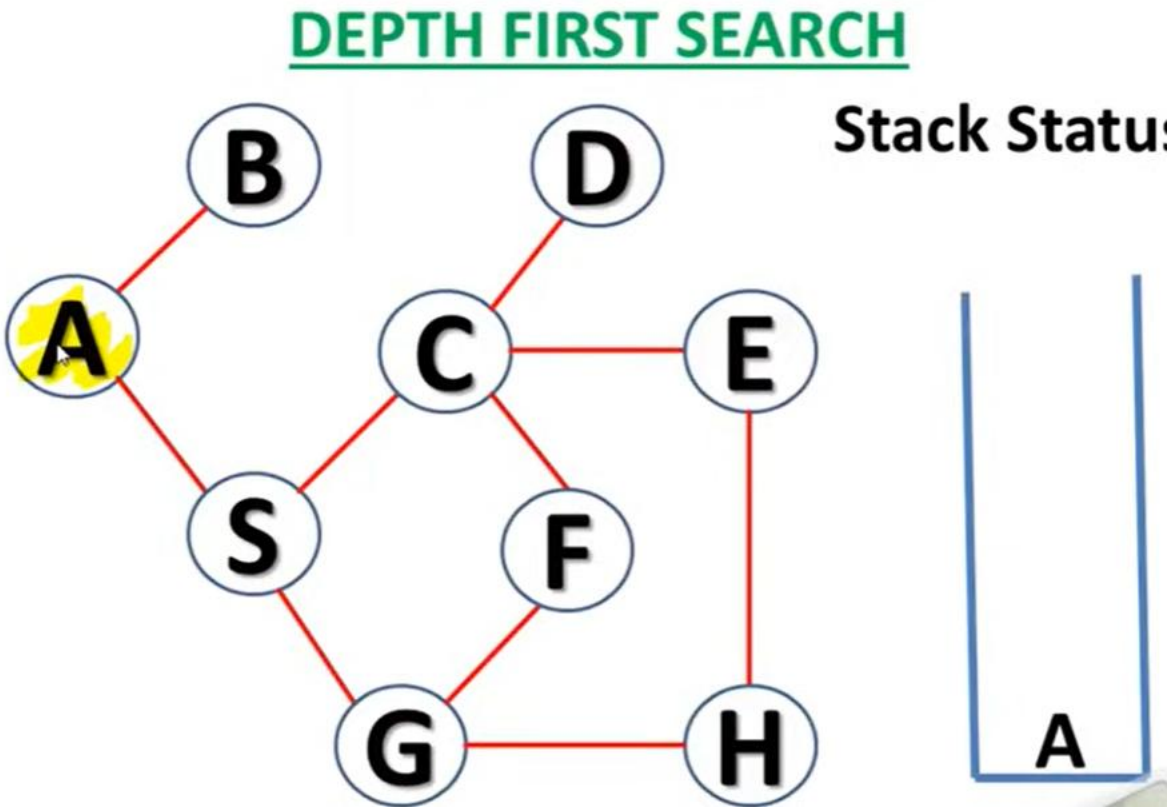
    for each unvisited neighbor *n* of v_1:
        if dfs(*n*, v_2, *path*) finds a path: a path is found!

    path -= v_1. // path is not found.

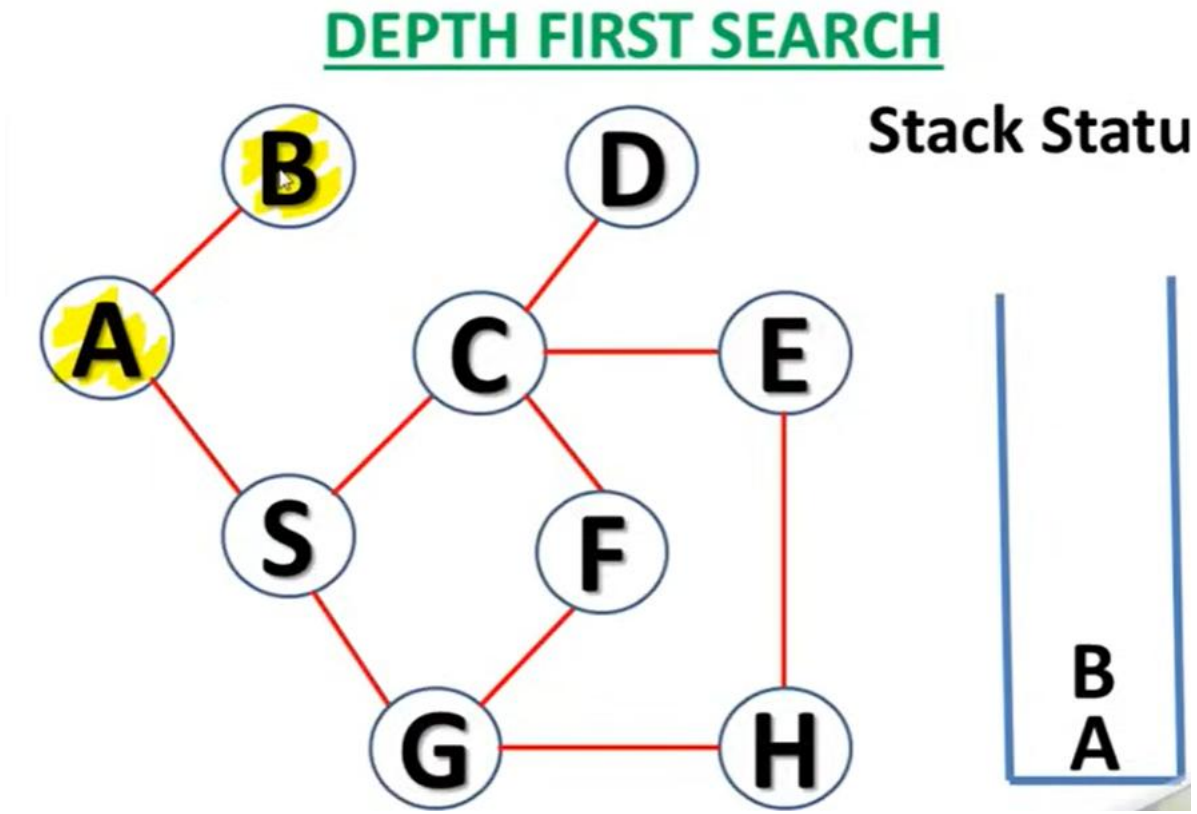
- The *path* param above is used if you want to have the path available as a list once you are done.
  - Trace dfs(*a*, *f*) in the above graph.



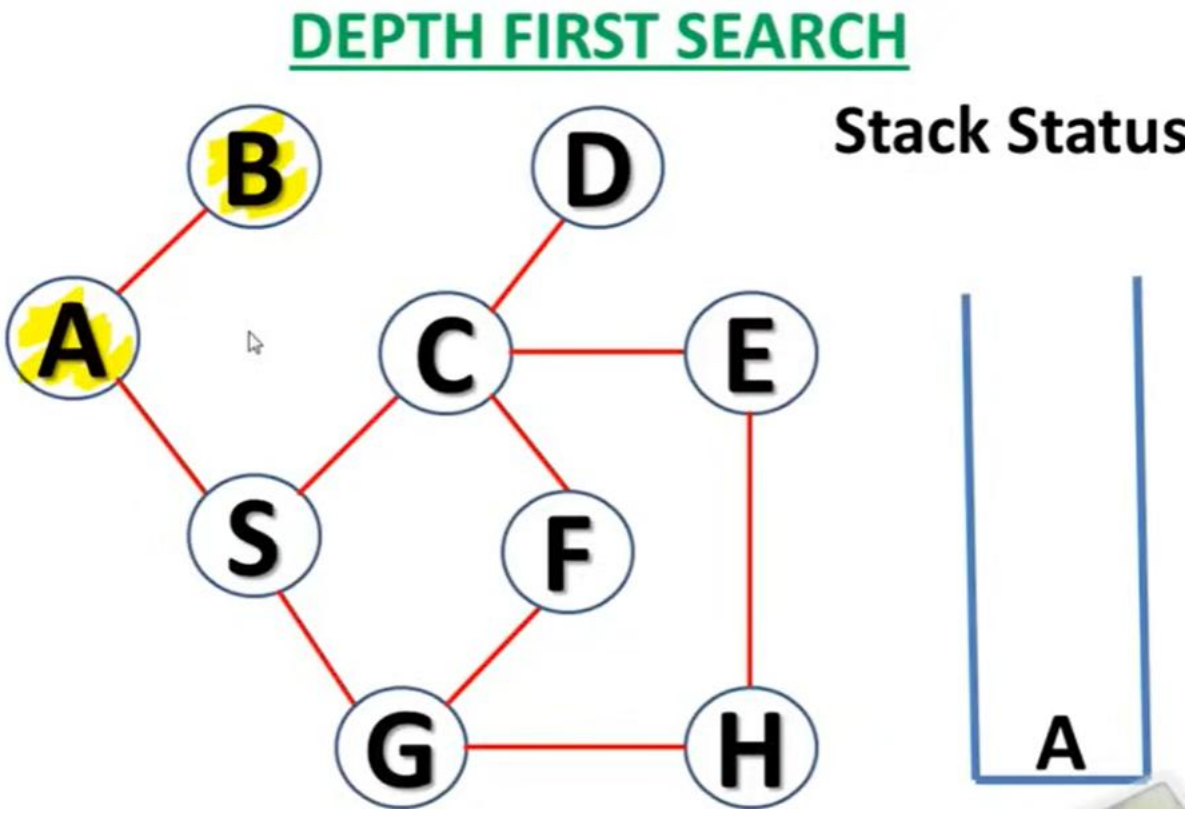
- OUTPUT:



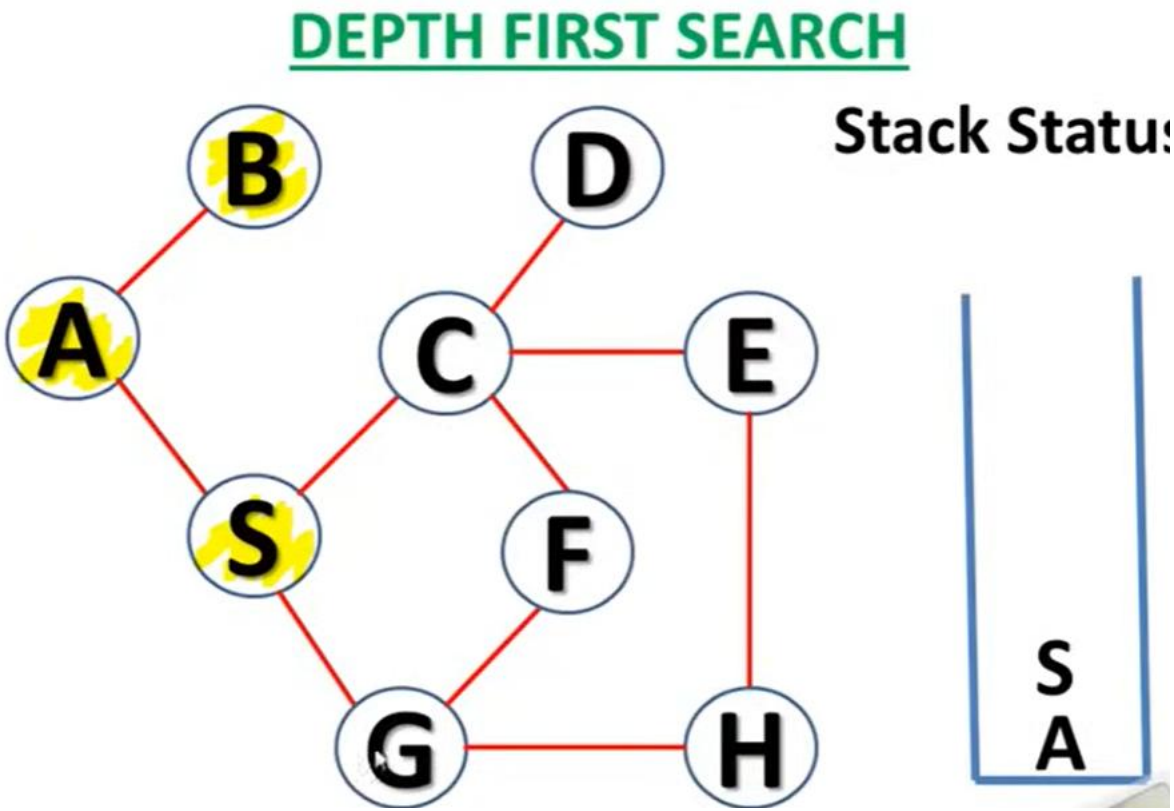
- OUTPUT: A



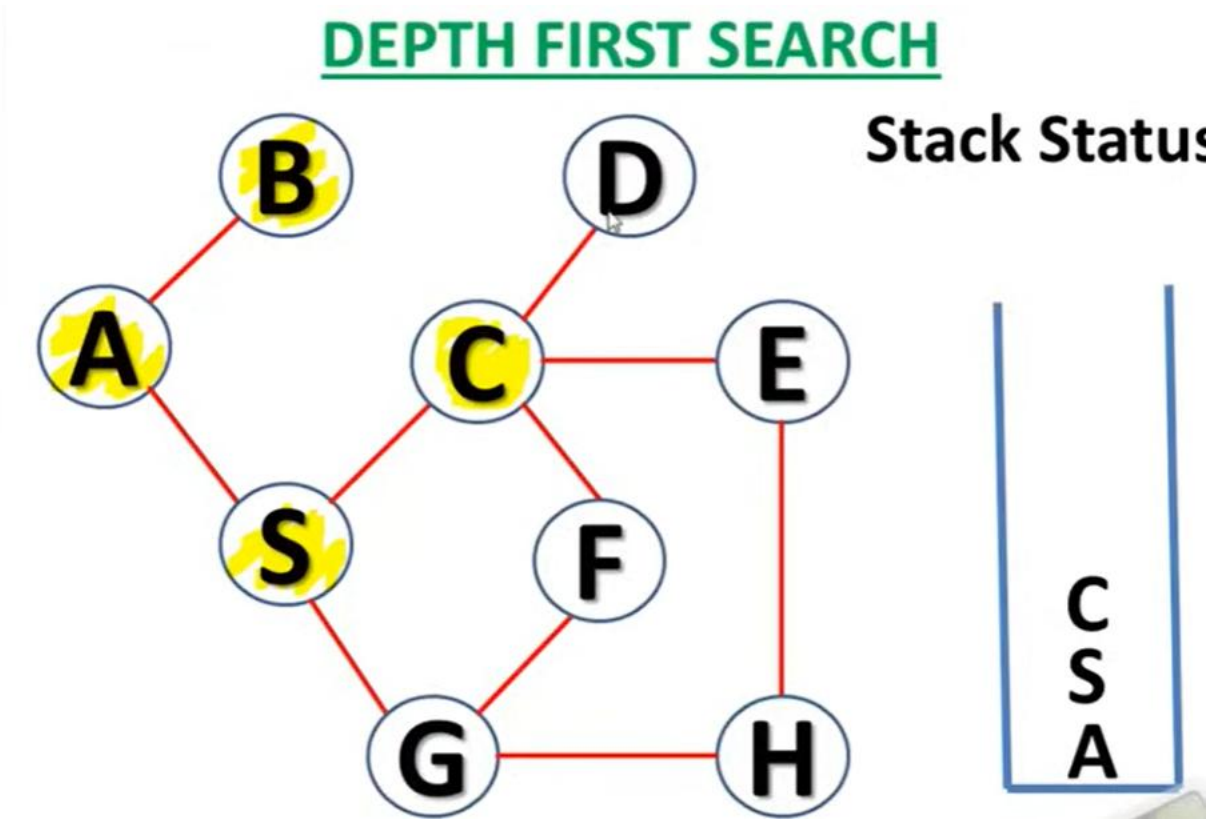
- OUTPUT: A B



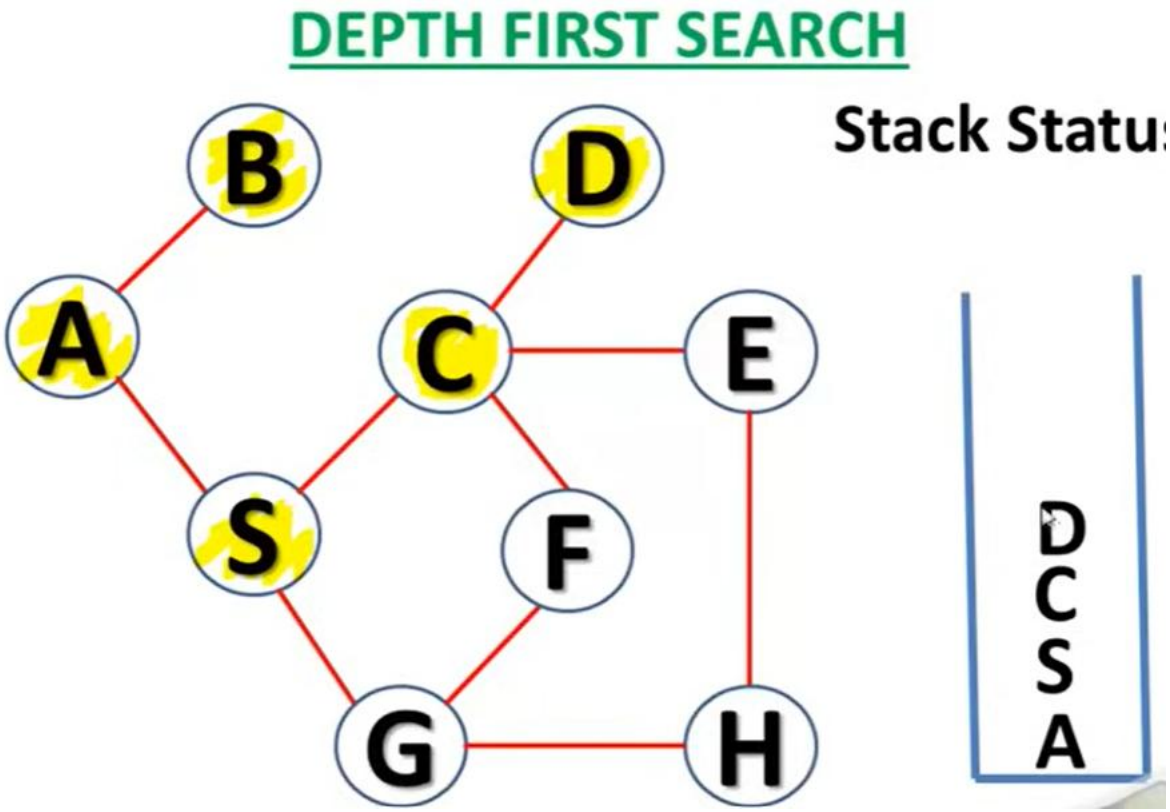
- OUTPUT: A B



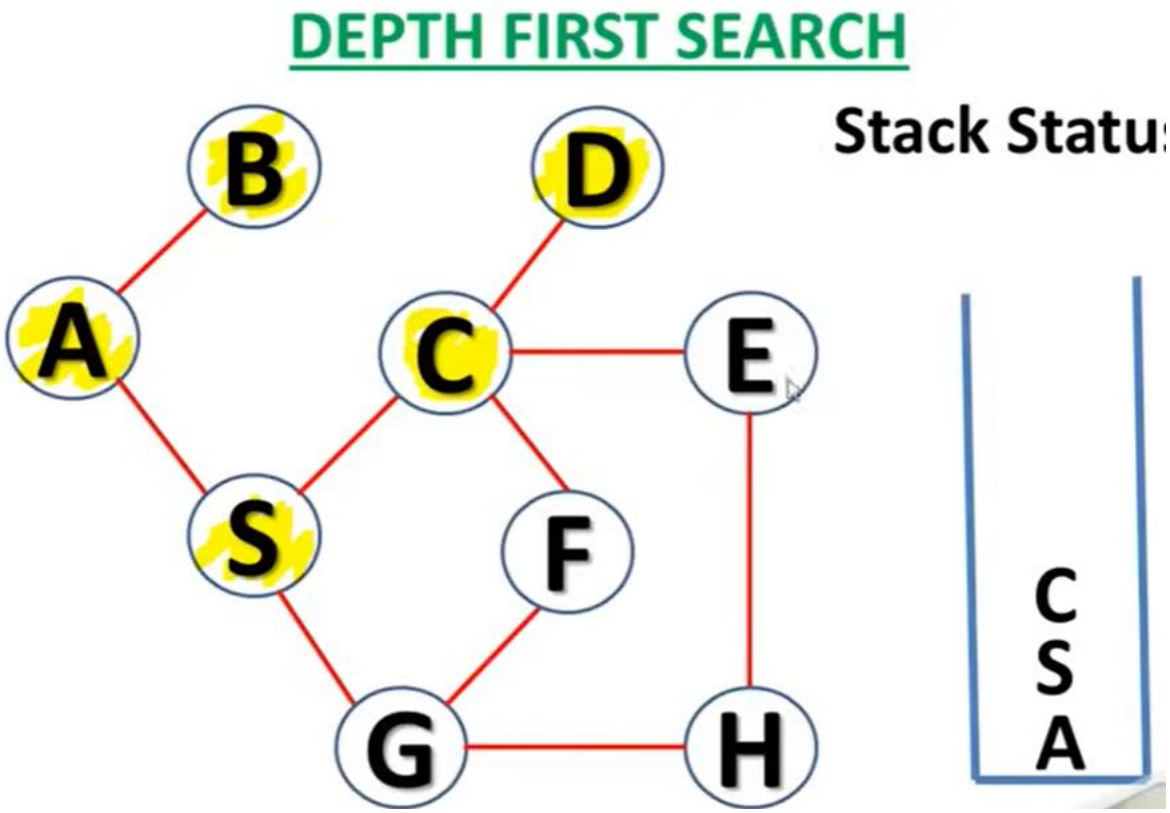
- OUTPUT: A B S



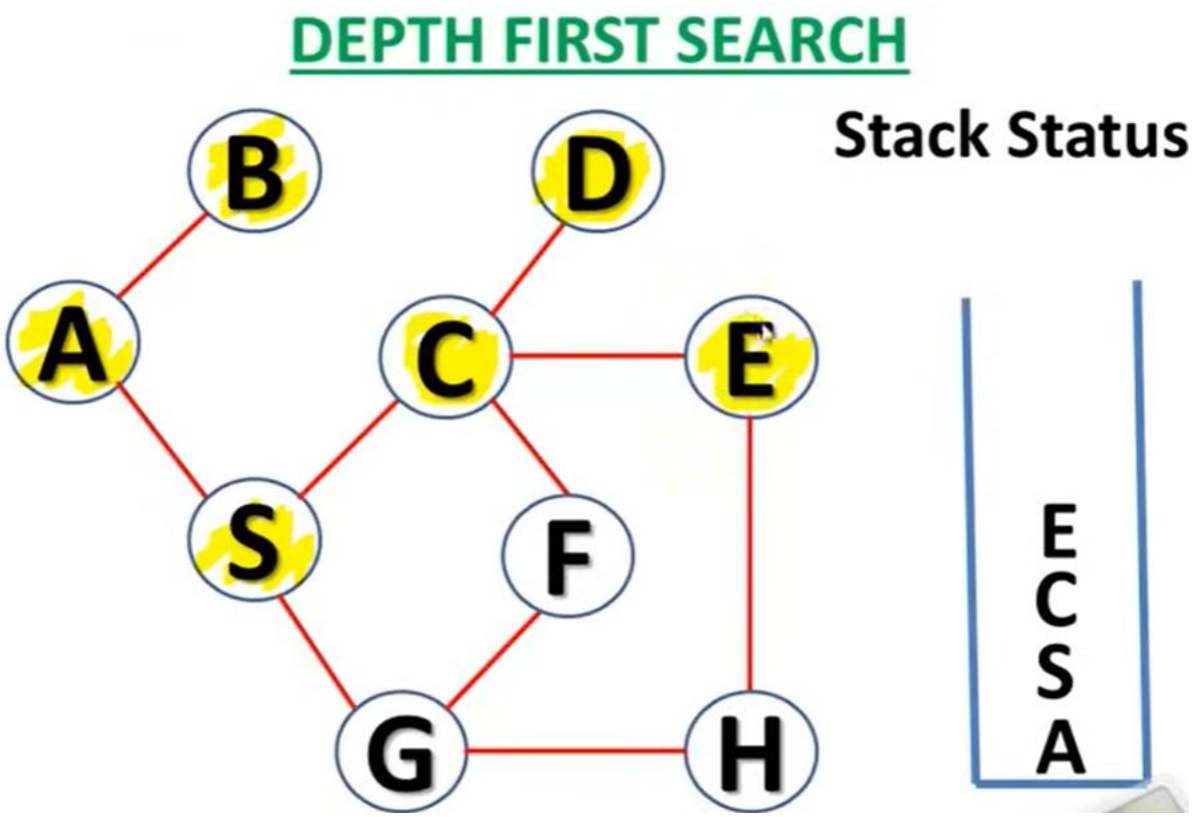
- OUTPUT: A B S C



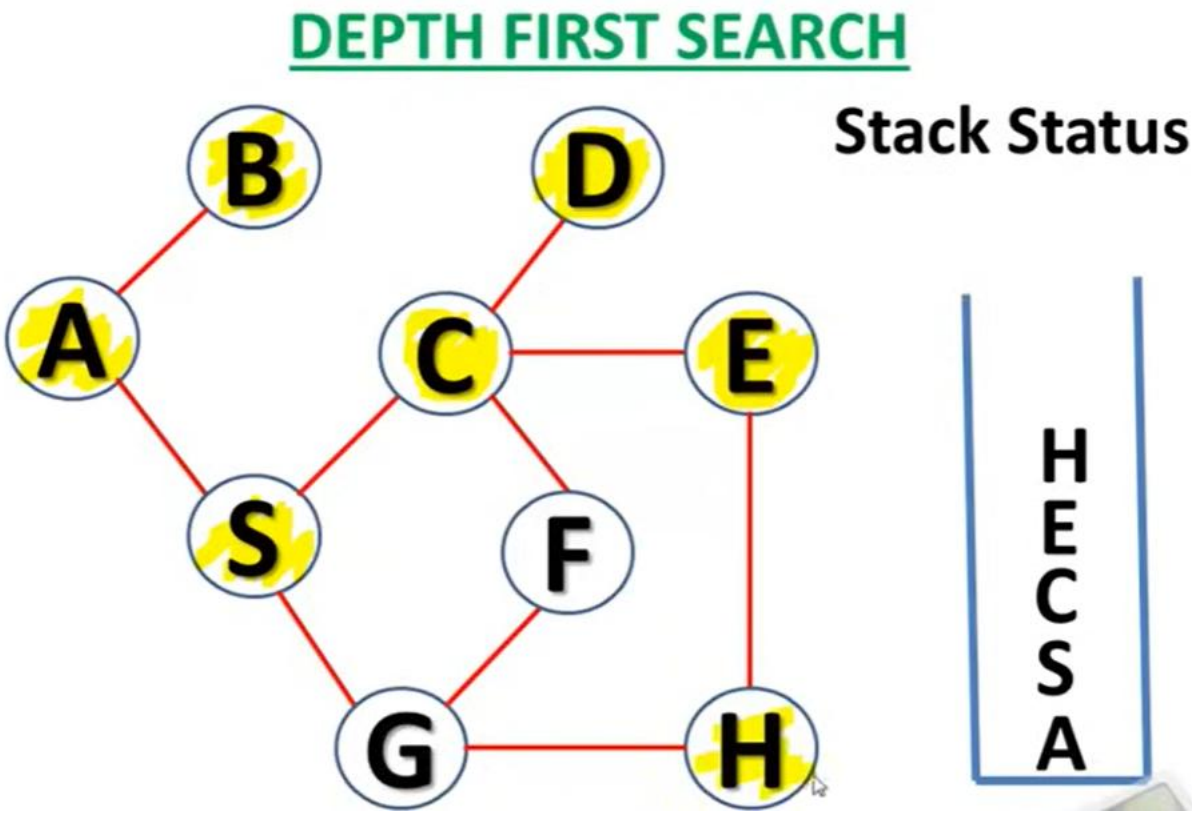
- OUTPUT: A B S C D



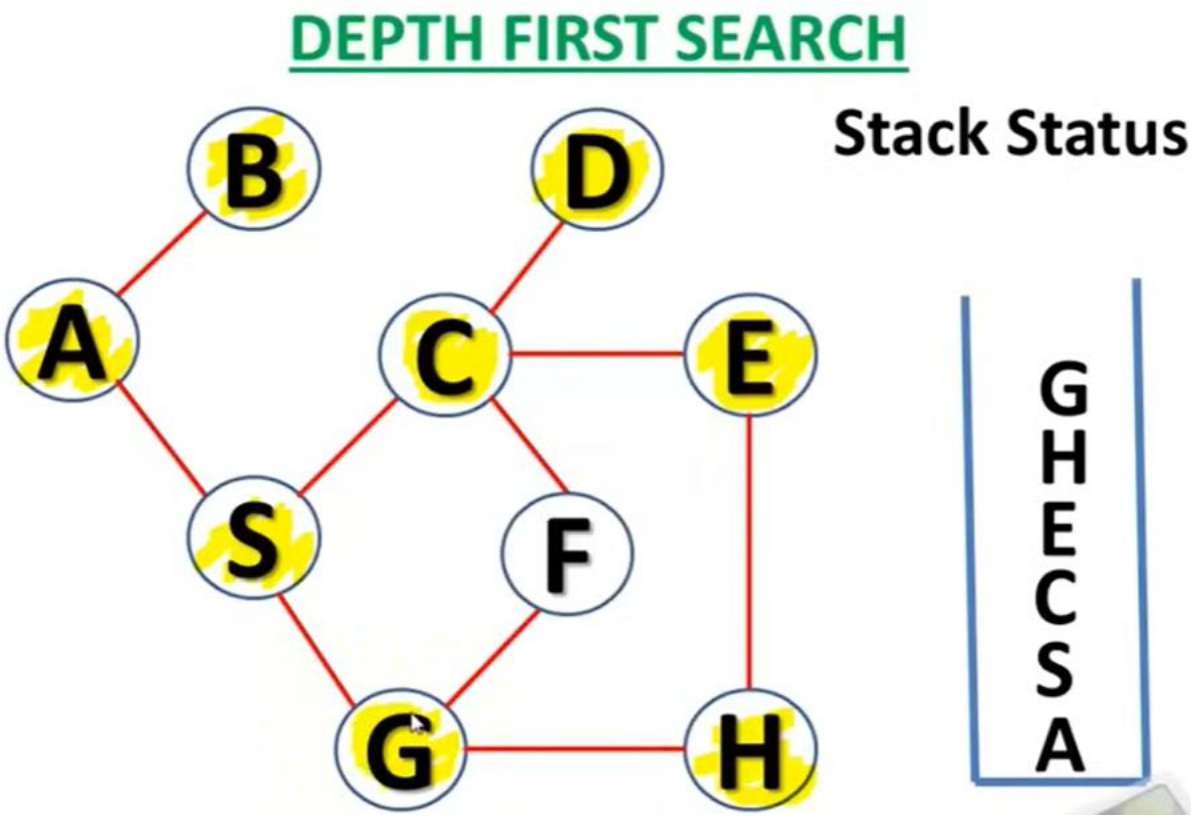
- OUTPUT: A B S C D



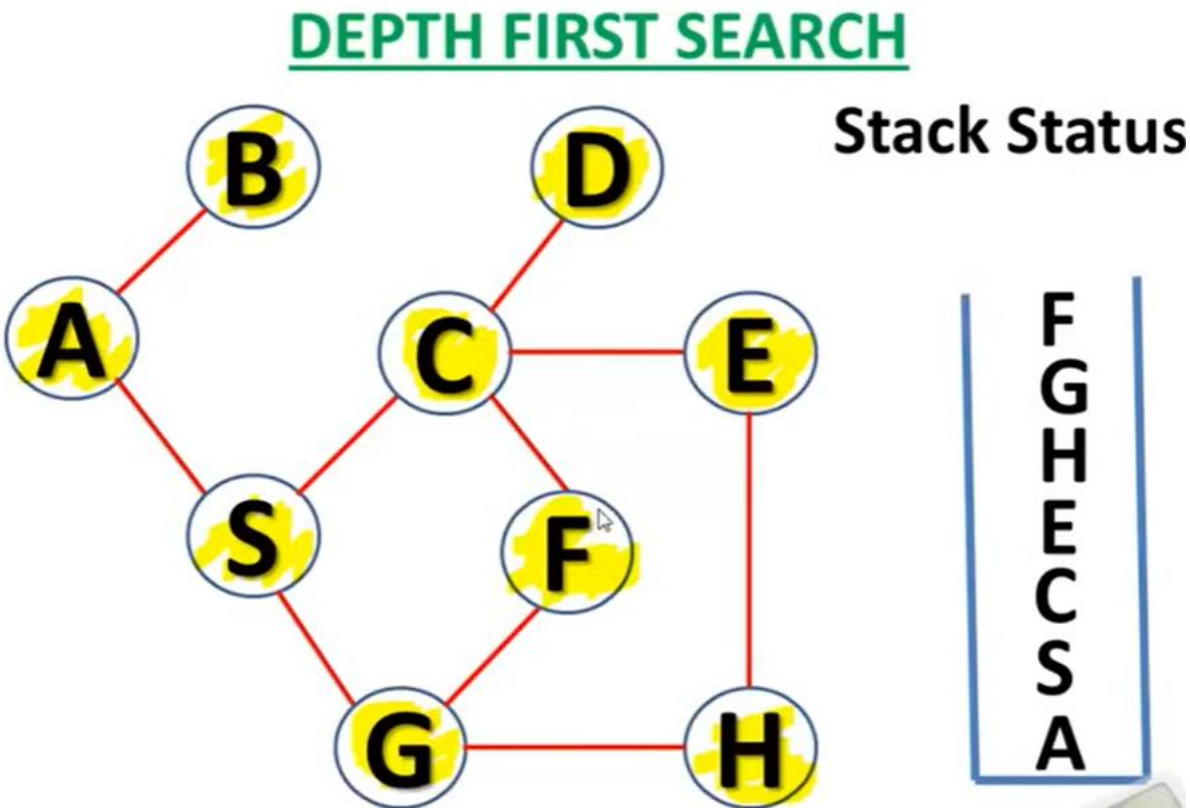
- OUTPUT: A B S C D E



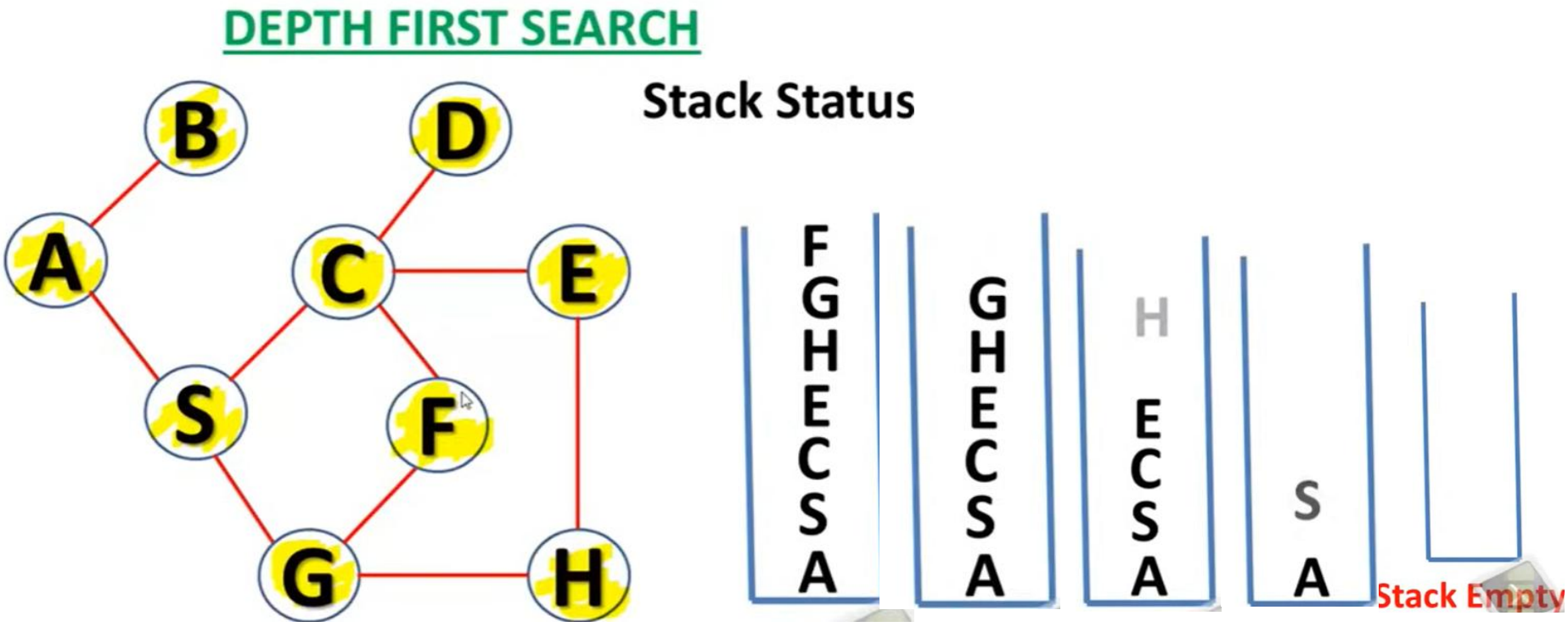
- OUTPUT: A B S C D E H



- OUTPUT: A B S C D E H G



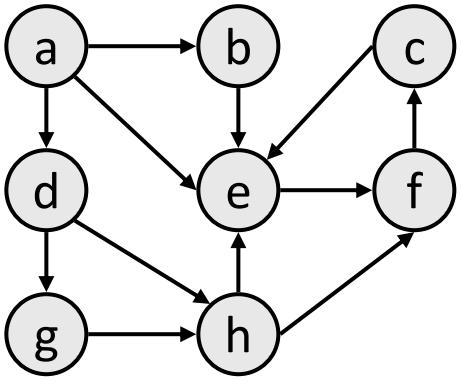
- OUTPUT: A B S C D E H G F



- OUTPUT: A B S C D E H G F

DFS Observations

- *discovery*: DFS is guaranteed to find *a* path if one exists.



- *retrieval*: It is easy to retrieve exactly what the path is (the sequence of edges taken) if we find it

- *optimality*: not optimal. DFS is guaranteed to find a path, not necessarily the best/shortest path
  - Example: dfs(a, f) returns {a, d, h, f} rather than {a, e, f}.

DFS Examples

Graph connection using DFS

```
#include<stdio.h>
#include<conio.h>
int a[1][10];
void dfs(int n, int cost[10][10], int u, int s[])
{
        int v;
        s[u]=1;
        for(v=0;v<n;v++)
        {
                if((cost[u][v]==1) && (s[v]==0))
                        dfs(n,cost,v,s);
        }
}
```

```
void main()
{
		int n,i,j,cost[10][10],s[10],con,flag;
		clrscr();
		printf("Enter the number of nodes\n");
		scanf("%d", &n);
		printf("Enter the adjacency matrix\n");
		for(i=0;i<n;i++)
		{
				for(j=0;j<n;j++)
						scanf("%d", &cost[i][j]);
		}
		con=0;
		for(j=0;j<n;j++)
		{
				for(i=0;i<n;i++)
				s[i]=0;
				dfs(n,cost,j,s);
				flag=0;
				for(i=0;i<n;i++)
				{
						if(s[i]==0)
								flag=1;
				}
				if(flag==0)
						con=1;
		}
		if(con==1)
				printf("Graph is connected\n");
		else
				printf("Graph is not connected\n");
		getch();
}
```

```c
#include <stdio.h>
#include <stdlib.h>

void dfs(int n, int cost[][10], int u, int s[]) {
    int v;
    s[u] = 1;
    for (v = 0; v < n; v++) {
        if (cost[u][v] == 1 && s[v] == 0)
            dfs(n, cost, v, s);
    }
}

int main(void) {
    int n, i, j, cost[10][10], s[10];
    printf("Enter number of nodes: ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > 10) return 1;

    printf("Enter adjacency matrix (%d x %d):\n", n, n);
    for (i = 0; i < n; ++i)
        for (j = 0; j < n; ++j)
            scanf("%d", &cost[i][j]);

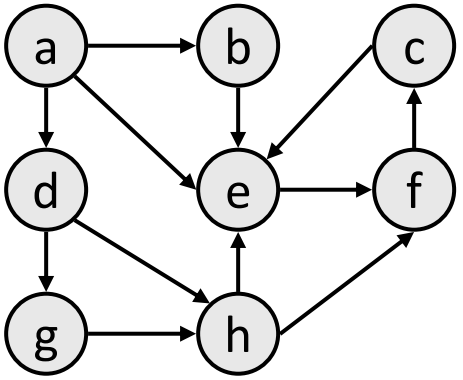
    for (i = 0; i < n; ++i) s[i] = 0;
    dfs(n, cost, 0, s);  // start from node 0

    for (i = 0; i < n; ++i) {
        if (s[i] == 0) {
            printf("Graph is not connected\n");
            return 0;
        }
    }
```

Breadth-first Search (BFS)

- **breadth-first search** (BFS): Finds a path between two nodes by taking one step down all paths and then immediately backtracking.
  - Often implemented by maintaining a queue of vertices to visit.

- BFS always returns the shortest path (the one with the fewest edges) between the start and the end vertices.
  - to b: {a, b}
  - to c: **{a, e, f, c}**
  - to d: {a, d}
  - to e: **{a, e}**
  - to f: **{a, e, f}**
  - to g: {a, d, g}
  - to h: **{a, d, h}**



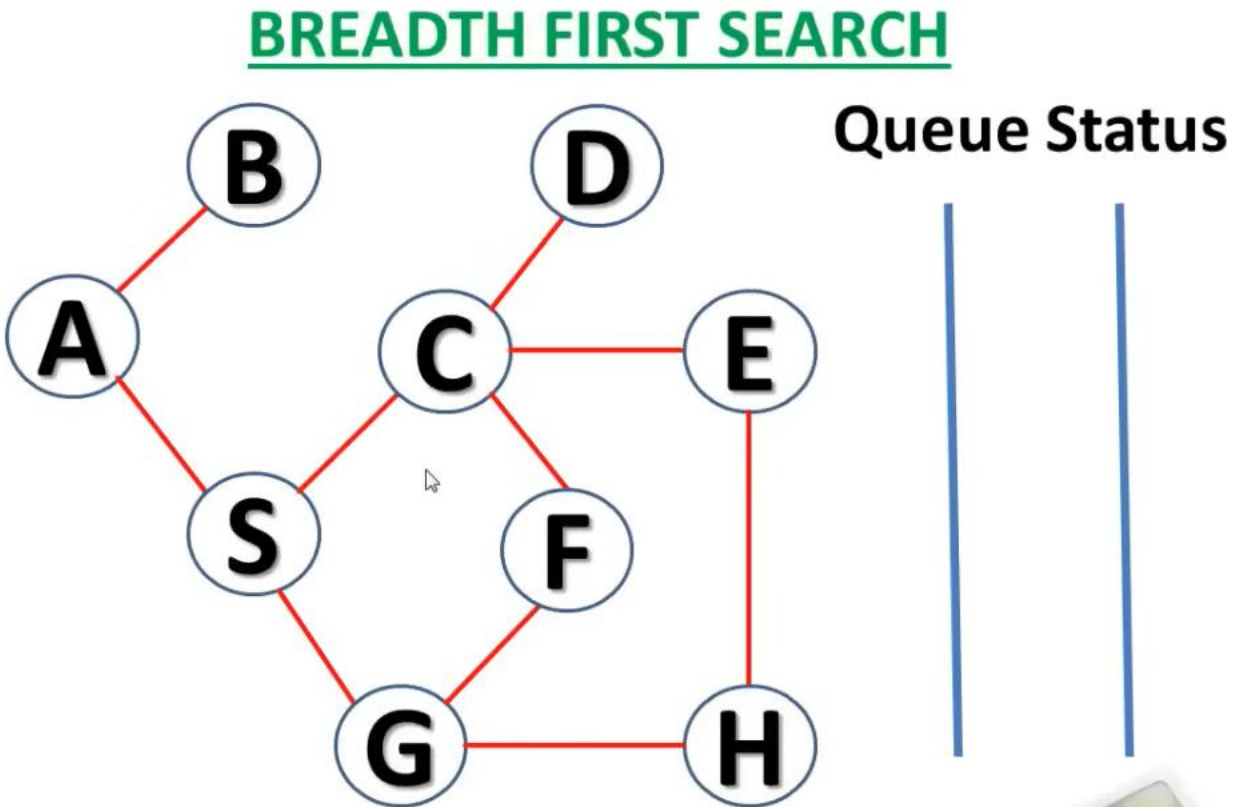
BFS Pseudocode

```
function bfs(v1, v2):
    queue := {v1}.
    mark v1 as visited.

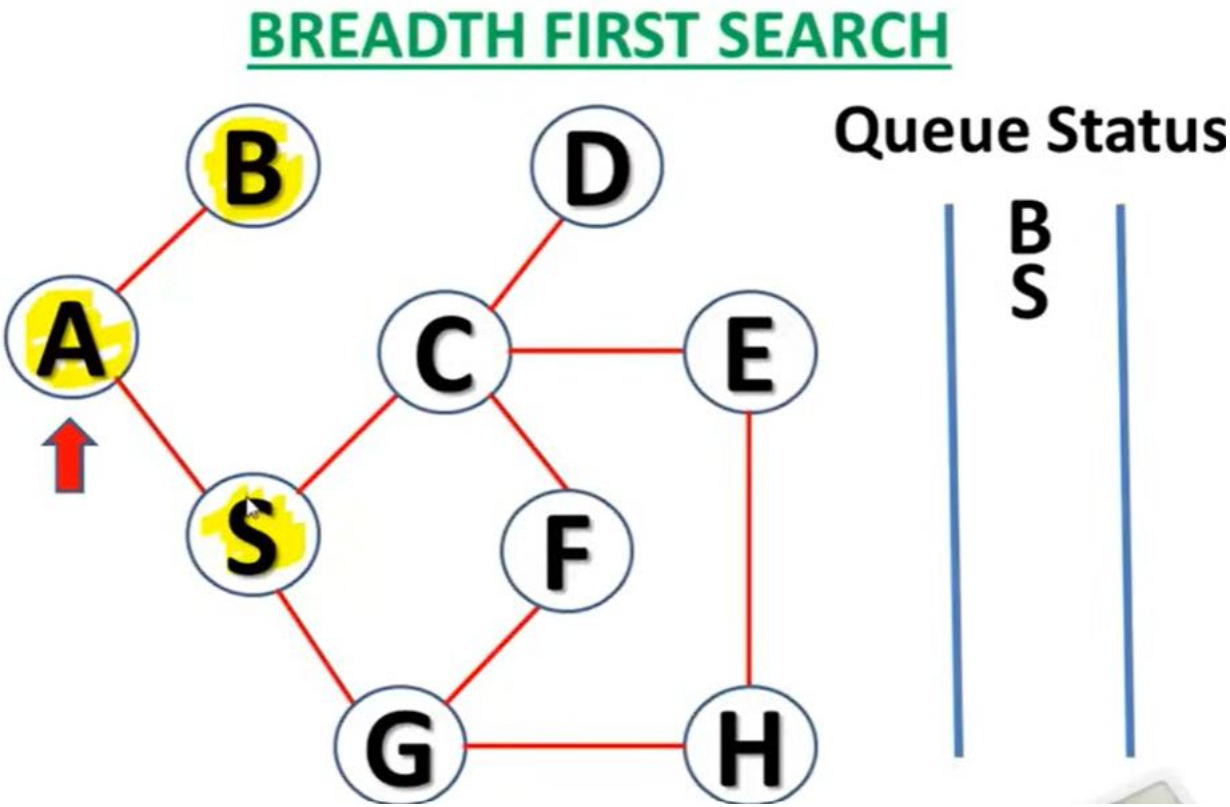
    while queue is not empty:
        v := queue.removeFirst().
        if v is v2:
            a path is found!

        for each unvisited neighbor n of v:
            mark n as visited.
            queue.addLast(n).

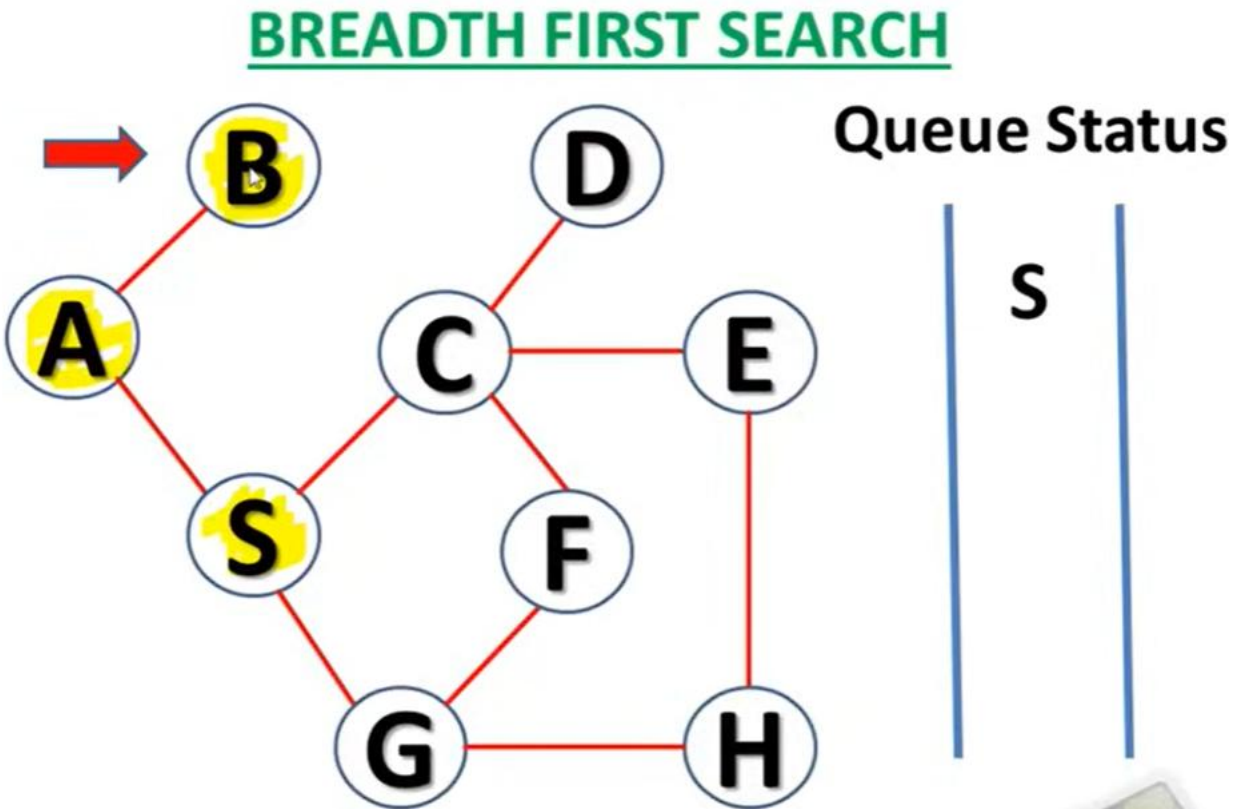
    // path is not found.
```



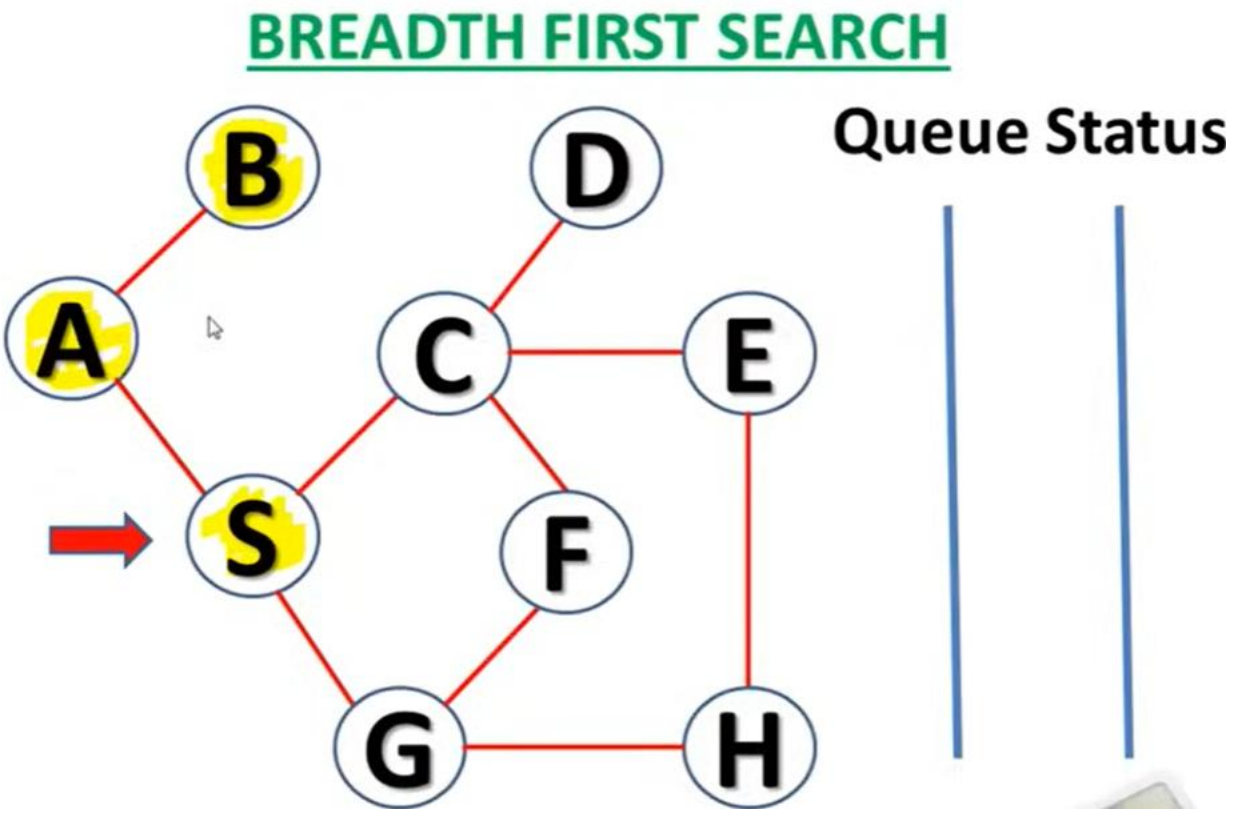
- OUTPUT:



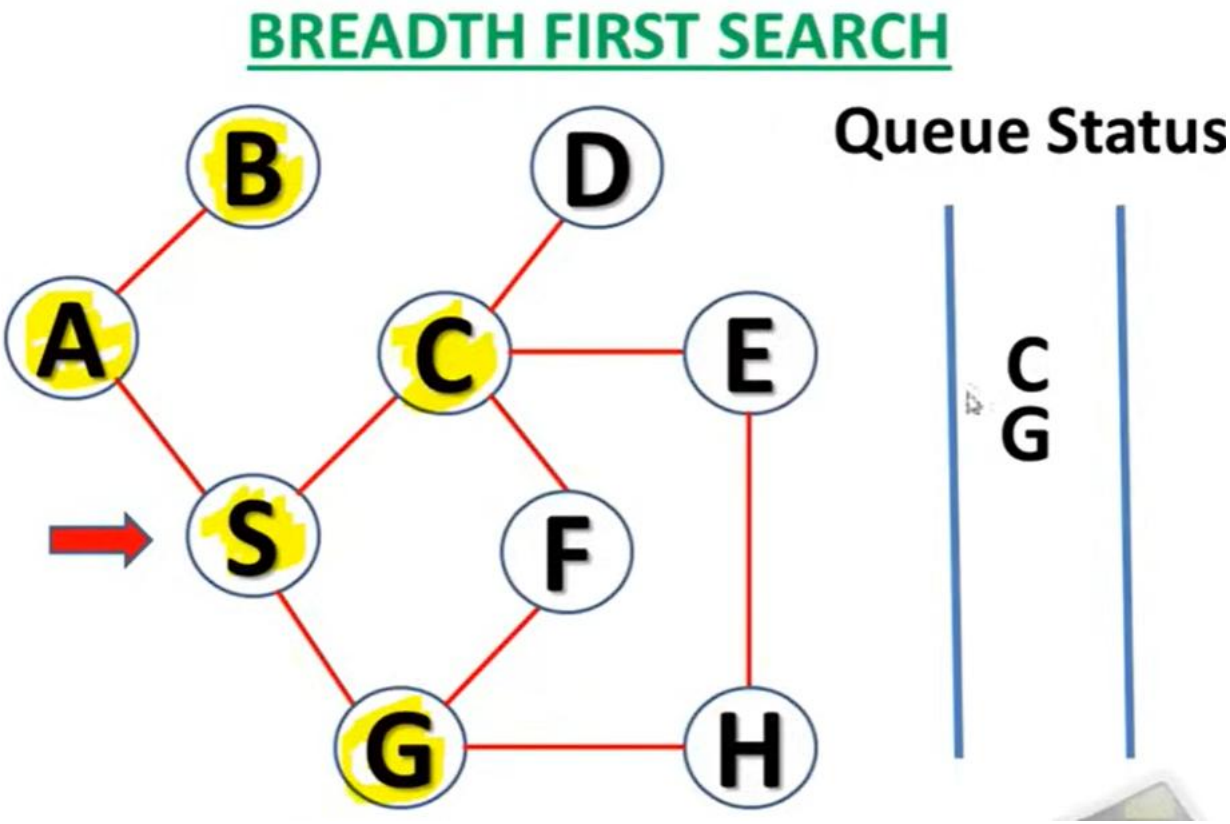
- OUTPUT: A B S



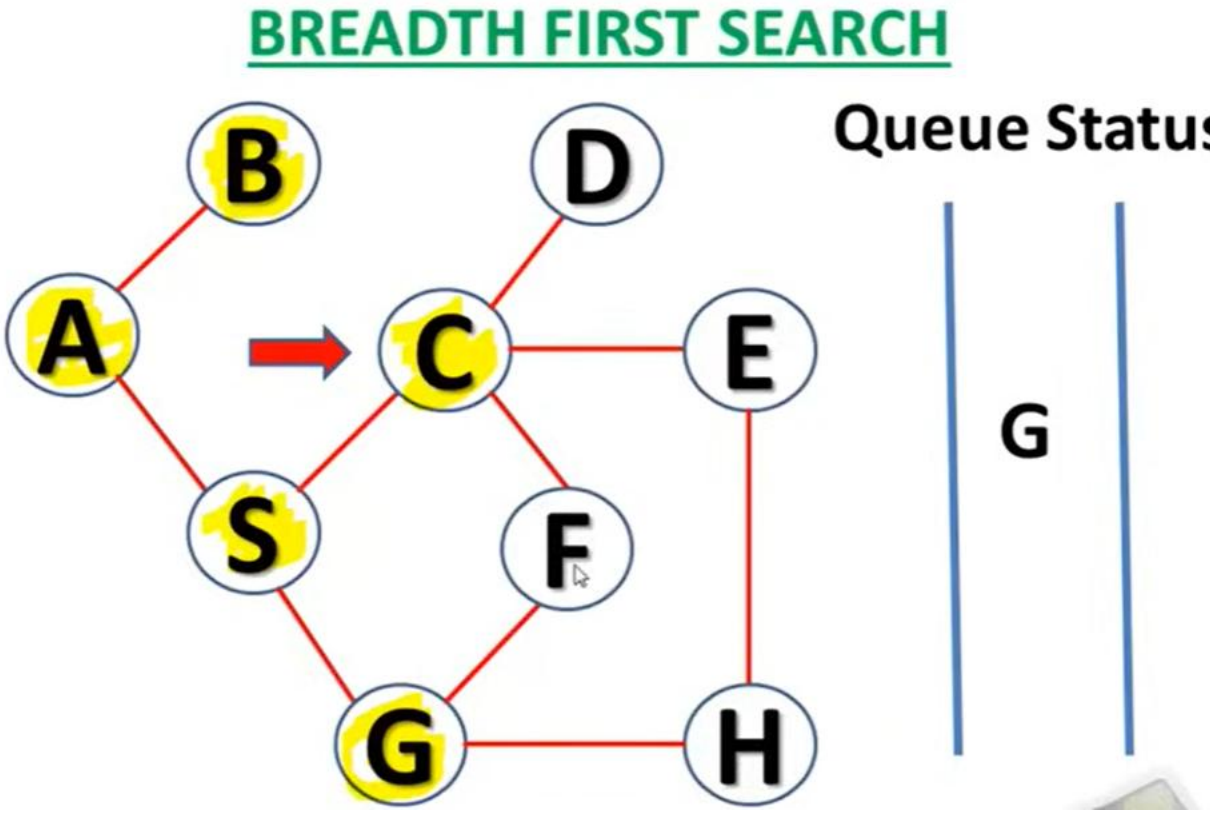
- OUTPUT: A B S



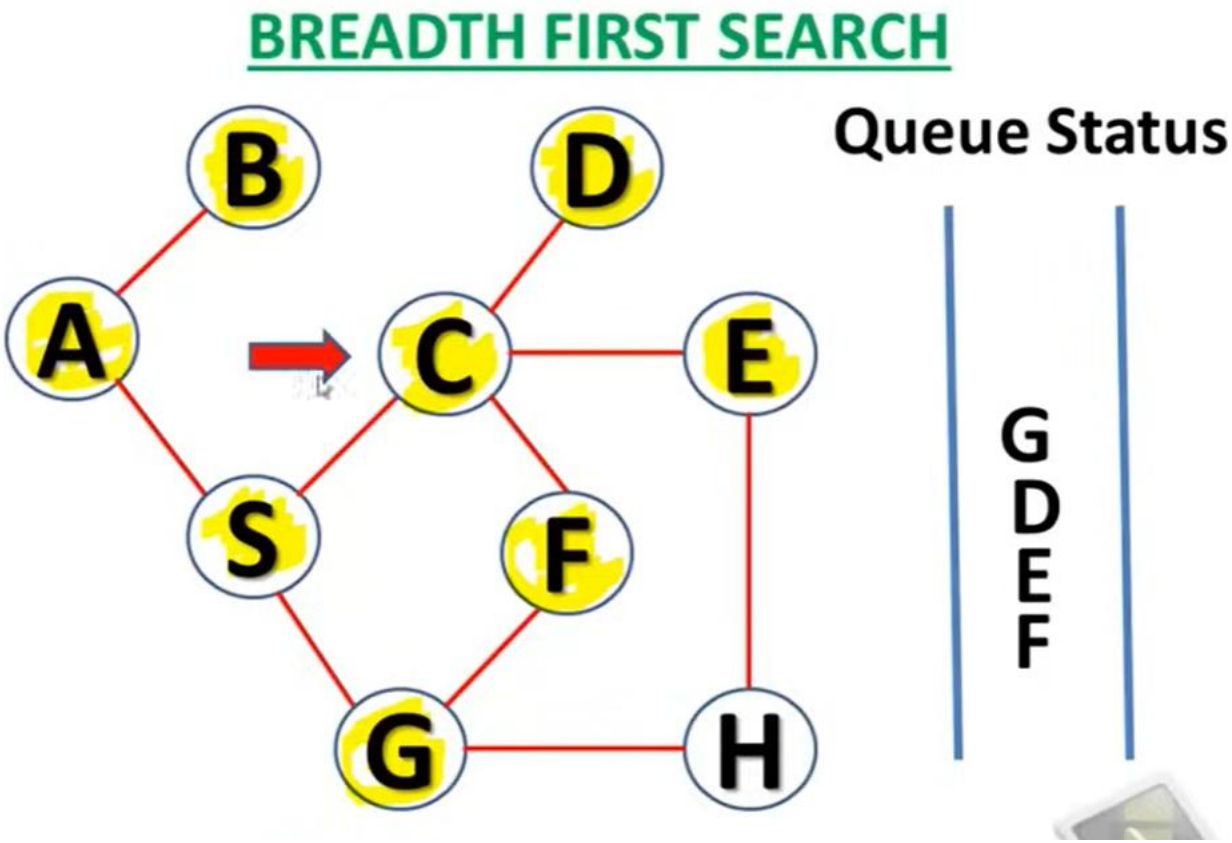
- OUTPUT: A B S



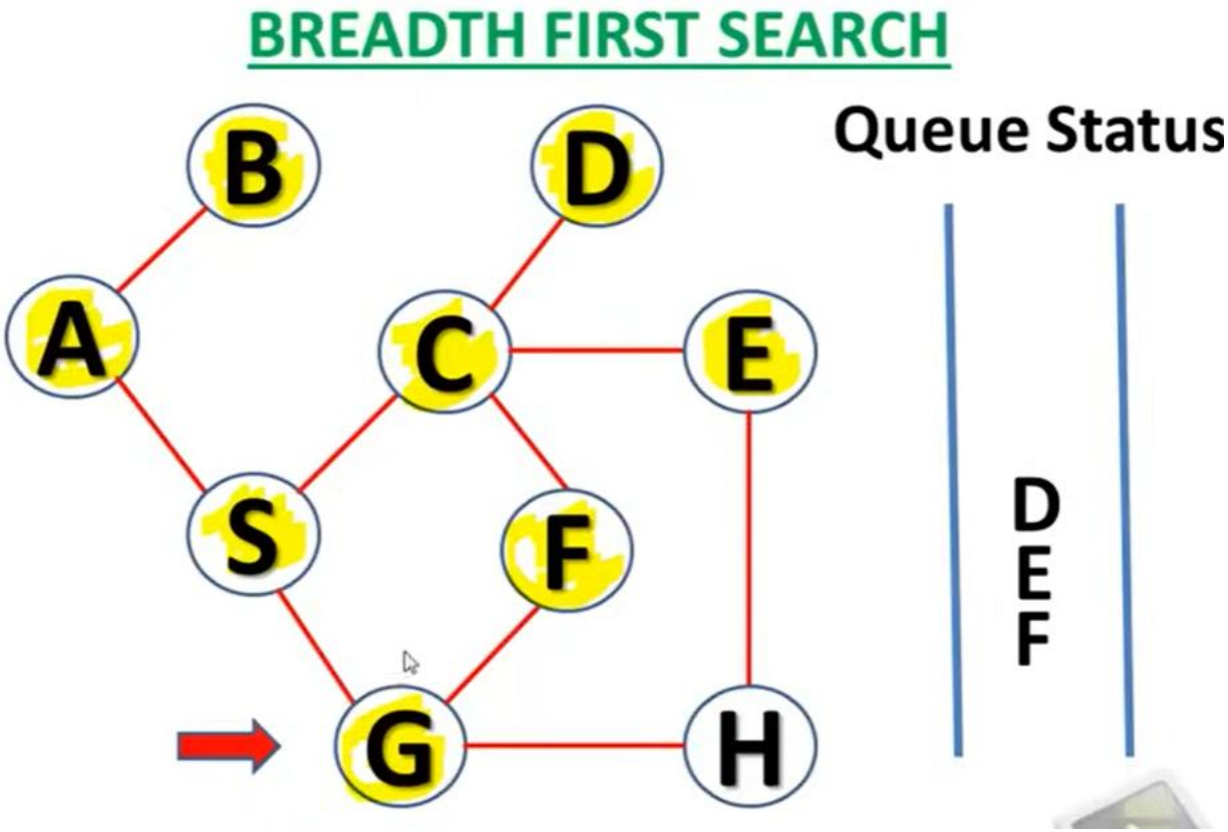
- OUTPUT: A B S C G



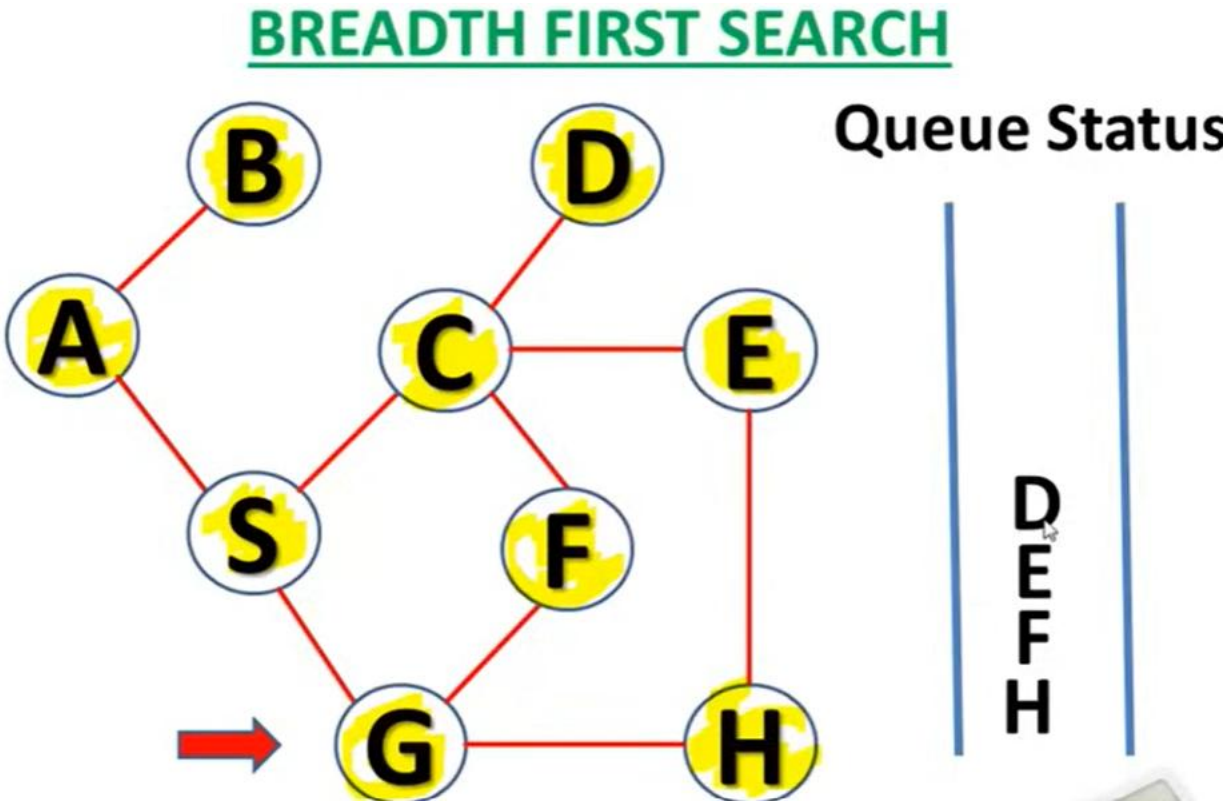
- OUTPUT: A B S C G



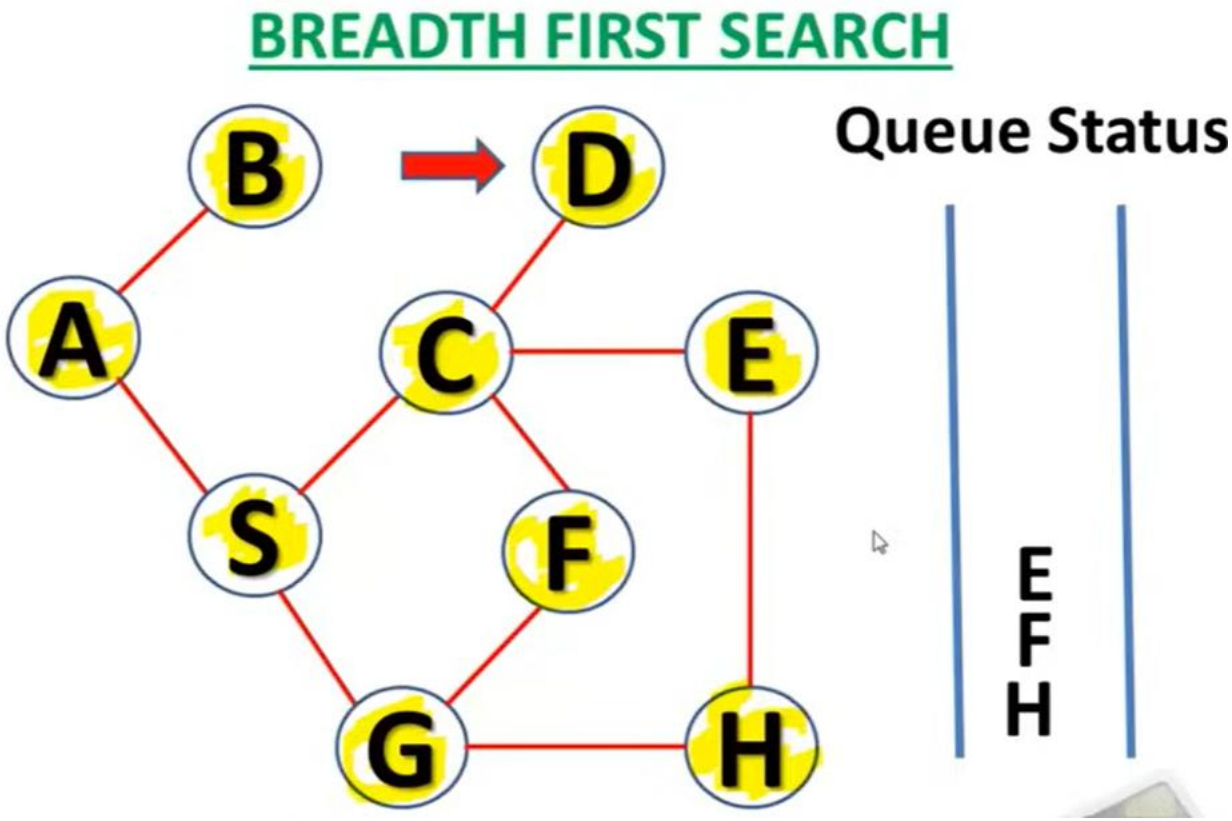
- OUTPUT: A B S C G D E F



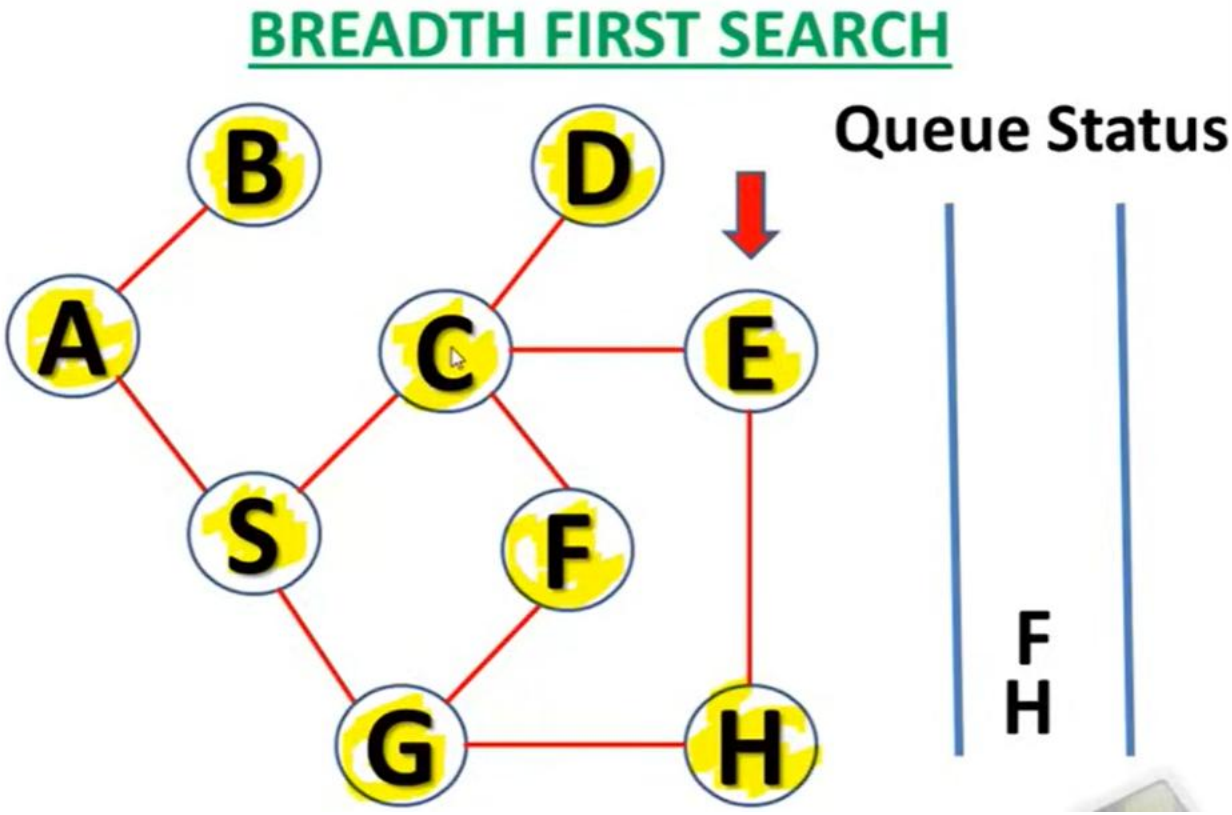
- OUTPUT: A B S C G D E F



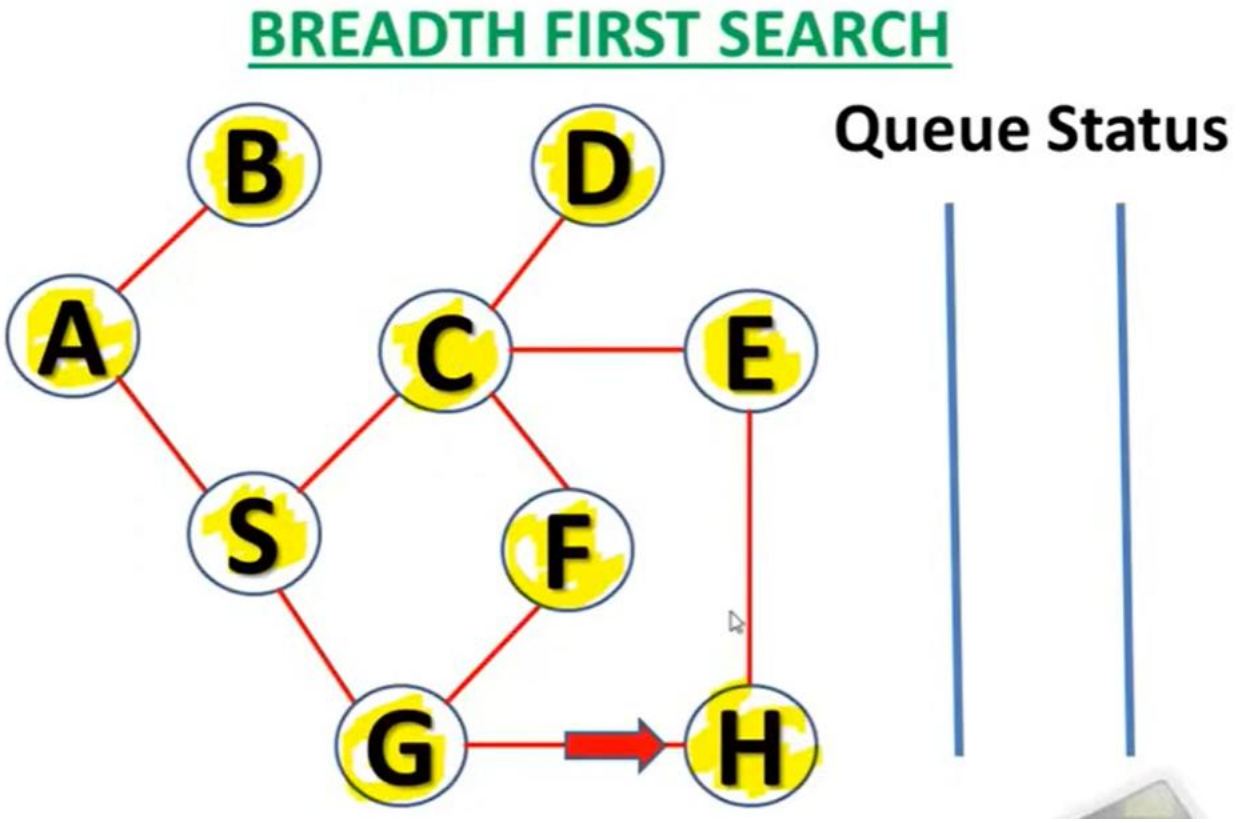
- OUTPUT: A B S C G D E F H



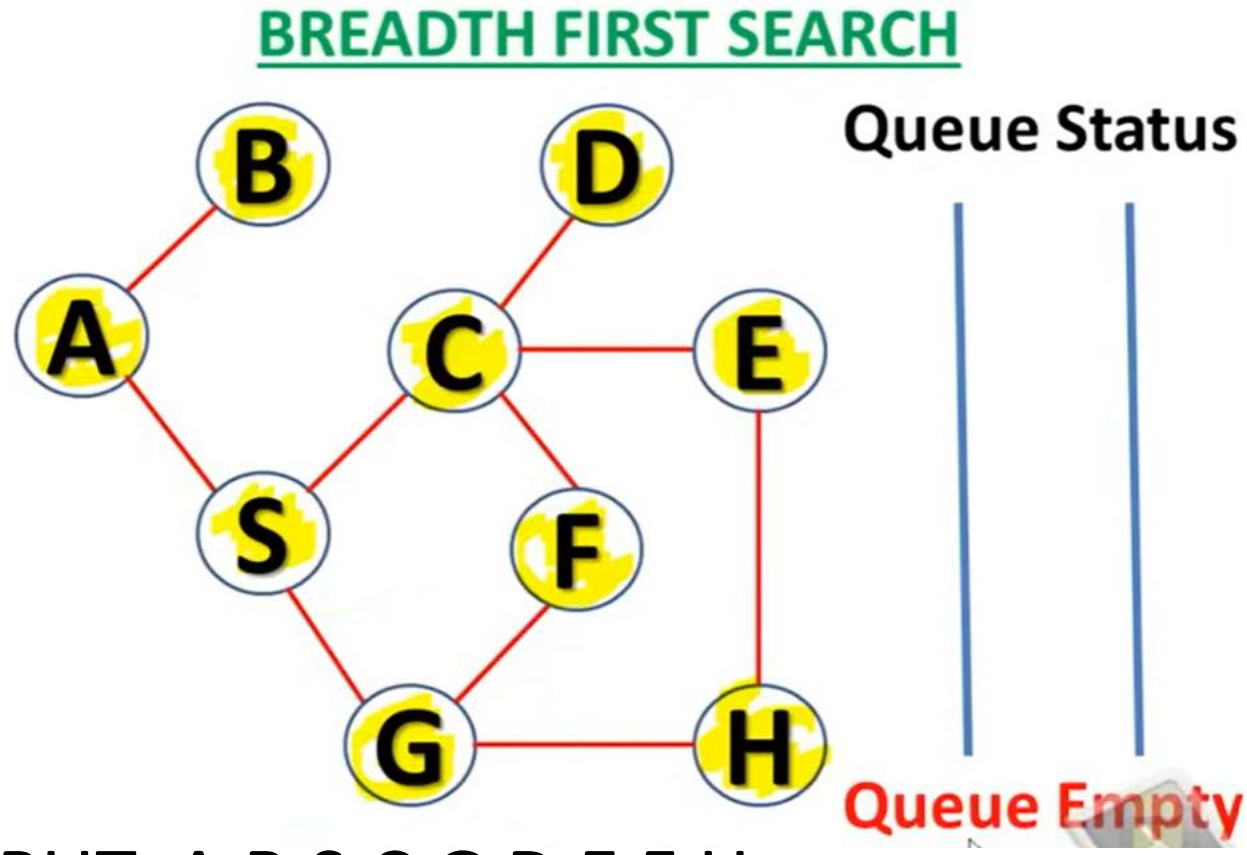
- OUTPUT: A B S C G D E F H



- OUTPUT: A B S C G D E F H



- OUTPUT: A B S C G D E F H



- OUTPUT: A B S C G D E F H

BFS Code

```
#include<stdio.h>
#include<conio.h>
void bfs(int a[20][20], int n, int src, int t[20][2], int s[])
{
        int f,r,q[20],u,v,k=0,i;
        for(i=1;i<=n;i++)
                s[i]=0;
        f=r=k=0;
        q[r]=src;
        s[src]=1;
        while(f<=r)
        {
                u=q[f++];
                for(v=1;v<=n;v++)
                {
                        if(a[u][v]==1 && s[v]==0)
                        {
                                s[v]=1;
                                q[++r]=v;
                                t[k][0]=u;
                                t[k][1]=v;
                                k++;
                        }
                }
        }
}
```

```
void main()
{
		int n,a[20][20],src,t[20][2],flag,s[20],i,j;
		clrscr();
		printf("Enter the number of nodes\n");
		scanf("%d", &n);
		printf("Enter the adjacency matrix\n");
		for(i=0;i<n;i++)
		{
				for(j=0;j<n;j++)
						scanf("%d", &a[i][j]);
		}
		printf("Enter the source\n");
		scanf("%d", &src);
		bfs(a,n,src,t,s);
		flag=0;
		for(i=0;i<n;i++)
		{
				if(s[i]==0)
				{
						printf("Vertex %d is not reachable\n", i);
						flag=1;
				}
				else
						printf("Vertex %d is reachable\n", i);
		}
		if(flag==1)
				printf("Some nodes are not visited\n");
		else
		{
				printf("The BFS traversal is\n");
				for(i=0;i<n;i++)
						printf("%d%d\n", t[i][0], t[i][1]);
		}
		getch();
}
```

```c
#include <stdio.h>

#define MAXN 20

void bfs(int a[MAXN][MAXN], int n, int src, int visited[], int order[], int *orderCount) {
    int q[MAXN], front = 0, rear = 0;
    for (int i = 0; i < n; ++i) visited[i] = 0;

    /* enqueue source */
    visited[src] = 1;
    q[rear++] = src;
    *orderCount = 0;

    while (front < rear) {
        int u = q[front++];          /* dequeue */
        order[(*orderCount)++] = u;  /* record traversal order */

        for (int v = 0; v < n; ++v) {
            if (a[u][v] == 1 && !visited[v]) {
                visited[v] = 1;      /* mark visited */
                q[rear++] = v;      /* enqueue */
            }
        }
    }
}
```

```c
int main(void) {
    int n;
    int a[MAXN][MAXN];
    int visited[MAXN], order[MAXN], orderCount;
    int src;

    printf("Enter number of nodes (max %d): ", MAXN);
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAXN) {
        printf("Invalid n\n");
        return 1;
    }

    printf("Enter adjacency matrix (%d x %d), rows of 0/1:\n", n, n);
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            scanf("%d", &a[i][j]);

    printf("Enter source vertex (0 to %d): ", n - 1);
    if (scanf("%d", &src) != 1 || src < 0 || src >= n) {
        printf("Invalid source\n");
        return 1;
    }

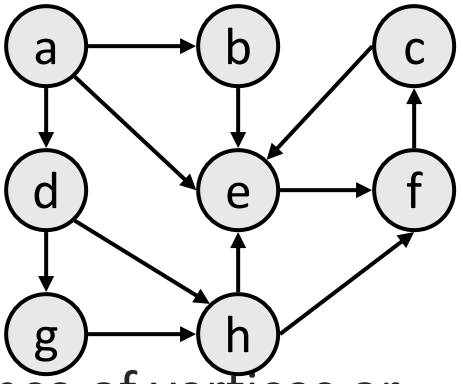
    bfs(a, n, src, visited, order, &orderCount);

    printf("\nBFS traversal order (dequeue sequence):\n");
    for (int i = 0; i < orderCount; ++i) printf("%d ", order[i]);
    printf("\n");

    printf("\nReachability from %d:\n", src);
    for (int i = 0; i < n; ++i)
        printf("Vertex %d : %s\n", i, visited[i] ? "reachable" : "NOT reachable");

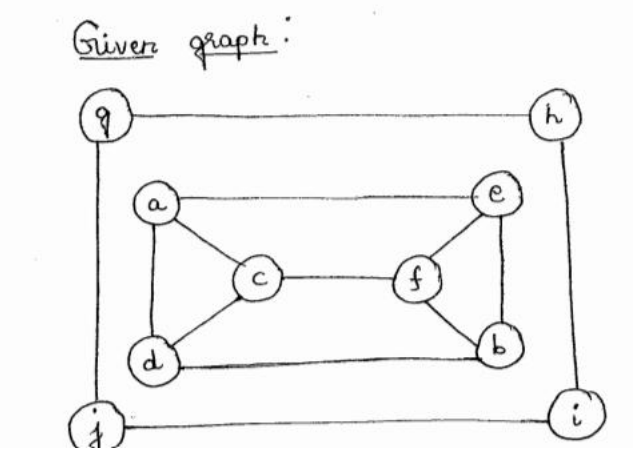
    return 0;
}
```

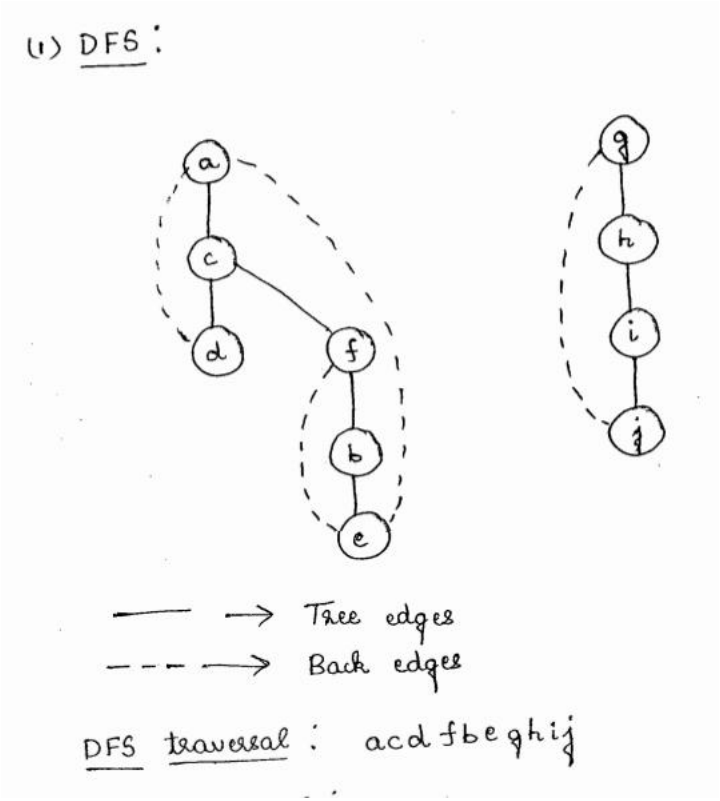
BFS Observation



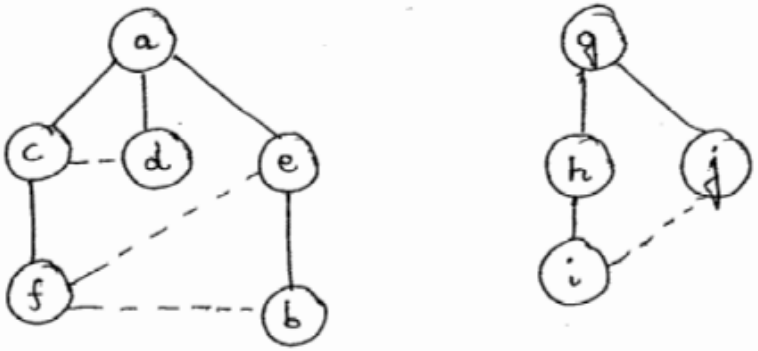
- *optimality*:
  - always finds the shortest path (fewest edges).
  - in unweighted graphs, finds optimal cost path.
  - In weighted graphs, *not* always optimal cost.
- *retrieval*: harder to reconstruct the actual sequence of vertices or edges in the path once you find it
  - conceptually, BFS is exploring many possible paths in parallel, so it's not easy to store a path array/list in progress
  - solution: We can keep track of the path by storing predecessors for each vertex (each vertex can store a reference to a *previous* vertex).
- DFS uses less memory than BFS, easier to reconstruct the path once found; but DFS does not always find shortest path. BFS does.

Practice


Given graph:
g
h
a
e
c
f
d
b
j
i


(i) DFS:
a
c
d
f
b
e
g
h
i
j
Tree edges
Back edges
DFS traversal: acdfbeghij

(2) BFS:



——→ Tree edges

- - -→ Cross edges

BFS traversal:

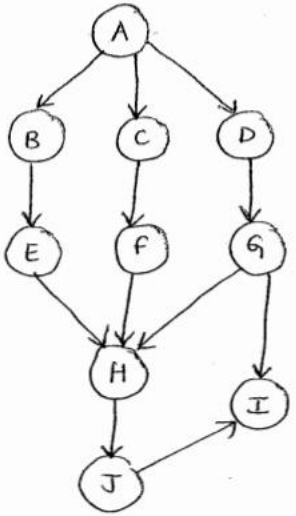
acdefbghji

Queue traversal:

a_1 c_2 d_3 e_4 f_5 b_6 g_7 h_8 j_9 i_{10}

Obtain DFS and BFS traversal for the following:

(1)



DFS:

Traversal: A B E H J I C F D G

Stack Traversal:

I 6,1
J 5,2
H 4,3
E 3,4 F 8,6 G 10,8
B 2,5 C 7,7 D 9,9
A 1,10

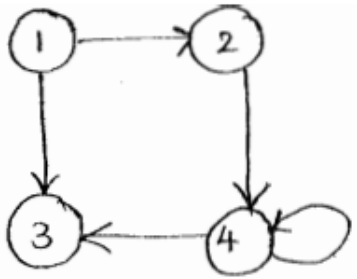
BFS:

Traversal: A B C D E F G H I J

Queue Traversal:

A_1 B_2 C_3 D_4 E_5 F_6 G_7 H_8 I_9 J_{10}

(2)



DFS :

Traversal : 1 2 4 3

Stack traversal :

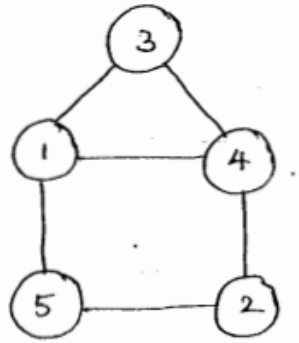
3	4, 1
4	3, 2
2	2, 3
1	1, 4

BFS :

Traversal : 1 2 3 4

Queue traversal : 1_1 2_2 3_3 4_4

(3)



DFS :

Traversal : 3 1 5 2 4

4 5,1
2 4,2
5 3,3
1 2.4

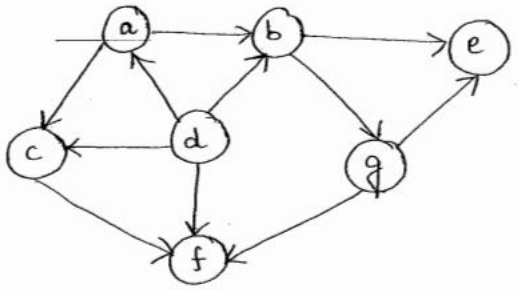
BFS :

Traversal : 3 1 4 5 2

Queue traversal :

3_1 1_2 4_3 5_4 2_5

(4)



DFS :

Traversal : a b e g f c d

Stack traversal :

e 3,1 f 5,2

b 2,4 g 4,3

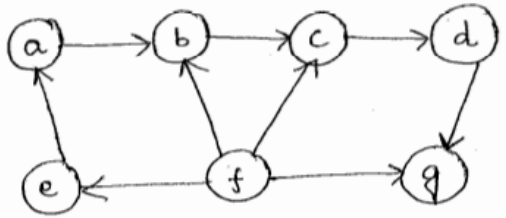
a 1,6 c 6,5 d 7,7

BFS :

Traversal : d a b c f g e

d_1 a_2 b_3 c_4 f_5 g_6 e_7

(5)



DFS :

Traversal : a b c d g f e

Stack traversal :

g 5, 1
d 4, 2
c 3, 3
b 2, 4
a 1, 5

e 7, 6
f 6, 7

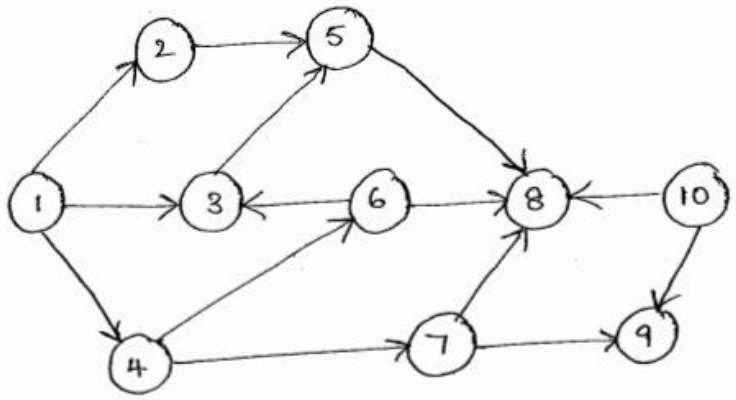
BFS :

Traversal : f b c e g d a

f_1 b_2 c_3 e_4 g_5 d_6 a_7

Queue traversal

(6)



DFS :

Traversal: 1 2 5 8 3 4 7 9 6 10

Hashing

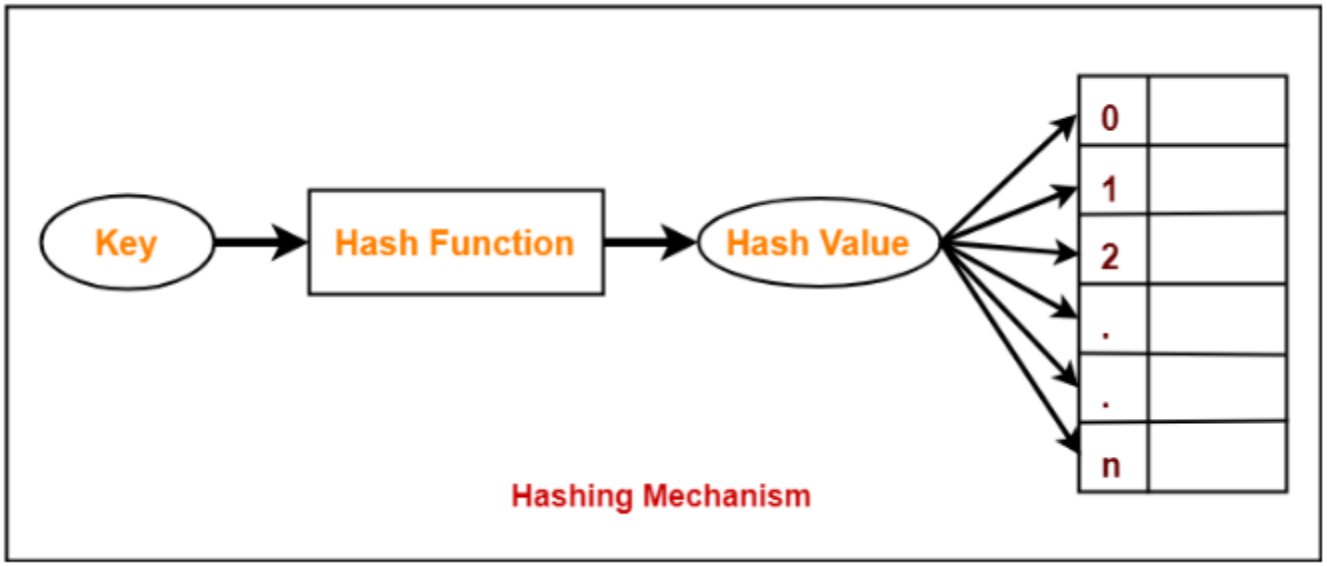
- Hashing is a well-known technique to search any particular element among several elements.
- It minimizes the number of comparisons while performing the search.
- Unlike other searching techniques,
  - Hashing is extremely efficient.
  - The time taken by it to perform the search does not depend upon the total number of elements.
  - It completes the search with constant time complexity O(1).

Hashing Mechanism

- An array data structure called as **Hash table** is used to store the data items.
- Based on the hash key value, data items are inserted into the hash table.

Hash Key value

- Hash key value is a special value that serves as an index for a data item.
- It indicates where the data item should be stored in the hash table.
- Hash key value is generated using a hash function.



Hash Function

- Hash function is a function that maps any big number or string to a small integer value.
- Hash function takes the data item as an input and returns a small integer value as an output.
- The small integer value is called as a hash value.
- Hash value of the data item is then used as an index for storing it into the hash table.

Types of Hash Functions

- There are various types of hash functions available such as-
- Mid Square Hash Function
- Division Hash Function
- Folding Hash Function etc

- It depends on the user which hash function wants to use.

Properties of Hash Function

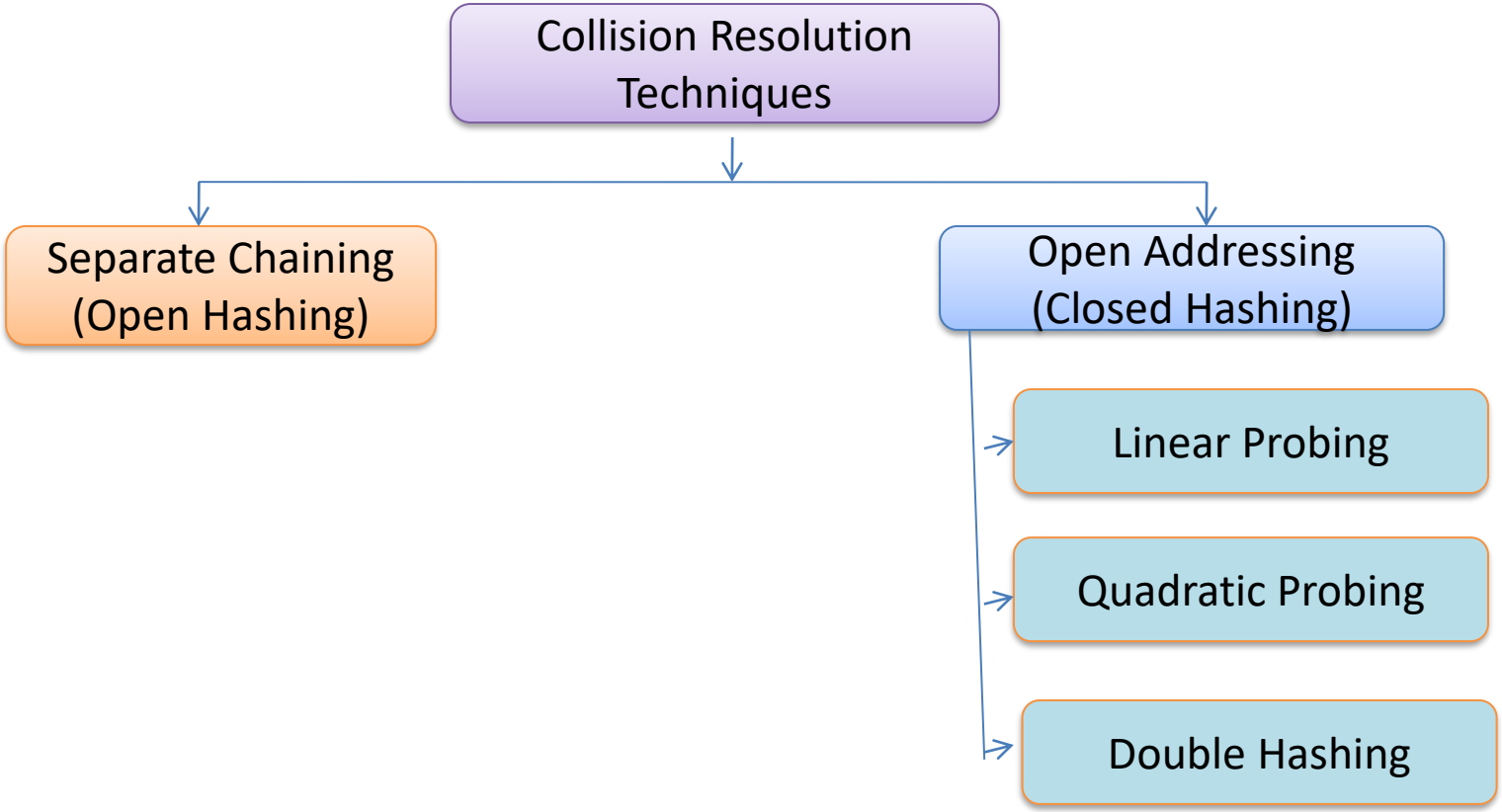
- The properties of a good hash function are-
  - It is efficiently computable.
  - It minimizes the number of collisions.
  - It distributes the keys uniformly over the table.

Collision in Hashing

- Hash function is used to compute the hash value for a key.
- Hash value is then used as an index to store the key in the hash table.
- Hash function may return the same hash value for two or more keys.
- When the hash value of a key maps to an already occupied bucket of the hash table, it is called as a **Collision**.

Collision Resolution Techniques

- Collision Resolution Techniques are the techniques used for resolving or handling the collision.



Separate Chaining

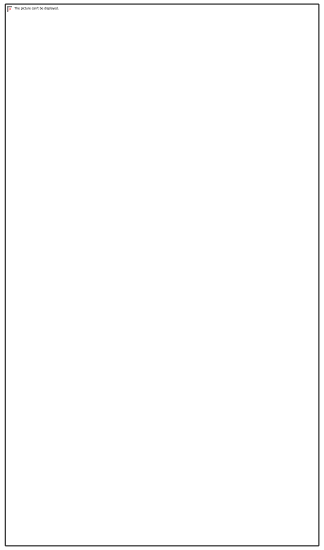
- To handle the collision,
  - This technique creates a linked list to the slot for which collision occurs.
  - The new key is then inserted in the linked list.
  - These linked lists to the slots appear like chains.
  - That is why, this technique is called as **separate chaining**.

Example-Separate Chaining

- Using the hash function ‘key mod 7’, insert the following sequence of keys in the hash table-
- 50, 700, 76, 85, 92, 73 and 101

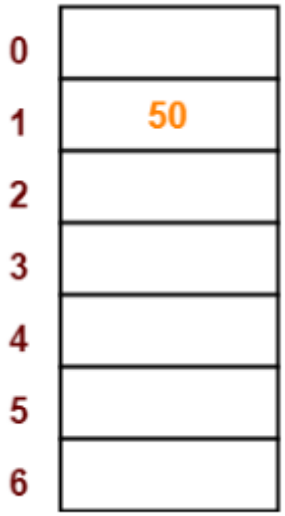
Step-1

- Draw an empty hash table.
- For the given hash function, the possible range of hash values is [0, 6].
- So, draw an empty hash table consisting of 7 buckets as-



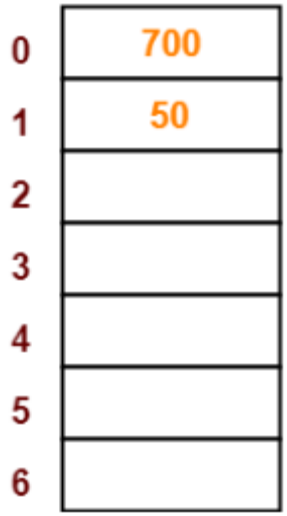
Step-2

- Insert the given keys in the hash table one by one.
- The first key to be inserted in the hash table = 50.
- Bucket of the hash table to which key 50 maps = 50 mod 7 = 1.
- So, key 50 will be inserted in bucket-1 of the hash table as-



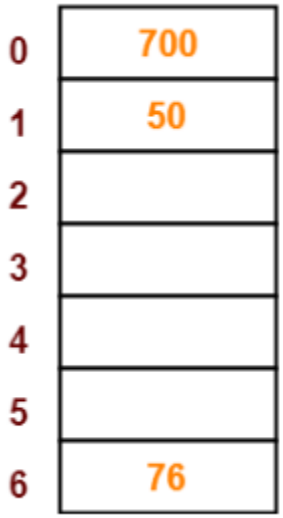
Step-3

- The next key to be inserted in the hash table = 700.
- Bucket of the hash table to which key 700 maps = 700 mod 7 = 0.
- So, key 700 will be inserted in bucket-0 of the hash table as-



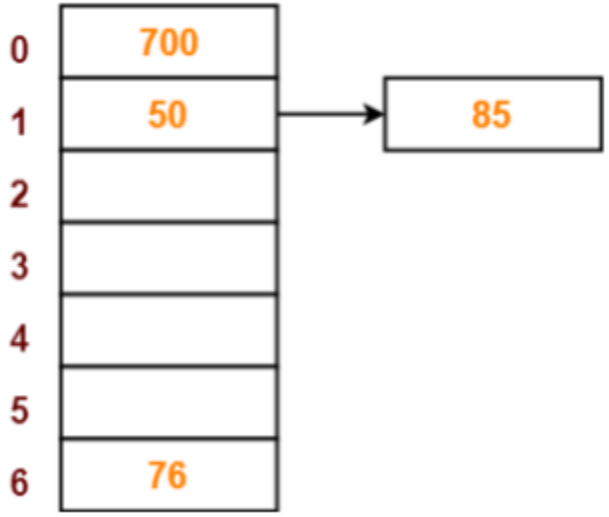
Step-4

- The next key to be inserted in the hash table = 76.
- Bucket of the hash table to which key 76 maps = 76 mod 7 = 6.
- So, key 76 will be inserted in bucket-6 of the hash table as-



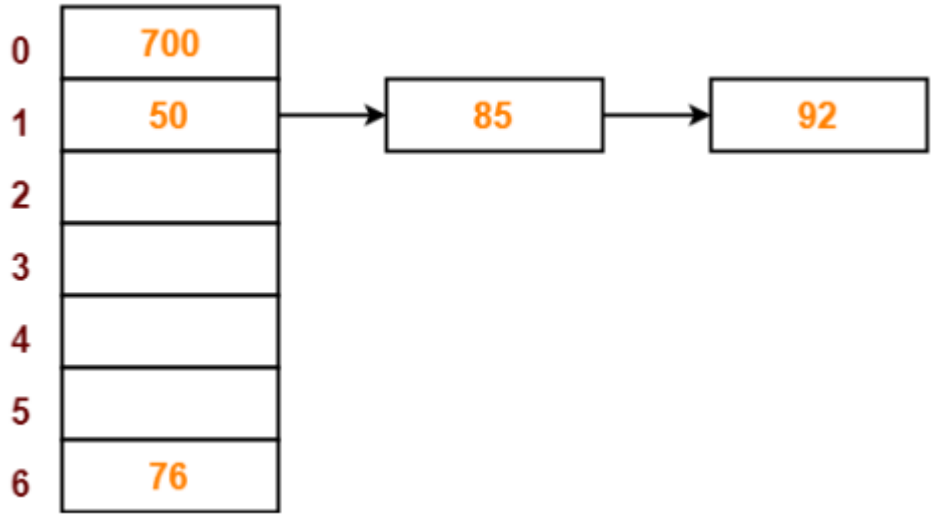
Step-5

- The next key to be inserted in the hash table = 85.
- Bucket of the hash table to which key 85 maps = 85 mod 7 = 1.
- Since bucket-1 is already occupied, so collision occurs.
- Separate chaining handles the collision by creating a linked list to bucket-1.
- So, key 85 will be inserted in bucket-1 of the hash table as-



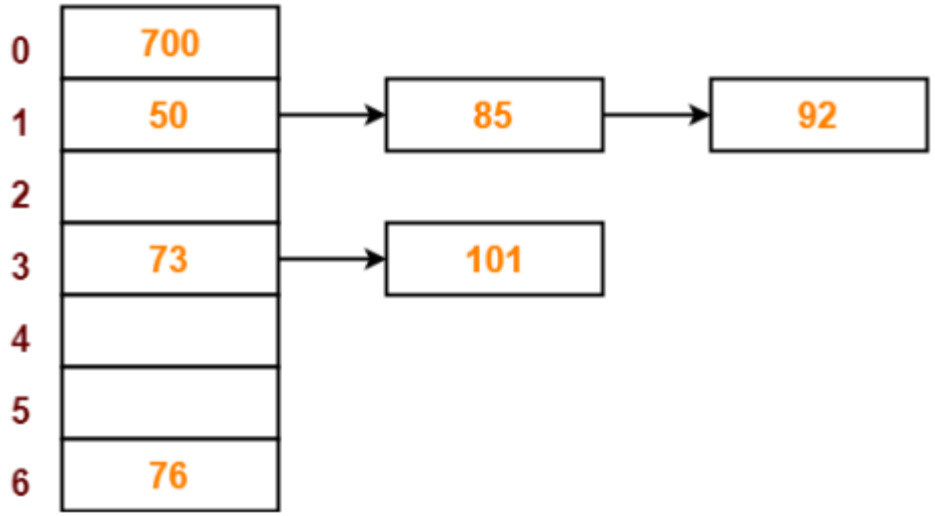
Step-6

- The next key to be inserted in the hash table = 92.
- Bucket of the hash table to which key 92 maps = 92 mod 7 = 1.
- Since bucket-1 is already occupied, so collision occurs.
- Separate chaining handles the collision by creating a linked list to bucket-1.
- So, key 92 will be inserted in bucket-1 of the hash table as-



Step-7

- The next key to be inserted in the hash table = 101.
- Bucket of the hash table to which key 101 maps = 101 mod 7 = 3.
- Since bucket-3 is already occupied, so collision occurs.
- Separate chaining handles the collision by creating a linked list to bucket-3.
- So, key 101 will be inserted in bucket-3 of the hash table as-



Open Addressing

- In open addressing,
  - Unlike separate chaining, all the keys are stored inside the hash table.
  - No key is stored outside the hash table.

- Techniques used for open addressing are-
  - Linear Probing
  - Quadratic Probing
  - Double Hashing

Operations in Open Addressing

- Insert Operation:
  - Hash function is used to compute the hash value for a key to be inserted.
  - Hash value is then used as an index to store the key in the hash table.

- In case of collision,
  - Probing is performed until an empty bucket is found.
  - Once an empty bucket is found, the key is inserted.
  - Probing is performed in accordance with the technique used for open addressing.

Open Addressing

- Search Operation:
- To search any particular key,
  - Its hash value is obtained using the hash function used.
  - Using the hash value, that bucket of the hash table is checked.
  - If the required key is found, the key is searched.
  - Otherwise, the subsequent buckets are checked until the required key or an empty bucket is found.
  - The empty bucket indicates that the key is not present in the hash table.

Open Addressing

- Delete Operation:
  - The key is first searched and then deleted.
  - After deleting the key, that particular bucket is marked as “deleted”.

1. Linear Probing

- In linear probing,
  - When collision occurs, we linearly probe for the next bucket.
  - We keep probing until an empty bucket is found.

- **Advantage-**

  - It is easy to compute.

- **Disadvantage-**

  - The main problem with linear probing is clustering.
  - Many consecutive elements form groups.
  - Then, it takes time to search an element or to find an empty bucket.

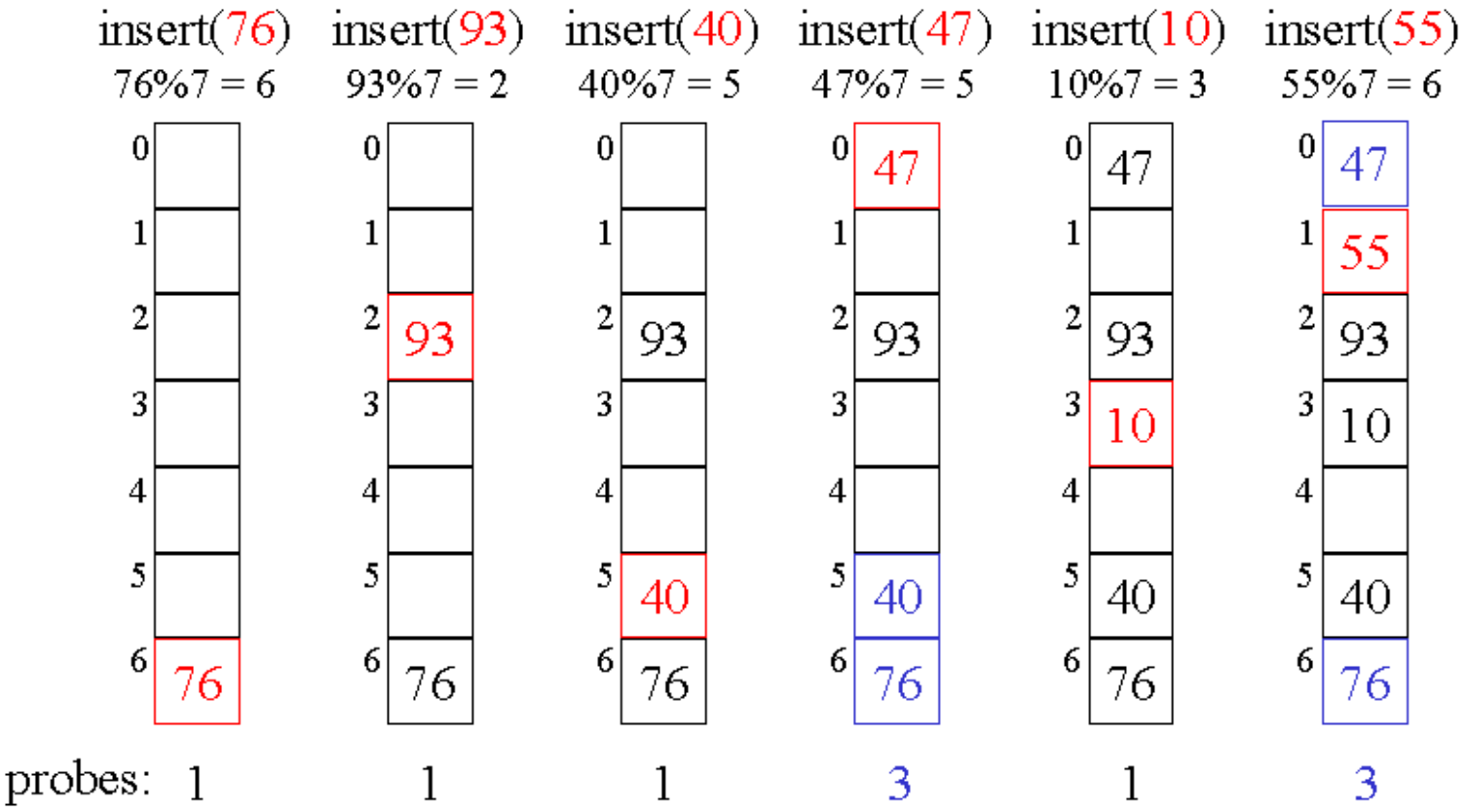
Linear Probing

$$f(i) = i$$

- Probe sequence is
  - h(k) mod size
  - h(k) + 1 mod size
  - h(k) + 2 mod size
  - …
- findEntry using linear probing:

```
bool findEntry(const Key & k, Entry *& entry) {
  int probePoint = hash1(k);
  do {
    entry = &table[probePoint];
    probePoint = (probePoint + 1) % size;
  } while (!entry->isEmpty() && entry->key != k);
  return !entry->isEmpty();
}
```

Example- Linear Probing



2. Quadratic Probing

- In quadratic probing,
- When collision occurs, we probe for i^2‘th bucket in i^{th} iteration.
- We keep probing until an empty bucket is found.

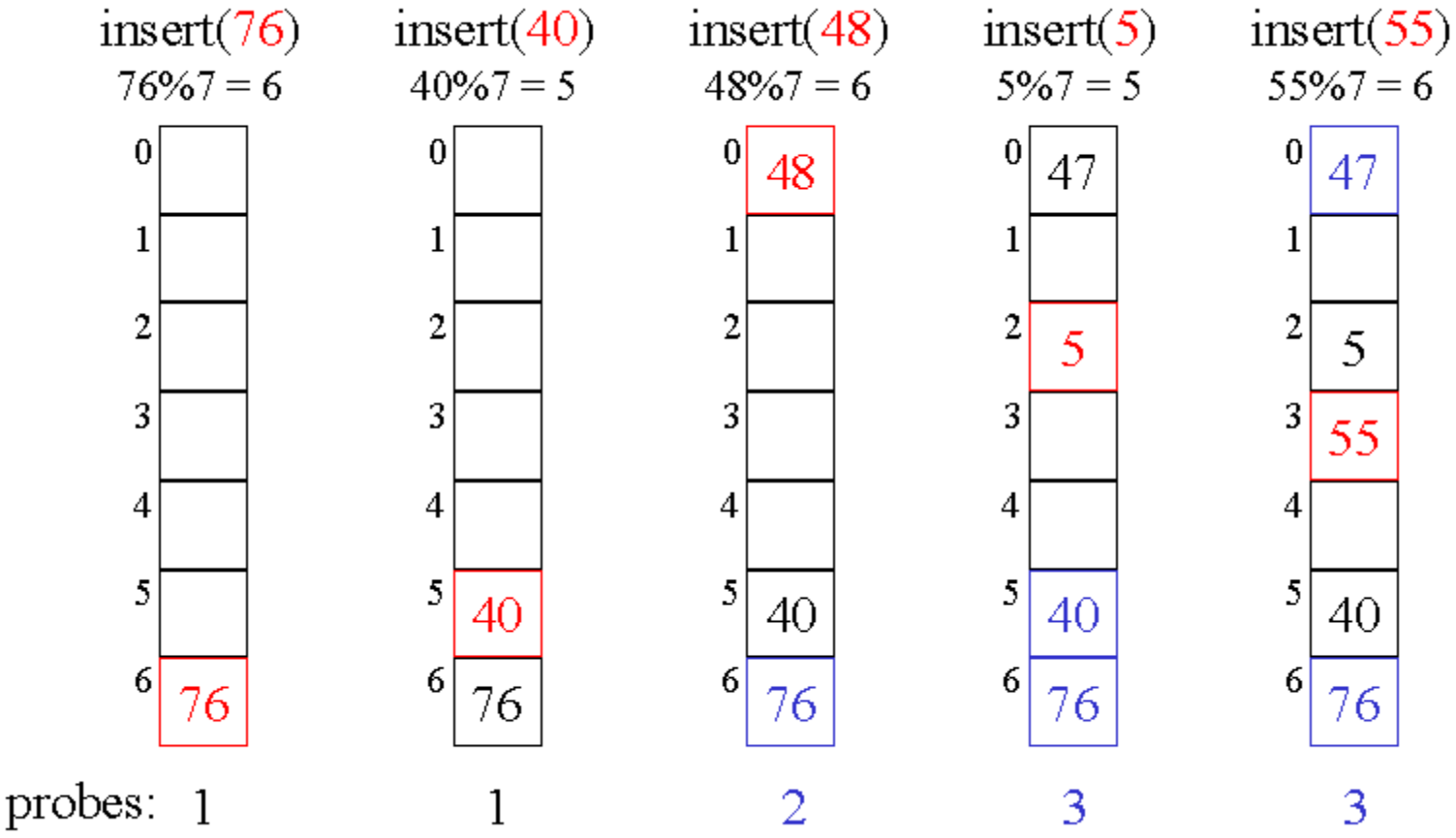
Quadratic Probing

$$f(i) = i^2$$

- Probe sequence is
  - h(k) mod size
  - (h(k) + 1) mod size
  - (h(k) + 4) mod size
  - (h(k) + 9) mod size
  - …
- findEntry using quadratic probing:

```
bool findEntry(const Key & k, Entry *& entry) {
  int probePoint = hash1(k), numProbes = 0;
  do {
    entry = &table[probePoint];
    numProbes++;
    probePoint = (probePoint + 2*numProbes - 1) % size;
  } while (!entry->isEmpty() && entry->key != key);
  return !entry->isEmpty();
}
```

Example – Quadratic Probing



Double Hashing

- In double hashing,
- We use another hash function hash2(x) and look for i * hash2(x) bucket in i^{th} iteration.
- It requires more computation time as two hash functions need to be computed.

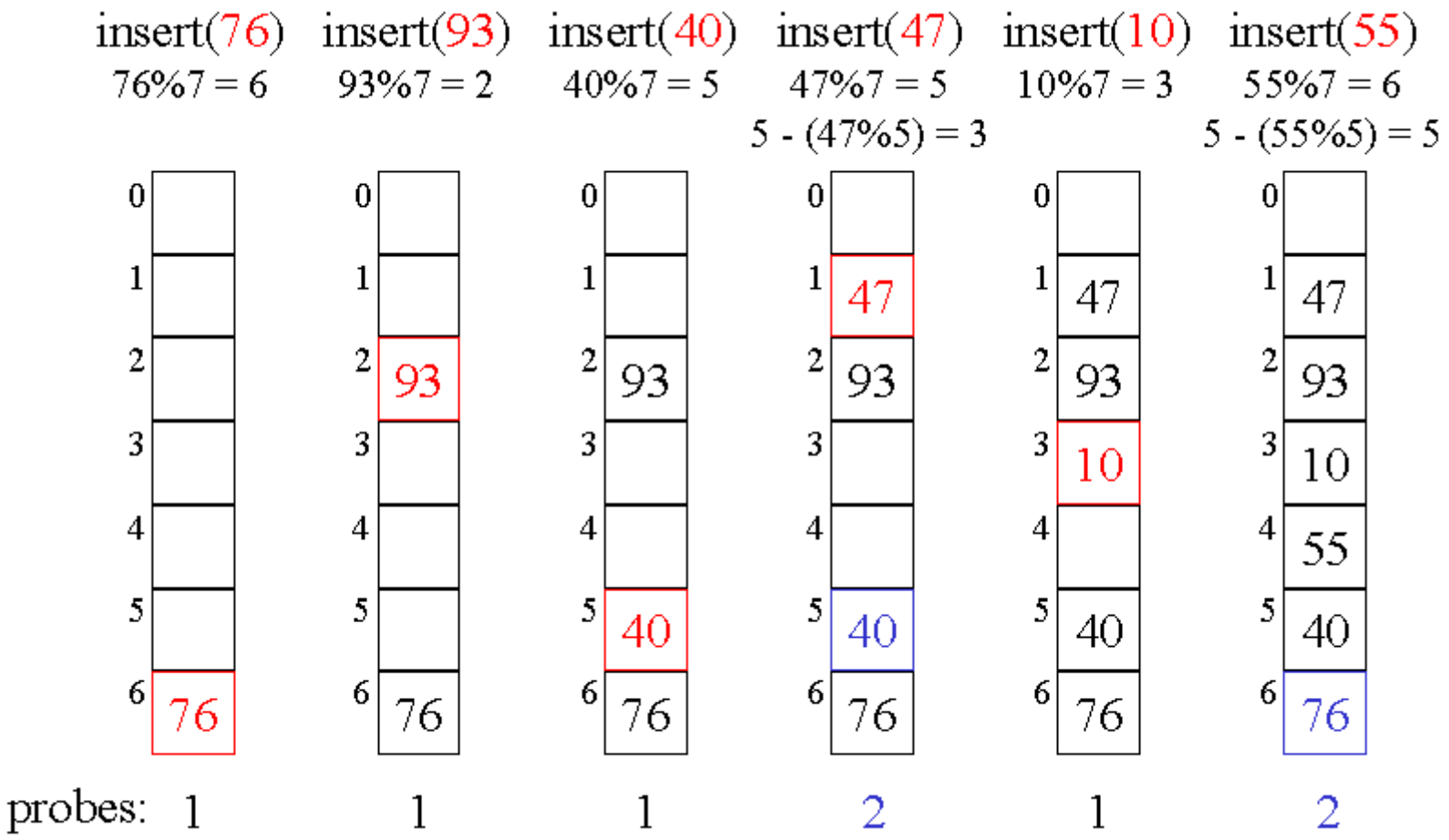
Double Hashing

$$f(i) = i \cdot hash_2(x)$$

- Probe sequence is
  - $h_1(k) \bmod size$
  - $(h_1(k) + 1 \cdot h_2(x)) \bmod size$
  - $(h_1(k) + 2 \cdot h_2(x)) \bmod size$
  - …
- Code for finding the next linear probe:

```
bool findEntry(const Key & k, Entry *& entry) {
  int probePoint = hash1(k), hashIncr = hash2(k);
  do {
    entry = &table[probePoint];
    probePoint = (probePoint + hashIncr) % size;
  } while (!entry->isEmpty() && entry->key != k);
  return !entry->isEmpty();
}
```

Example –Double Hashing



Separate chaining Vs Open Addressing

Separate Chaining	Open Addressing
Keys are stored inside the hash table as well as outside the hash table.	All the keys are stored only inside the hash table. No key is present outside the hash table.
The number of keys to be stored in the hash table can even exceed the size of the hash table.	The number of keys to be stored in the hash table can never exceed the size of the hash table.
Deletion is easier.	Deletion is difficult.
Extra space is required for the pointers to store the keys outside the hash table.	No extra space is required.
Cache performance is poor. This is because of linked lists which store the keys outside the hash table.	Cache performance is better. This is because here no linked lists are used.
Some buckets of the hash table are never used which leads to wastage of space.	Buckets may be used even if no key maps to those particular buckets.

Comparison of Open Addressing Techniques

	Linear Probing	Quadratic Probing	Double Hashing
Primary Clustering	Yes	No	No
Secondary Clustering	Yes	Yes	No
Number of Probe Sequence (m = size of table)	m	m	m^2
Cache performance	Best	Lies between the two	Poor

Example

```c
#include<stdio.h>

#define size 7

int arr[size];


void init()
{
    int i;
    for(i = 0; i < size; i++)
        arr[i] = -1;
}

void insert(int value)
{
    int key = value % size;

    if(arr[key] == -1)
    {
        arr[key] = value;
        printf("%d inserted at arr[%d]\n", value,key);
    }
    else
    {
        printf("Collision : arr[%d] has element %d already!\n",ke
        printf("Unable to insert %d\n",value);
    }
}
```

```c
void del(int value)
{
    int key = value % size;
    if(arr[key] == value)
        arr[key] = -1;
    else
        printf("%d not present in the hash table\n",value);
}

void search(int value)
{
    int key = value % size;
    if(arr[key] == value)
        printf("Search Found\n");
    else
        printf("Search Not Found\n");
}

void print()
{
    int i;
    for(i = 0; i < size; i++)
        printf("arr[%d] = %d\n",i,arr[i]);
}
```

```c
int main()
{
    init();
    insert(10); //key = 10 % 7 ==> 3
    insert(4);  //key = 4 % 7  ==> 4
    insert(2);  //key = 2 % 7  ==> 2
    insert(3);  //key = 3 % 7  ==> 3 (collision)

    printf("Hash table\n");
    print();
    printf("\n");

    printf("Deleting value 10..\n");
    del(10);
    printf("After the deletion hash table\n");
    print();
    printf("\n");

    printf("Deleting value 5..\n");
    del(5);
    printf("After the deletion hash table\n");
    print();
    printf("\n");

    printf("Searching value 4..\n");
    search(4);
    printf("Searching value 10..\n");
    search(10);

    return 0;
}
```

THANK YOU

EXTRA CONTENT

Rehashing

- Hash tables offer exceptional performance when not overly full.
- This is the traditional dilemma of all array-based data structures:
- Make the table too small, performance degrades and the table may overflow
- Make the table too big, and memory gets wasted.
- Rehashing or variable hashing attempts to circumvent this dilemma by expanding the hash table size whenever it gets too full.

Load Factor and Rehashing

- Load factor λ (lambda) is the ratio of the number of elements to the size of the hash table **λ = n/N** where n denotes the number of elements and N the number of locations in the hash table
- For the open addressing scheme, **λ** is between 0 and 1
- If the hash table is empty, then **λ** = 0
- If the hash table is full, then **λ** = 1
- For the separate chaining scheme, **λ** can be any value.

How is the Load Factor Used?

- The Load Factor decides “when to increase the size of the hash Table.”
- The load factor can be decided using the following formula:
  Initial capacity of the HashTable * Load factor of the HashTable

- E.g. If The initial capacity of HashTable is = 16
  and the load factor of HashTable = 0.75

- According to the formula as mentioned above: 16 * 0.75 = 12

- It represents that the 12th key-value pair of a HashTable will keep its size to 16.
- As soon as the 13th element (key-value pair) will come into the HashTable, it will increase its size from 16 buckets to 16 * 2 = 32 buckets.

Rehashing Example

- Say we had HashTable with Initial Capacity of 4.
- We need to insert 4 Keys: 100, 101, 102, 103
- And say the Hash function used was division method: Key % ArraySize

- So Hash(100) = 1, so Element2 stored at 1st Index.
- So Hash(101) = 2, so Element3 stored at 2nd Index.
- So Hash(102) = 0, so Element1 stored at 3rd Index.

- With the insertion of 3 elements, the load on Hash Table = ¾ = 0.74

- So we can add this 4th element to this Hash table, and we need to increase its size to 6 now.

Example continued..

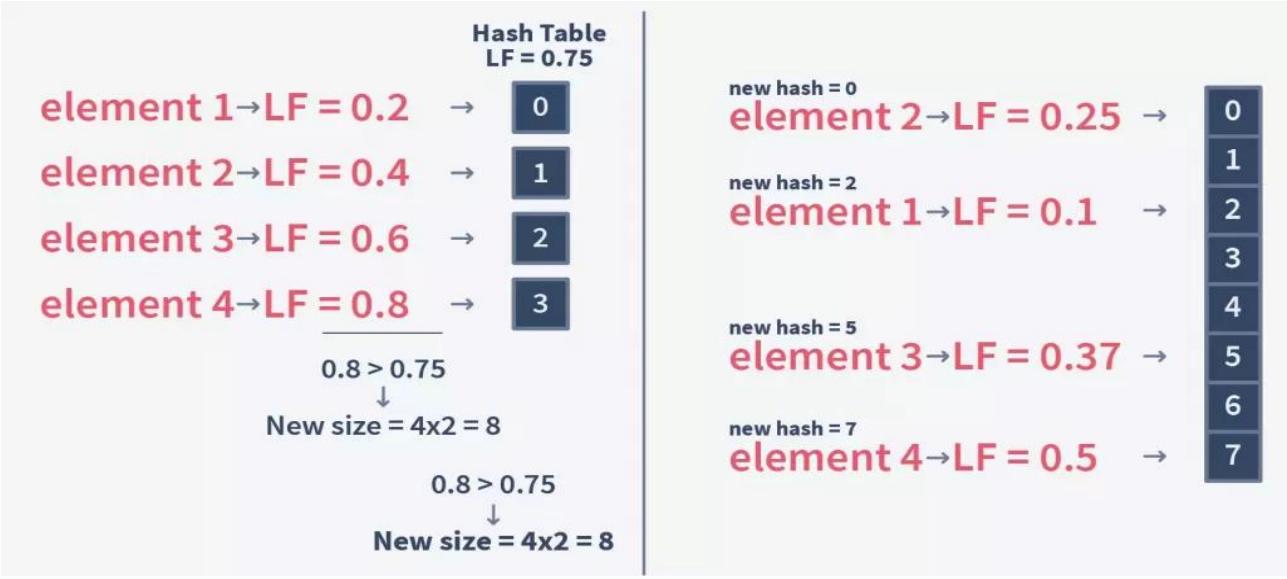
- E.g. The earlier hash function was Key%3 and now it is Key%6.
- So Hash(100) = 4, so Element stored at 4th Index.
- So Hash(101) = 5, so Element stored at 5th Index.
- So Hash(102) = 0, so Element stored at 0th Index.
- So Hash(103) = 1, so Element stored at 1st Index.

Example continued..

- Element1: Hash(100) = 100%6 = 4, so Element1 will be rehashed and will be stored at 5th Index in this newly resized HashTable, instead of 1st Index as on previous HashTable.
- Element2: Hash(101) = 101%6 = 5, so Element2 will be rehashed and will be stored at 6th Index in this newly resized HashTable, instead of 2nd Index as on previous HashTable.
- Element3: Hash(102) = 102%6 = 6, so Element3 will be rehashed and will be stored at 4th Index in this newly resized HashTable, instead of 3rd Index as on previous HashTable.
- Since the Load Balance now is 3/6 = 0.5, we can still insert the 4th element now.
- Element4: Hash(103) = 103%6 = 1, so Element4 will be stored at 1st Index in this newly resized HashTable.

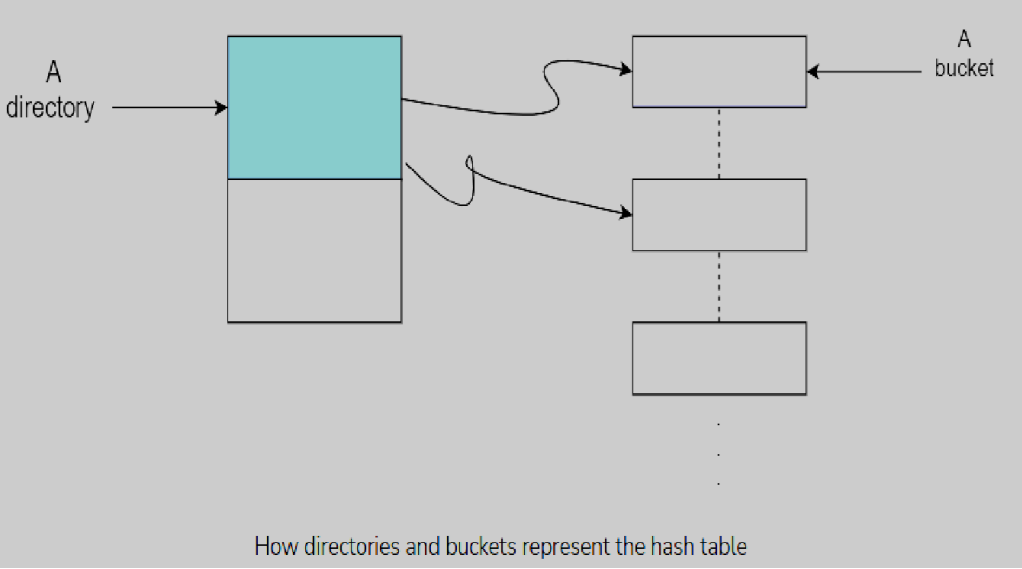
Rehashing Steps

- For each addition of a new entry to the map, check the current load factor.
- If it's greater than its pre-defined value, then Rehash.
- For Rehash, make a new array of double the previous size and make it the new bucket array.
- Then traverse to each element in the old bucketArray and insert them back so as to insert it into the new larger bucket array.



Extendible hashing

- **Extendible hashing** is a dynamic approach to managing data. In this hashing method, flexibility is a crucial factor. This method caters to flexibility so that even the hashing function dynamically changes according to the situation and data type.



How directories and buckets represent the hash table

- **Directories** and **buckets** are two key terms in this algorithm.
- ***Buckets*** are the holders of hashed data, while ***directories*** are the holders of pointers pointing towards these buckets.
- Each directory has a unique ID.

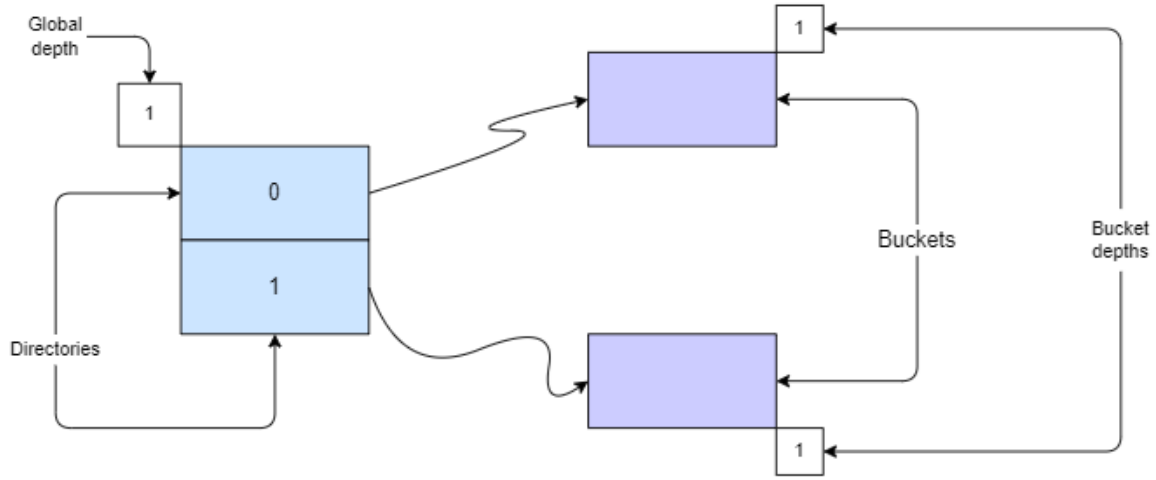
Algorithm

1. Initialize the bucket depths and the global depth of the directories.
2. Convert data into a binary representation.
3. Consider the "global depth" number of the least significant bits (LSBs) of data.
4. Map the data according to the ID of a directory.
5. Check for the following conditions if a bucket overflows (if the number of elements in a bucket exceeds the set limit):
    1. **Global depth == bucket depth**: Split the bucket into two and increment the global depth and the buckets' depth. Re-hash the elements that were present in the split bucket.
    2. **Global depth > bucket depth**: Split the bucket into two and increment the bucket depth only. Re-hash the elements that were present in the split bucket.
6. Repeat the steps above for each element.

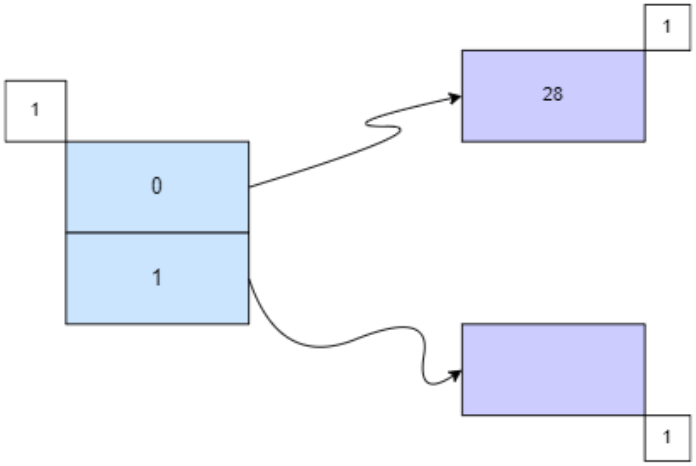
Example

- Let's take the following example to see how this hashing method works where:
  - Data = {28,4,19,1,22,16,12,0,5,7}
  - Bucket limit = 3

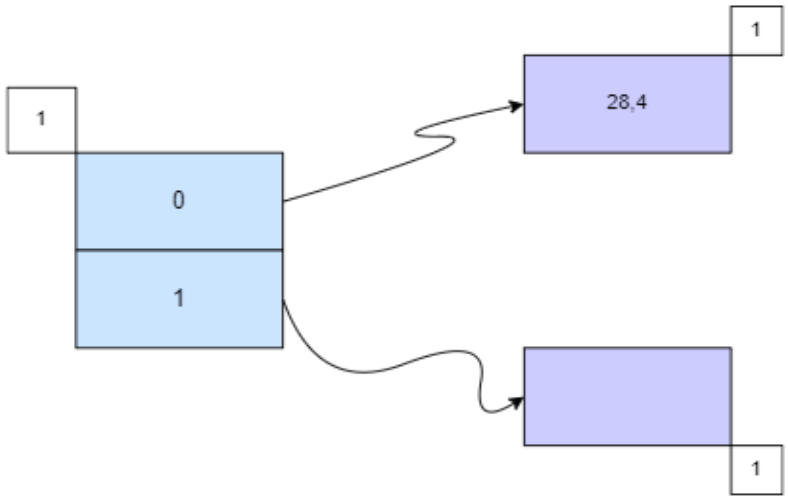
- Convert the data into binary representation:
  - 28 = 11100
  - 4 = 00100
  - 19 = 10011
  - 1 = 00001
  - 22 = 10110
  - 16 = 10000
  - 12 = 01100
  - 0 = 00000
  - 5 = 00101
  - 7 = 00111



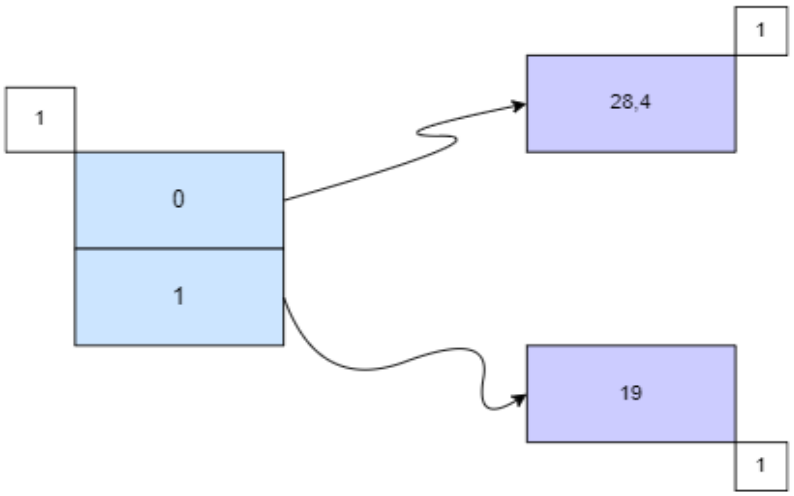
Initialize the hash table with two initial directories and buckets. Set the global depth and bucket depth to 1



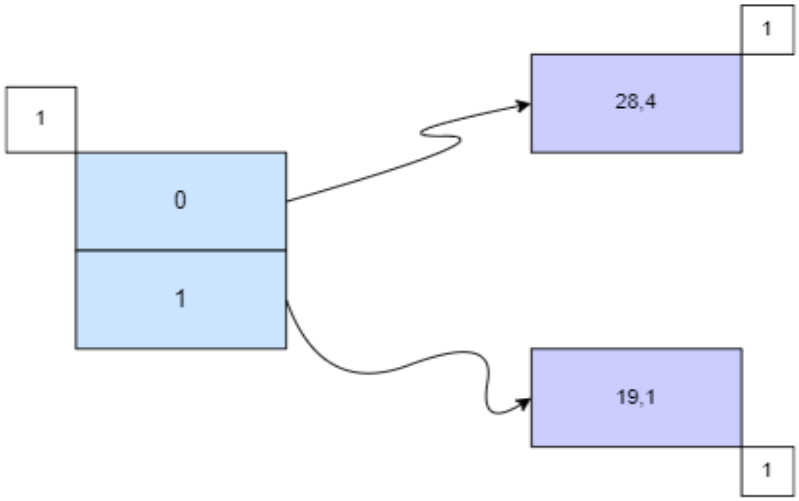
Consider 28 = 11100. Take the global depth number of LSBs, which is currently one. We consider 0 as it is the rightmost LSB in 11100, and 28 is placed in the "0" ID bucket



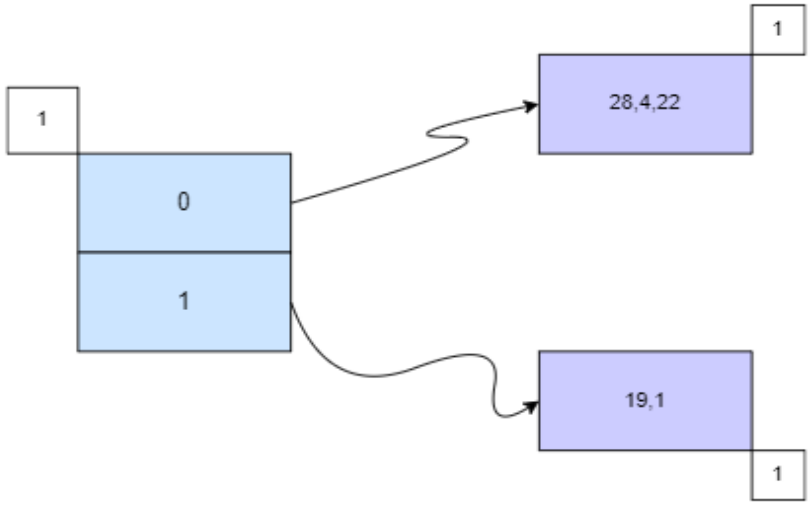
Consider 4 = 00100. Add it to the "0" ID bucket as one LSBs will be considered



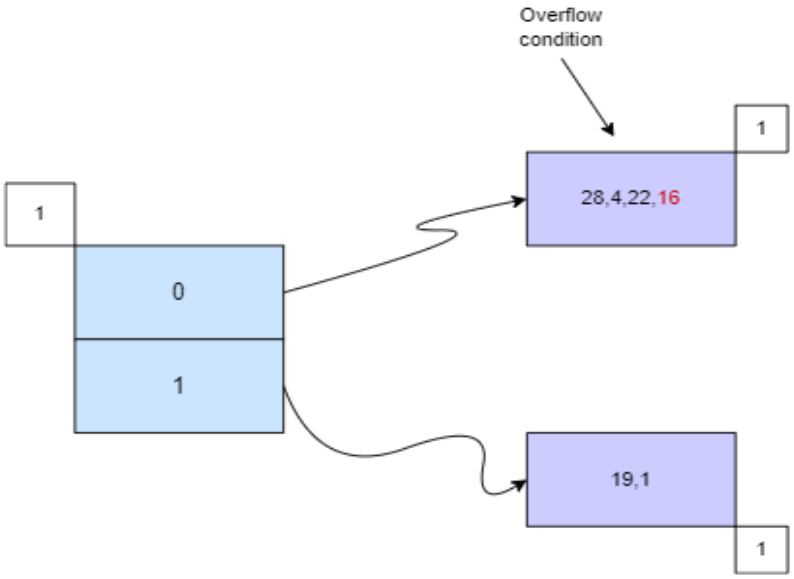
Consider 19 = 10011. Add it to the "1" ID bucket as one LSBs will be considered



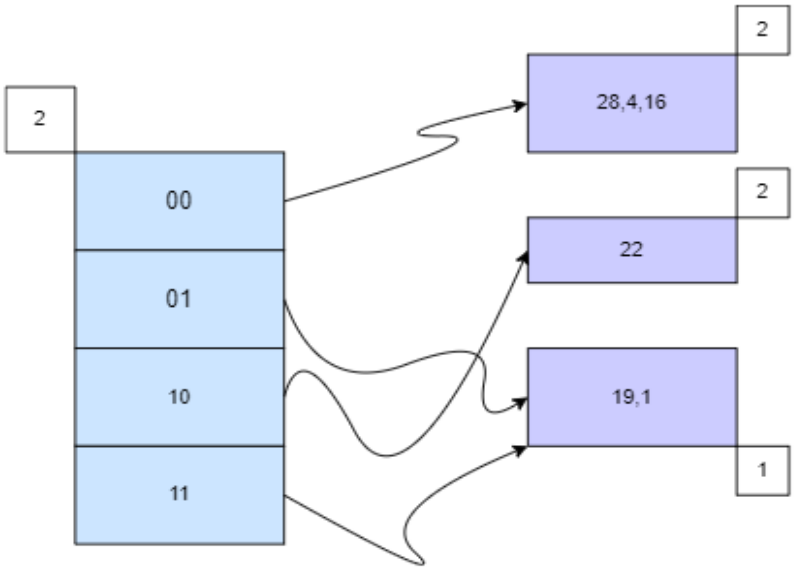
Consider 1 = 00001. Add it to the "1" ID bucket as one LSBs will be considered



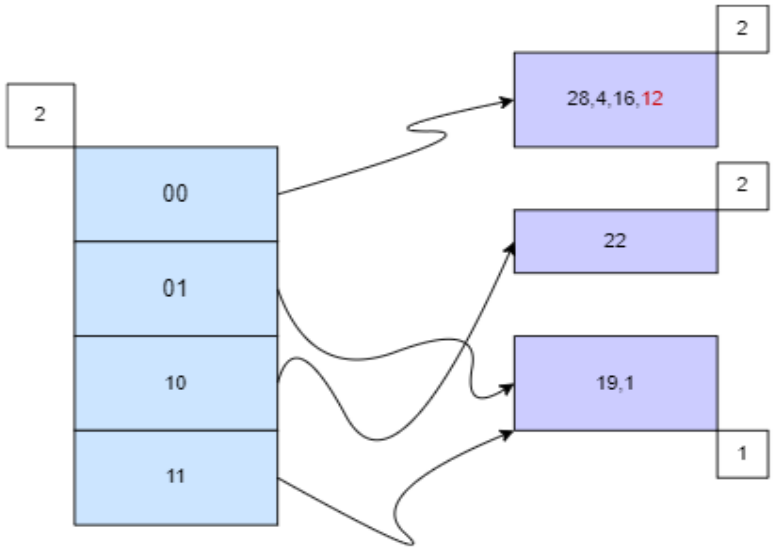
Consider 22 = 10110. Add it to the "0" ID bucket as one LSBs will be considered



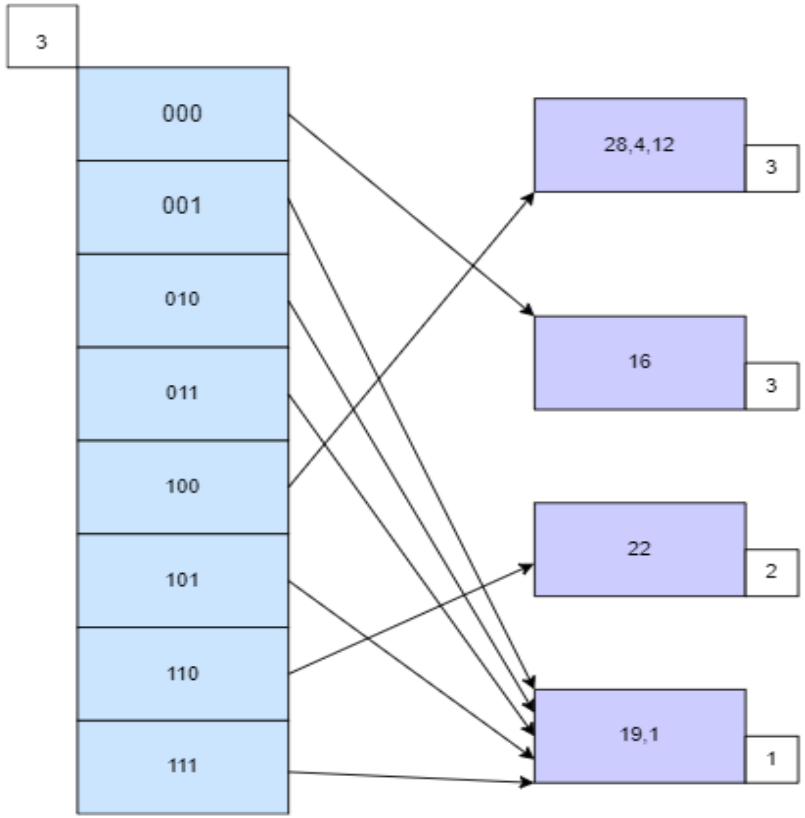
Consider 16 = 10000. Adding it to the "0" ID bucket makes it overflow, and the first condition is satisfied where global depth == bucket depth



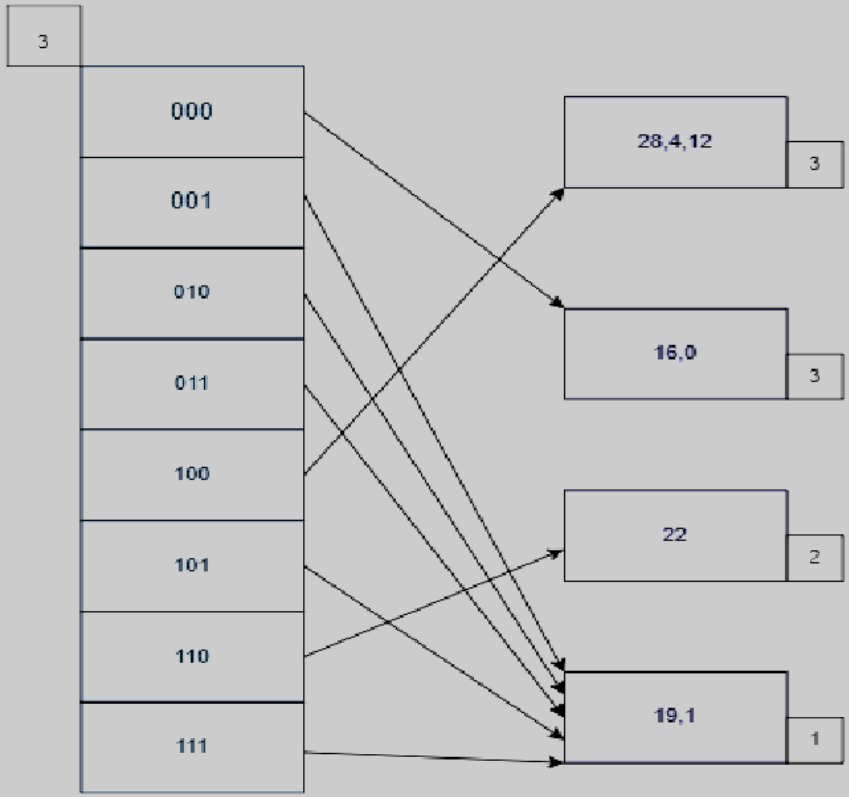
Split the bucket, grow directories, increment the global and bucket depths, and re-hash the bucket elements according to two LSBs



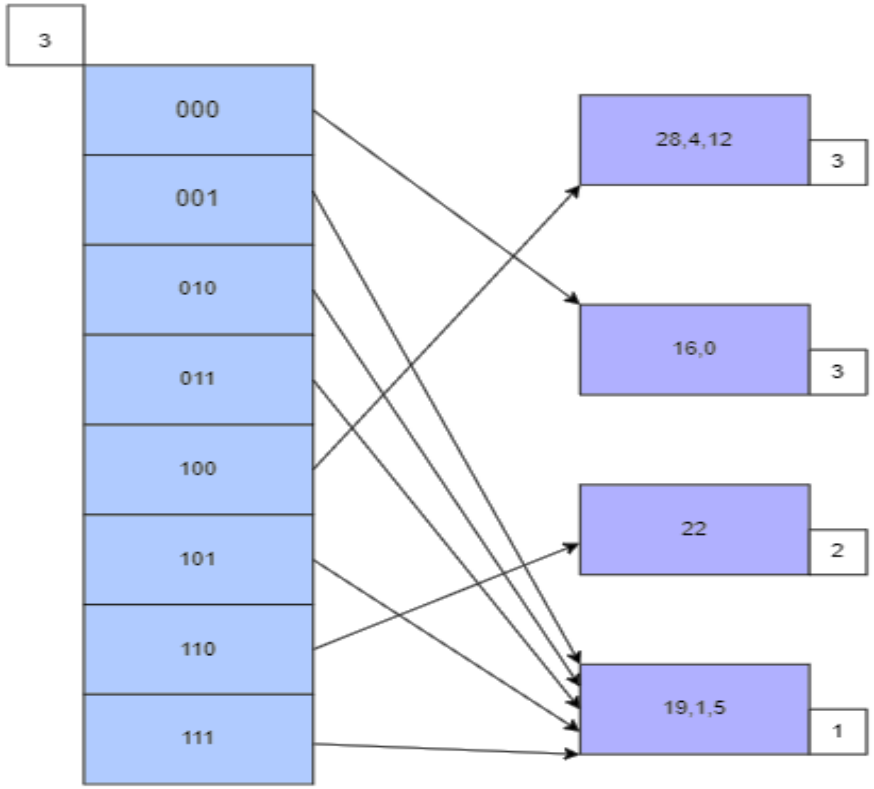
Consider 12 = 01100. Adding it to the "00" ID bucket according to two LSBs makes it overflow, and satisfies the first condition



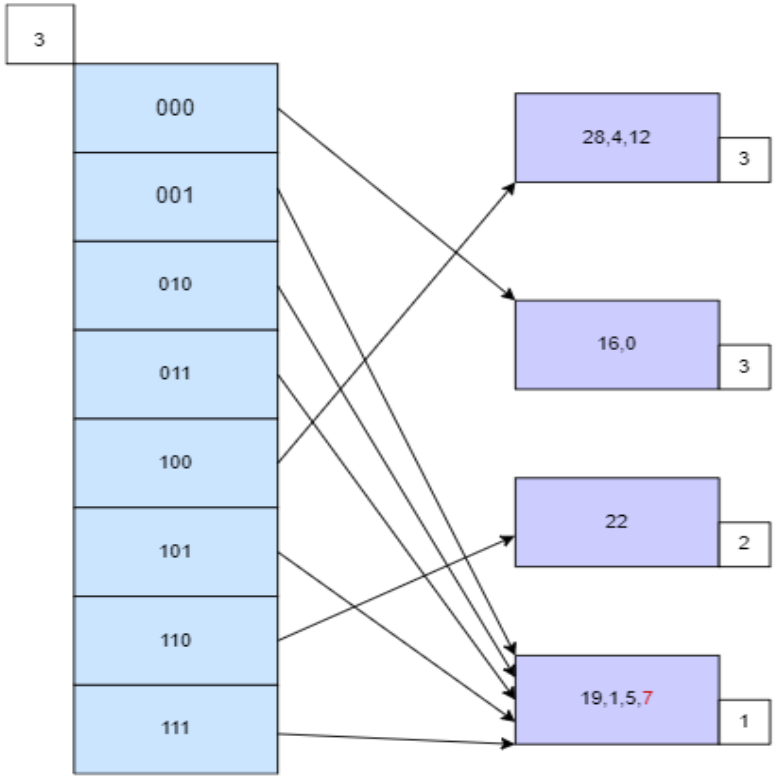
Repeat the previous step and re-hash the elements of the bucket according to three LSBs



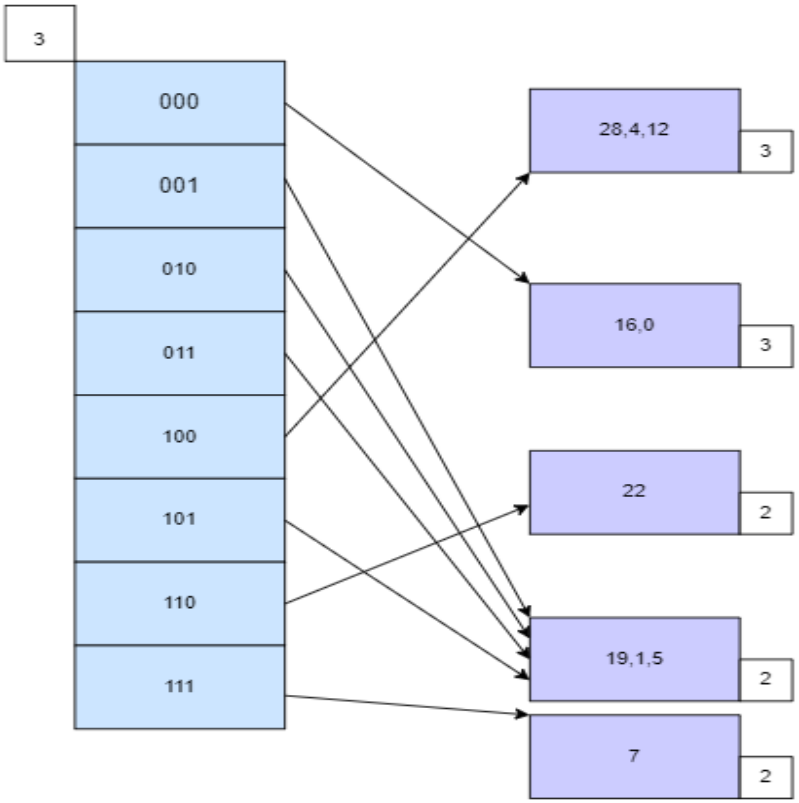
Consider 0 = 00000. Add it to the "000" ID bucket as three LSBs are considered



Consider 5 = 00101. Add it to the "101" bucket as three LSBs are considered



Consider 7 = 00111. Adding it to the "111" bucket according to three LSBs makes it overflow



The bucket depth is now less than global depth. So, the second condition is fulfilled, and now only the bucket split step is executed. Elements are re-hashed according to the global depth, which is still 3.

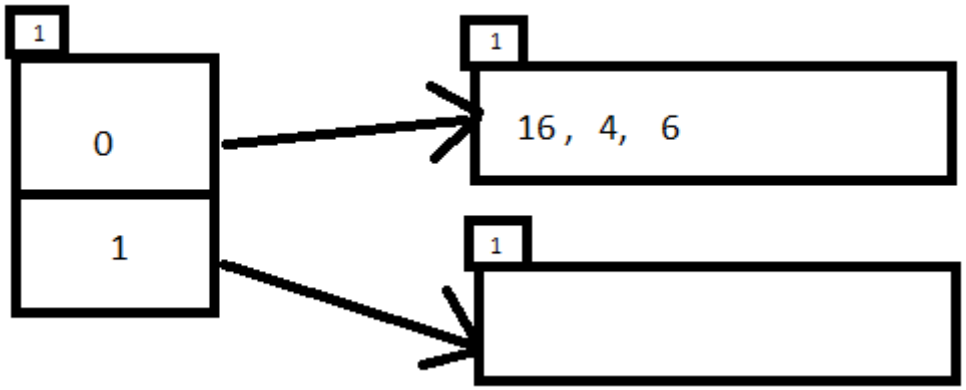
Assignment

- Write the solution for the below given data applying Extendible Hashing Technique.
- 16, 4,6, 22, 24,10, 31, 7, 9, 20, 26
- Bucket size assumed to be 3.

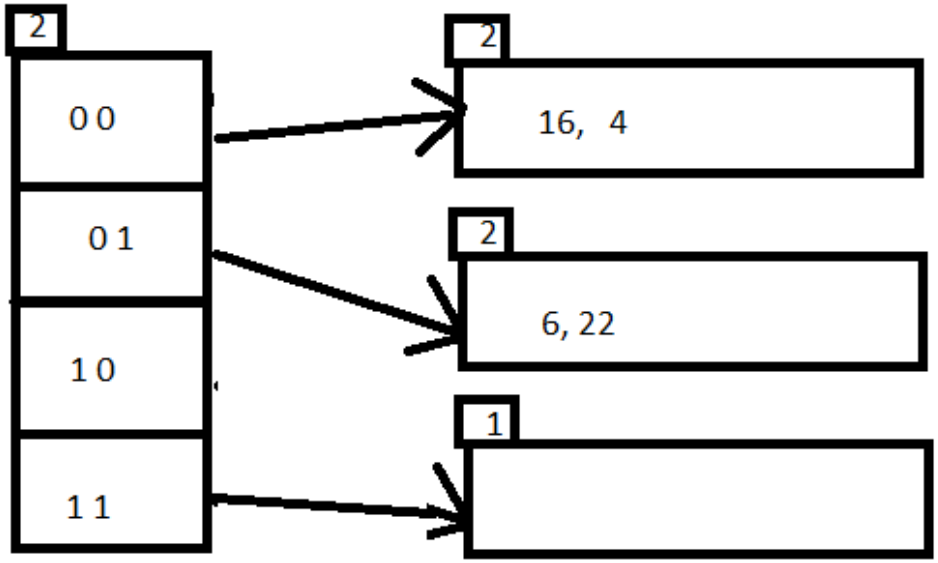
Solution

- First convert to binary format:
- 16: 100000
- 4: 00100
- 6: 00110
- 22:10110
- 24:11000
- 10:01010
- 31:11111
- 7:00111
- 9:01001
- 20:10100
- 26:11010

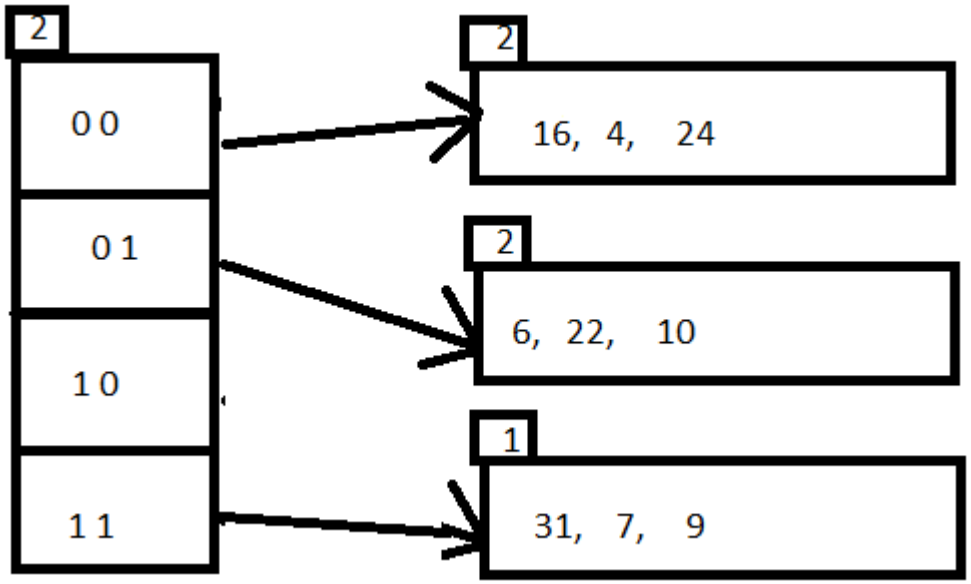
- Initially global depth = local depth = 1
- Insert 16 , 4 , 6



- Insert 22
- Overflow occurs since bucket is full.
- Here, local depth = global depth
- So, Do the Directory expansion & bucket splitting

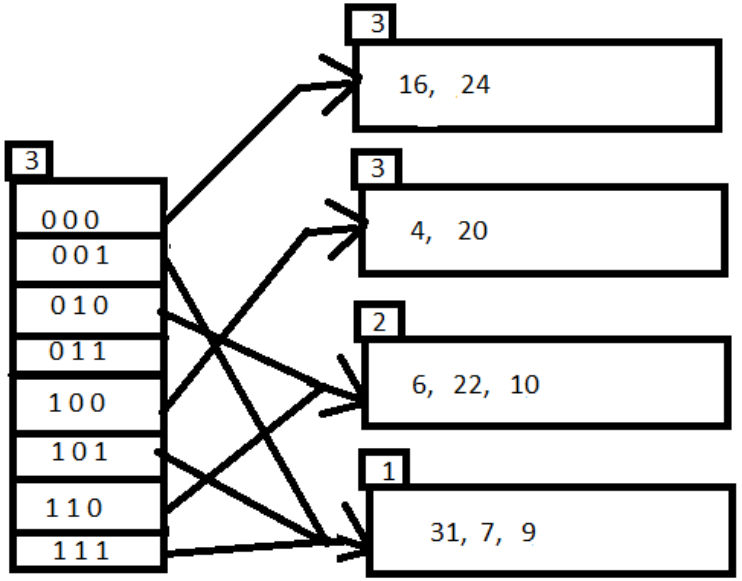


- Insert 24 and Insert 10

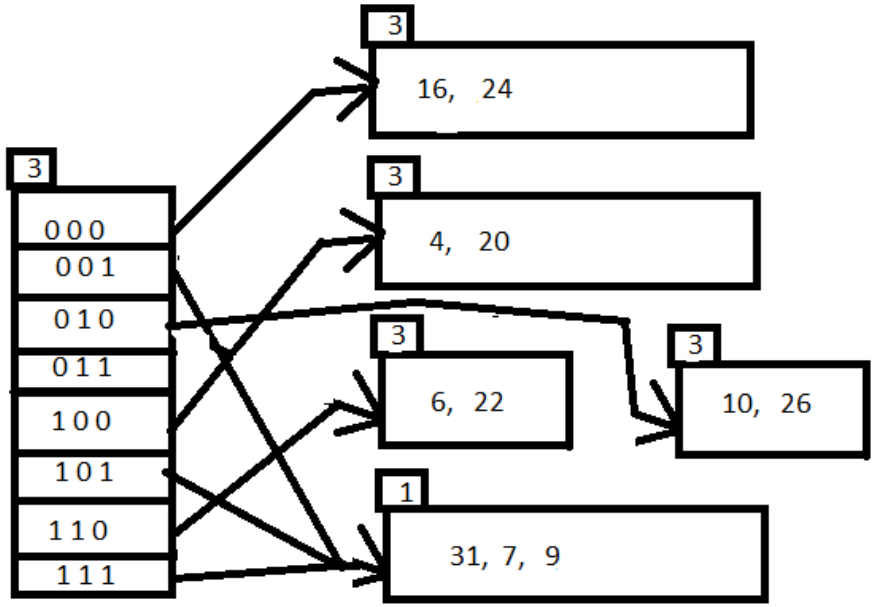


- Insert 31, 7 and 9

- Insert 20
- Overflow condition occurs
- Local depth = Global depth
- So, Directory expansion & bucket splitting



- Insert 26
- Overflow condition occurs
- Local depth < Global depth
- So, Only bucket splitting occurs



Questions

- implementation to find leaf count of a given Binary tree

```
int getLeafCount(Node node)
 {
    if (node == null)
        return 0;
    if (node.left == null && node.right == null)
        return 1;
    else
        return getLeafCount(node.left) +
 getLeafCount(node.right);
 }
```

- *counting the number of nodes in a Tree*

```
int countnodes(struct node *root)
{
if(root != NULL)
{
countnodes(root->left);
count++;
countnodes(root->right);
}
return count;
}
```

```
// Function to find k'th smallest element in BST
// Here i denotes the number of nodes processed so far
int kthSmallest(Node* root, int *i, int k)
{
    // base case
    if (root == nullptr)
        return INT_MAX;

    // search in left subtree
    int left = kthSmallest(root->left, i, k);

    // if k'th smallest is found in left subtree, return it
    if (left != INT_MAX)
        return left;

    // if current element is k'th smallest, return its value
    if (++*i == k)
        return root->data;

    // else search in right subtree
    return kthSmallest(root->right, i, k);
}

// Function to find k'th smallest element in BST
int kthSmallest(Node* root, int k)
{
    // maintain index to count number of nodes processed so far
    int i = 0;

    // traverse the tree in in-order fashion and return k'th element
    return kthSmallest(root, &i, k);
}
```

```
// Function to determine if given Binary Tree is a BST or not by keeping a
// valid range (starting from [MIN_VALUE, MAX_VALUE]) and keep shrinking
// it down for each node as we go down recursively
bool isBST(Node* node, int minKey, int maxKey)
{
   // base case
   if (node == NULL)
      return true;

   // if node's value fall outside valid range
   if (node->data < minKey || node->data > maxKey)
      return false;

   // recursively check left and right subtrees with updated range
   return isBST(node->left, minKey, node->data) &&
          isBST(node->right, node->data, maxKey);
}
```

```
int maxDepth(struct node* node)
{
  if (node==NULL)
     return 0;
  else
  {
     /* compute the depth of each subtree */
     int lDepth = maxDepth(node->left);
     int rDepth = maxDepth(node->right);

     /* use the larger one */
     if (lDepth > rDepth)
        return(lDepth+1);
     else return(rDepth+1);
  }
}
```

```
Input: inorder[] = {5, 10, 40, 30, 28}
Output: root of following tree
         40
       /   \
      10     30
     /         \
    5          28

Input: inorder[] = {1, 5, 10, 40, 30,
                    15, 28, 20}
Output: root of following tree
          40
        /   \
       10     30
      /         \
     5          28
    /          /  \
   1         15    20
```

For example, if the given traversal is {1, 7, 5, 50, 40, 10}, then following tree should be constructed and root of the tree should be returned.

```
     10
   /   \
  5     40
 /  \      \
1    7      50
```

